UNIVERSIDAD DE ALICANTE

Escuela Politécnica Superior ▪ Grado en Ingeniería Multimedia

SISTEMAS MULTIMEDIA

Apuntes completos de la asignatura

Programa oficial de contenidos — diez temas

Curso 2026–2027

Pedro Pernias

Ana Lavalle

Jaume Aragonés

Departamento de Lenguajes y Sistemas Informáticos

Universidad de Alicante

p.pernias@ua.es

Índice general

Introducción a la asignatura **7**
Enfoque y metodología . . . 7
Cómo están organizados estos apuntes . . . 7

1. El Círculo Dorado de Simon Sinek **9**
1.1. Sentido de la sesión . . . 9
1.2. El enigma de partida . . . 9
1.3. El Círculo Dorado . . . 10
1.4. De fuera hacia adentro frente a de adentro hacia afuera . . . 11
1.5. La biología del Círculo Dorado . . . 12
1.6. Tres casos para fijar la idea . . . 12
1.7. La Ley de Difusión de la Innovación . . . 13
1.8. Líderes y los que lideran . . . 14
1.9. Del concepto a tu proyecto: cómo ordenar tus ideas . . . 14
1.10. Aplicación al proyecto del curso . . . 15
1.11. Ideas clave para recordar . . . 15

2. Running Lean: el Lean Startup Canvas reducido **17**
2.1. Sentido del tema . . . 17
2.2. ¿Qué es una startup? . . . 18
2.3. Startup frente a empresa tradicional . . . 18
2.4. Cómo manejar la incertidumbre: modo cohete y modo coche . . . 19
2.5. El ciclo Crear–Medir–Aprender . . . 19
2.6. Crear un plan: el Lean Startup Canvas . . . 20
2.7. Las cinco casillas del Lean Startup Canvas reducido . . . 21
2.8. Idea no es lo mismo que demanda . . . 23
2.9. Por qué fracasan muchas startups . . . 23
2.10. Las tres fases y la priorización de riesgos . . . 24
2.11. Pivotar o perseverar . . . 25
2.12. Ejemplo trabajado: experiencias gastronómicas auténticas . . . 25
2.13. Actividades . . . 26
2.14. Aplicación al proyecto del curso . . . 27
2.15. Ideas clave para recordar . . . 27

3. Análisis DAFO y análisis de contingencias **29**
3.1. Sentido de la sesión . . . 29
3.2. El problema que resuelve el DAFO . . . 30
3.3. La pregunta primero: ¿de quién es el DAFO? . . . 31
3.4. La matriz: dos ejes, cuatro casillas . . . 32
3.5. Las cuatro casillas, una a una . . . 33
3.6. Ejemplos comentados . . . 35
3.7. El análisis cruzado: de las casillas a las estrategias . . . 36
3.8. Ejercicios propuestos (DAFO) . . . 37
3.9. Del diagnóstico al plan: el análisis de contingencias . . . 38
3.10. La tabla de contingencias . . . 39
3.11. Cómo se interpreta la tabla . . . 40
3.12. Ejemplo comentado de contingencias . . . 40
3.13. Ejercicios propuestos (contingencias) . . . 41
3.14. Aplicación al proyecto del curso . . . 42
3.15. Ideas clave para recordar . . . 42

4. Diseño de la interacción **45**
4.1. Sentido de la sesión . . . 45
4.2. El problema: ¿qué es la interactividad? . . . 46
4.3. El método: el diseño de la interacción . . . 48
4.4. Las fases del diseño de la interacción . . . 49
4.5. El storyboard: el puente hacia las especificaciones y el mockup . . . 51
4.6. La economía de la atención: el engagement como meta . . . 52
4.7. La segunda ola: la atención como moneda simbólica universal . . . 54
4.8. La IA reconfigura el mecanismo . . . 55
4.9. Aplicación al proyecto del curso . . . 56
4.10. Ideas clave para recordar . . . 56

5. Análisis y diseño de proyectos digitales **59**
5.1. Sentido de la sesión . . . 59
5.2. ¿Qué es un proyecto? . . . 60
5.3. Metodologías de gestión de proyectos . . . 61
5.4. La fase de análisis . . . 65
5.5. La fase de diseño . . . 67
5.6. Arquitectura del sistema . . . 68
5.7. Actividades . . . 73
5.8. Aplicación al proyecto del curso . . . 74
5.9. Ideas clave para recordar . . . 74

6. El diagrama de Gantt y cómo aplicarlo **77**
6.1. Sentido de la sesión . . . 77
6.2. Fundamentos: qué es y para qué sirve . . . 78
6.3. Un poco de historia . . . 79

6.4. ¿Por qué sigue vigente hoy? . . . 79
6.5. Elementos esenciales y cómo leer un Gantt . . . 80
6.6. Antes del diagrama: preparar la planificación . . . 81
6.7. Construcción paso a paso . . . 83
6.8. Herramientas y entornos de creación . . . 84
6.9. Seguimiento, control y gestión de cambios . . . 85
6.10. Ejemplo completo: desarrollo de un producto multimedia . . . 86
6.11. Aplicación al proyecto del curso . . . 88
6.12. Ideas clave para recordar . . . 88
6.13. Glosario básico . . . 89
6.14. Autoevaluación rápida . . . 89

7. La IA en la cadena de valor de Porter 91
7.1. Sentido de la sesión . . . 91
7.2. La cadena de valor de Porter . . . 92
7.3. El papel tradicional de las tecnologías de la información . . . 93
7.4. Cuando la IA repite el guion: la IA como apoyo . . . 93
7.5. Cuando la IA se sale del guion: la IA como núcleo . . . 94
7.6. Cómo distinguir lo accesorio de lo principal . . . 94
7.7. Cómo justificarlo en vuestro proyecto . . . 95
7.8. Aplicación al proyecto del curso . . . 96
7.9. Ideas clave para recordar . . . 96

8. Impacto social, ética y responsabilidad del ingeniero 99
8.1. Sentido de la sesión . . . 99
8.2. Propiedad intelectual: de quién es lo que usas . . . 100
8.3. Derecho a la privacidad: qué haces con los datos de las personas . . . 102
8.4. Ciberseguridad: proteger antes de publicar . . . 104
8.5. Ética y responsabilidad: el hilo que une los tres bloques . . . 106
8.6. Actividades . . . 107
8.7. Aplicación al proyecto del curso . . . 108
8.8. Ideas clave para recordar . . . 108

9. Plan de difusión digital: del producto al mercado 109
9.1. Sentido de la sesión . . . 109
9.2. Del canvas al mercado: reutilizar lo que ya sabéis . . . 110
9.3. El mensaje: qué dices . . . 111
9.4. Medios y canales digitales: dónde lo dices . . . 111
9.5. El embudo y las métricas: cómo sabes si funciona . . . 113
9.6. El plan como plan: calendario y recursos . . . 113
9.7. Comunicación responsable: el ciclo se cierra con ética . . . 114
9.8. Actividades . . . 114
9.9. Aplicación al proyecto del curso . . . 115
9.10. Ideas clave para recordar . . . 115

9.11. Glosario 116

10. Integración y validación: el proyecto final **119**

10.1. Sentido de la sesión 119
10.2. De los entregables al relato único: integrar 120
10.3. Validación empírica: ¿de verdad resuelve el problema? 122
10.4. De los hallazgos a las mejoras: iterar 123
10.5. La presentación y la defensa 124
10.6. Actividades 124
10.7. Aplicación al proyecto del curso 125
10.8. Ideas clave para recordar 126
10.9. Glosario 126

Introducción a la asignatura

Estos apuntes reúnen, en un único documento, los diez temas que componen el **programa oficial de contenidos** de *Sistemas Multimedia*, asignatura básica de segundo curso del Grado en Ingeniería Multimedia de la Universidad de Alicante. Cada tema corresponde a un capítulo y se ordena siguiendo la secuencia con la que se imparte durante el semestre.

La idea que vertebra el curso

El objetivo principal de la asignatura no es *terminar* técnicamente un proyecto, sino aprender a **plantearlo, estructurarlo, analizar su complejidad y detectar los conocimientos necesarios** para desarrollarlo. Es el marco global en el que encajan el resto de asignaturas de la titulación.

Enfoque y metodología

La asignatura se articula sobre el **Aprendizaje Basado en Proyectos (ABP)** y combina dos planos complementarios. El *plano teórico* —estos temas— constituye el compromiso de contenidos y el referente de evaluación. El *plano práctico* desarrolla ese programa mediante **diez entregables progresivos**: cada entregable produce un artefacto profesional que materializa uno o varios temas. Cada semana combina una sesión de teoría (marco conceptual del tema y presentación del entregable asociado) con una sesión de práctica (exposición del entregable anterior y desarrollo del nuevo).

La **Inteligencia Artificial** recorre el curso como eje transversal, distinguiendo siempre entre dos usos: la IA como *herramienta auxiliar* del equipo de trabajo y la IA como *parte de la propuesta de valor* del sistema multimedia que se diseña.

Cómo están organizados estos apuntes

Los diez temas se agrupan en tres bloques temáticos más un tema final de integración. La siguiente tabla resume esa estructura y el entregable con el que cada tema se materializa en la práctica.

Tema	Contenido	Entregable asociado
Bloque I — Problema y mercado		
1	El Círculo Dorado de Simon Sinek	E1 – Golden Circle
2	Running Lean (Lean Startup Canvas)	E2 – Lean Startup Canvas reducido
3	Análisis DAFO y contingencias	E3 – DAFO y contingencias
Bloque II — Ingeniería del producto		
4	Diseño de la interacción	E4 – Diseño de interacción
5	Análisis y diseño; especificación	E5 – Especificaciones
6	El diagrama de Gantt	E6 – Planificación GANTT
7	La IA en la cadena de valor de Porter	E7 – IA en la cadena de valor
Bloque III — Responsabilidad y viabilidad		
8	Impacto social, ética y responsabilidad	E8 – Aspectos legales
9	Plan de difusión digital	E9 – Plan de difusión
Integración		
10	Integración y validación empírica	E10 – Proyecto final

Cómo leer estos apuntes

Cada tema abre con un apartado de *Sentido de la sesión* y sus objetivos de aprendizaje. A lo largo del texto encontrarás cajas de color con un significado constante: **azul** para ideas clave, **verde** para ejemplos, **naranja** para advertencias, **rojo** para errores graves y **gris** para actividades de aula.

TEMA 1

El Círculo Dorado de Simon Sinek

1.1. Sentido de la sesión

Antes de aprender a **construir** un producto, conviene aprender a **pensarlo**. La mayoría de los proyectos que fracasan no lo hacen por un fallo técnico, sino porque nadie supo explicar **por qué deberían existir**. Saben fabricar, programar y diseñar; lo que no saben es para qué.

En esta sesión vamos a darle la vuelta a ese orden. Aprenderemos un modelo muy sencillo —el **Círculo Dorado**— que cambia la pregunta de partida de cualquier proyecto. En lugar de empezar por *¿qué vamos a hacer?*, empezaremos por *¿por qué lo hacemos?*

Idea central

Las personas no compran lo que haces; compran **por qué** lo haces.

1.1.1. Qué deberías saber hacer al terminar

- **Distinguir** los tres niveles del Círculo Dorado: *qué*, *cómo* y *por qué*.
- **Explicar** por qué las organizaciones que inspiran comunican de dentro hacia afuera, y no al revés.
- **Analizar** una empresa o producto real e identificar su *qué*, su *cómo* y su *por qué*.
- **Aplicar** este orden de pensamiento a tus propios desarrollos a lo largo del curso.

1.2. El enigma de partida

Empecemos con tres hechos que, a primera vista, no encajan.

- **Apple** aparece año tras año entre las empresas más innovadoras del mundo. Y sin embargo es, en apariencia, “solo una empresa de ordenadores”. Tiene acceso al mismo

talento, a las mismas agencias, a los mismos proveedores y a los mismos medios que su competencia. ¿Por qué se diferencia de forma tan constante?

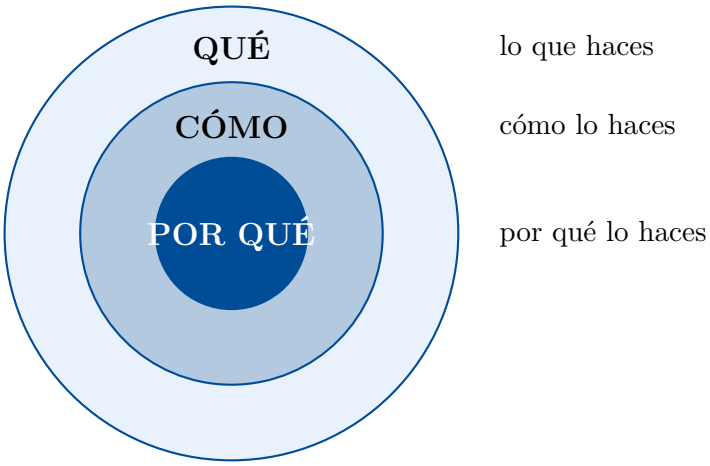
- **Martin Luther King Jr.** lideró el movimiento por los derechos civiles en Estados Unidos. No era el único que sufría discriminación, ni el único gran orador de su tiempo. ¿Por qué él?

- Los **hermanos Wright** consiguieron el primer vuelo motorizado teniendo enfrente a equipos mejor financiados, mejor formados y más reconocidos. ¿Cómo lo lograron?

Simon Sinek observó que todos los grandes líderes y las organizaciones que inspiran **piensan, actúan y se comunican igual**. Y, además, lo hacen **justo al revés** que la mayoría. A ese patrón lo llamó el **Círculo Dorado**.

1.3. El Círculo Dorado

Imaginemos tres círculos concéntricos, como una diana. De fuera hacia dentro:



1.3.1. El *qué* (What)

Es **lo que una persona u organización hace**: los productos que fabrica o los servicios que presta. Es el nivel más visible y tangible. El **100 %** de las organizaciones sabe *qué* hace: fabricar ordenadores, vender seguros, impartir formación... todo el mundo lo tiene claro.

1.3.2. El *cómo* (How)

Es **la manera** en que se hace ese *qué*: una propuesta de valor, un proceso propio, una metodología diferenciadora, una ventaja competitiva. Solo **algunas** organizaciones saben explicarlo bien. Suele ser lo que se presenta como "lo que nos hace especiales".

1.3.3. El *por qué* (Why)

Es el nivel más profundo y el más difícil. **Muy pocas** personas y organizaciones saben explicar *por qué* hacen lo que hacen.

El porqué no es ganar dinero

El dinero es un **resultado**, no un propósito. El *porqué* es la **causa, la creencia o el propósito** que justifica la existencia de la organización. La pregunta central no es solo "¿qué hace esta organización?", sino "¿por qué existe y por qué debería importarle a alguien?".

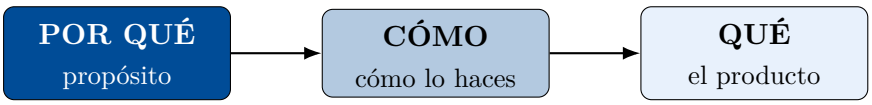
1.4. De fuera hacia adentro frente a de adentro hacia afuera

1.4.1. La forma habitual (de fuera hacia adentro)

La mayoría comunica empezando por lo más claro y terminando por lo más difuso: **Qué** → **Cómo** → **(a veces) Por qué**. Es lo cómodo, pero **no mueve a la acción**.

1.4.2. La forma que inspira (de adentro hacia afuera)

Los líderes y las organizaciones que inspiran invierten el orden:



Empiezan por el propósito, continúan con la forma de llevarlo a cabo y terminan en el producto concreto.

Ejemplo: cómo comunica Apple

Si Apple comunicara como la mayoría, diría algo así:

> "Hacemos ordenadores geniales. Están muy bien diseñados, son sencillos y fáciles de usar. ¿Quieres comprar uno?"

Empieza por el *qué* y espera una compra. Suena correcto... y olvidable. Pero Apple lo cuenta al revés:

> "En todo lo que hacemos, creemos en desafiar el *status quo*. Creemos en pensar diferente. La forma en que lo hacemos es creando productos bellamente diseñados, sencillos y fáciles de usar. Y resulta que hacemos ordenadores geniales. ¿Quieres comprar uno?"

Mismo producto, casi las mismas palabras... y sin embargo el segundo mensaje **convence**.

Por eso mucha gente compra de Apple no solo ordenadores, sino también móviles, auriculares o relojes, mientras que no compraría esos mismos productos a otra marca técnicamente igual de capaz.

Conclusión del bloque

El objetivo no es venderle a todo el que necesita lo que ofreces. El objetivo es **conectar con quienes creen en lo mismo que tú**.

1.5. La biología del Círculo Dorado

Esto no es marketing ni psicología; según Sinek, está **en cómo está construido nuestro cerebro**. Si hacemos un corte transversal, los niveles del Círculo Dorado encajan con dos zonas:

- **Neocórtex → el *qué*.** Es la parte racional y analítica, la del lenguaje. Procesa datos, características, beneficios e información compleja. Entiende, pero no decide.
- **Cerebro límbico → el *cómo* y el *por qué*.** Gestiona los sentimientos (confianza, lealtad) y la **toma de decisiones**. El problema: **no tiene capacidad para el lenguaje**.

Esto explica algo que todos hemos vivido: tomamos decisiones de forma intuitiva y *después* las justificamos con argumentos. Cuando algo nos encaja por dentro pero no sabemos explicar por qué, solo acertamos a decir: "No sé... simplemente no me convence".

Por qué importa al comunicar

Cuando comunicas de fuera hacia adentro, la gente entiende mucha información, pero esa información **no la mueve a actuar**. Cuando comunicas de adentro hacia afuera, hablas directamente con la parte del cerebro que decide; las características técnicas vienen *después*, para racionalizar una decisión ya tomada.

1.6. Tres casos para fijar la idea

1.6.1. Los hermanos Wright contra Samuel Pierpont Langley

A comienzos del siglo XX, **Langley** parecía tenerlo todo: 50.000 dólares del Departamento de Guerra, un puesto en Harvard, acceso a las mejores mentes y la cobertura del *New York Times*. Pero su motivación era el **resultado**: quería ser rico y famoso.

En Dayton (Ohio), **Orville y Wilbur Wright** trabajaban sin apenas recursos, financiando el sueño con su tienda de bicicletas, sin un solo titulado universitario en el equipo. Lo que tenían era una **causa**: creían que, si lograban volar, cambiarían el mundo. El **17 de diciembre de 1903** volaron. Y aquí está la prueba del porqué: cuando Langley se

enteró del éxito de los Wright, **abandonó el proyecto**. Ya no podía ser el primero ni famoso. Su motivación nunca fue volar.

Lección

Un equipo movido por una creencia trabaja con un compromiso que el dinero no compra.

1.6.2. TiVo: el buen producto que comunicó mal

TiVo tenía un gran producto y buena financiación. Pero comunicó desde el *qué*:

> "Tenemos un producto que pausa la televisión en directo, salta los anuncios y memoriza tus hábitos."

Funciones impecables... que no conectan con ninguna creencia.

Por qué fracasó el mensaje

Una lista de prestaciones habla al neocórtex, pero no a la parte del cerebro que decide. El mensaje habría sido mucho más potente desde el *porqué*: *"Si eres de esas personas a las que les gusta tener el control total de cada aspecto de su vida, tenemos un producto para ti."* Así el producto es la **consecuencia** de una creencia compartida, no un catálogo de funciones.

1.6.3. Martin Luther King: "Tengo un sueño", no "Tengo un plan"

Ejemplo: el liderazgo de King

En el verano de 1963, unas **250.000 personas** se reunieron en Washington para escucharle. Sin invitaciones formales, sin webs donde consultar la fecha. King no recorría el país explicando *qué* había que cambiar: comunicaba **lo que creía** ("Yo creo..."). Quienes compartían esa creencia hicieron suya la causa y la difundieron. Las 250.000 personas no fueron solo por él; **fueron por ellas mismas y por aquello en lo que creían**. Por eso su discurso se titula "I have a dream" y no "I have a plan".

1.7. La Ley de Difusión de la Innovación

Tener un buen *porqué* no basta si no entiendes **cómo se adopta** algo nuevo. La población se reparte así:

Grupo	Porcentaje
Innovadores	2,5 %
Primeros adoptantes	13,5 %
Mayoría temprana	34 %
Mayoría tardía	34 %
Rezagados	16 %

El punto de inflexión

No se alcanza la **aceptación masiva** hasta superar un punto de inflexión situado en torno al **15–18 %** del mercado. La *mayoría temprana* no prueba nada hasta que otros lo han probado antes; los *innovadores* y *primeros adoptantes* deciden de forma intuitiva, movidos por lo que creen sobre el mundo.

¿Recordáis las colas de horas para comprar el primer iPhone? Esa gente no hacía cola solo por la tecnología, sino porque la compra **decía algo sobre quiénes eran y cómo querían verse**. Si quieres llegar a la mayoría, primero tienes que **ganarte a quienes creen lo que tú crees**.

1.8. Líderes y los que lideran

Sinek cierra con una distinción:

- **Los líderes** ocupan una posición de poder o autoridad.
- **Los que lideran** son quienes **inspiran**.

Seguimos a quienes lideran no porque *tengamos* que hacerlo, sino porque **queremos**. No los seguimos por ellos, sino por nosotros mismos y por aquello en lo que creemos.

Síntesis

Empezar por el *porqué* es lo que da a una persona o a una organización la capacidad de inspirar.

1.9. Del concepto a tu proyecto: cómo ordenar tus ideas

A partir de hoy, **cuando diseñéis cualquier producto invertiréis el orden de pensamiento**:

1. **Empezad por el *PORQUÉ*.** ¿Qué creéis? ¿Qué problema os indigna o emociona? ¿Por qué el mundo estaría un poco mejor si vuestro proyecto existiera?

2. **Seguid con el *CÓMO*.** ¿Qué hacéis de forma distinta para honrar ese porqué? Vuestro proceso, vuestros principios, vuestra manera particular de resolverlo.

3. **Terminad con el *QUÉ*.** Ahora sí: el producto, la app, el servicio concreto. El *qué* es la **prueba tangible** de vuestra creencia.

El error más común: el falso porqué

Si vuestra única respuesta a "¿por qué?" es **ganar dinero** o **aprobar la asignatura**, todavía no tenéis un porqué: tenéis un resultado. El porqué es una creencia que existiría aunque no hubiera ni nota ni beneficio. No confundáis el motor con la recompensa.

Mini-actividad: el test de Apple

Escribid vuestro proyecto como un único párrafo siguiendo la estructura del segundo mensaje de Apple:

> "Creemos en... La forma en que lo hacemos es... Y por eso hacemos..."

Si al leerlo en voz alta suena hueco o genérico, vuestro *porqué* aún no está claro. Reescribidlo hasta que alguien ajeno al grupo entienda en qué creéis.

Puente al Entregable 1

La habilidad que entrenáis aquí —descubrir el porqué antes que el qué— es exactamente la que pondréis en práctica en el primer entregable, analizando una empresa real. En los entregables siguientes la aplicaréis a vuestros propios proyectos.

1.10. Aplicación al proyecto del curso

El Círculo Dorado se aplica al curso en dos momentos: primero lo entrenaréis sobre una empresa real y después lo usaréis para formular el *porqué* de vuestro propio producto, que será la raíz de la que cuelgue todo lo demás. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

1.11. Ideas clave para recordar

- El Círculo Dorado tiene tres niveles: **qué**, **cómo** y **por qué**.
- Casi todos saben el *qué*; muy pocos saben el *por qué*.
- Las personas no compran lo que haces, sino **por qué** lo haces.
- Quienes inspiran comunican de adentro hacia afuera: por qué → cómo → qué.

- Tiene base biológica: el cerebro límbico decide pero no habla; el neocórtex razona pero no mueve a actuar.
- Para la adopción masiva hay que superar el **15–18 %** convenciendo primero a quienes creen lo mismo que tú.
- Diseñar bien empieza por preguntarse **por qué** debería existir tu producto.

Apuntes elaborados a partir de la transcripción de la charla TED de Simon Sinek (2009), "How great leaders inspire action".

TEMA 2

Running Lean: el Lean Startup Canvas reducido

2.1. Sentido del tema

Crear un producto innovador no se parece a gestionar una empresa ya conocida. En una empresa tradicional muchas variables están claras: quién es el cliente, cómo se vende, cuánto cuesta operar o qué propuesta de valor funciona. En una **startup** casi todo eso está por descubrir. Por eso una startup no puede gestionarse como una empresa pequeña convencional.

El enfoque *Lean Startup*, y su aplicación práctica en *Running Lean* de Ash Maurya, parte de esa idea: emprender es trabajar bajo incertidumbre. La consecuencia es un cambio de objetivo. No se trata de *planificar mejor* un producto cerrado, sino de **aprender más rápido** qué producto, para qué cliente y con qué modelo de negocio merece la pena construir.

Idea central

Una startup trabaja bajo **incertidumbre extrema**. Su tarea no es ejecutar a la perfección un plan cerrado, sino **aprender lo antes posible** qué producto, para qué cliente y con qué modelo de negocio tiene sentido construir.

2.1.1. Qué deberías saber hacer al terminar

- **Definir** qué es una startup y por qué se distingue de una empresa tradicional.
- **Explicar** el ciclo Crear–Medir–Aprender y la metáfora del modo coche.
- **Construir** un Lean Startup Canvas reducido como un conjunto de hipótesis.
- **Definir** un segmento acotado, distinguir cliente de usuario e identificar a los *early adopters*.

- **Relacionar** los tres problemas principales con una solución mínima y una propuesta de valor única.
- **Justificar** una ventaja especial y seleccionar el riesgo clave que debe validarse primero.

2.2. ¿Qué es una startup?

Definición

Una startup es una **institución humana** diseñada para crear un **nuevo** producto o servicio bajo condiciones de **incertidumbre extrema**.

La definición esconde tres ideas que conviene desplegar.

Es una institución humana. Una startup no es solo una app, una web, una idea o un algoritmo: es ante todo un grupo de personas que organiza el trabajo, toma decisiones, formula hipótesis, prueba supuestos y aprende del mercado. El foco no está únicamente en el producto técnico, sino en *cómo el equipo averigua qué debe construir*.

Está diseñada para crear algo nuevo. La novedad puede estar en el problema, en el segmento de cliente, en la forma de entregar valor, en la monetización o en el modelo completo. No hace falta inventar una tecnología radicalmente nueva; basta con que el modelo todavía no esté validado.

Opera bajo incertidumbre extrema. Al principio la startup no sabe con certeza quién es su cliente, qué problema merece la pena resolver, si ese problema duele lo suficiente, si el cliente pagaría, qué solución aceptará ni cómo convertir todo eso en un negocio sostenible. Por eso no parte de certezas, sino de **hipótesis**.

2.3. Startup frente a empresa tradicional

Una startup no es simplemente una empresa pequeña. La diferencia no es el tamaño, sino el **grado de incertidumbre**.

	Empresa tradicional	**Startup**
Cliente	Conocido	Aún sin definir del todo
Problema	Validado	Posiblemente sin validar
Solución	Estable	Provisional
Monetización	Conocida	Incierta
Enfoque	Optimizar lo conocido	Descubrir y validar

Idea clave

La **empresa tradicional optimiza** un modelo de negocio que ya conoce. La **startup descubre y valida** un modelo todavía incierto. Por eso la planificación clásica encaja con la primera, pero no con la segunda.

Ejemplo comparativo

Caso A — cafetería de barrio. Se sabe bastante bien quién es el cliente, qué se vende, cómo se ingresa dinero y qué operaciones hacen falta. Hay incertidumbre, pero no extrema.

Caso B — plataforma nueva para una necesidad no validada. No está claro quién la usará, si el problema importa, si la solución encaja ni si el modelo funcionará. Este segundo caso encaja con la definición de startup.

2.4. Cómo manejar la incertidumbre: modo cohete y modo coche

Para decidir cómo actuar bajo incertidumbre, Lean Startup usa una metáfora.

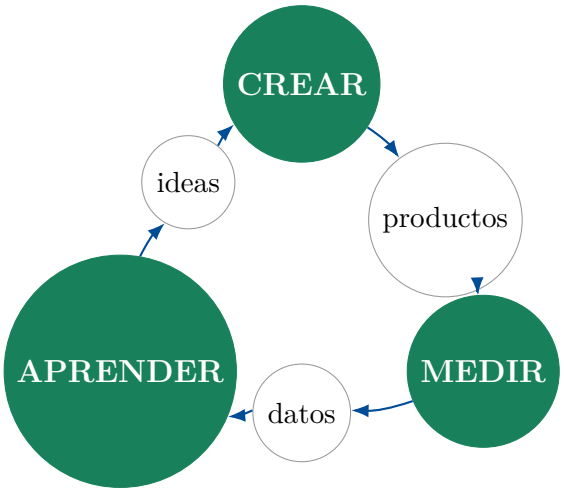
Modo cohete
Todo programado antes del lanzamiento. Poca maniobrabilidad: corregir el rumbo es difícil. Útil si el entorno es *predecible*.

Modo coche
Nada está fijado del todo: se avanza observando el entorno y se corrige de continuo. Alta maniobrabilidad. Útil si el entorno es *incierto*.

El modo coche no significa improvisar sin criterio, sino ajustar según la realidad. Una startup no se *lanza* sin más: se **conduce** en un entorno incierto. Por eso su actividad central se resume en crear ideas y convertirlas en productos, medir cómo responden los consumidores y aprender cuándo **pivotar o perseverar**.

2.5. El ciclo Crear–Medir–Aprender

Si una startup vive bajo incertidumbre, su actividad principal no puede ser solo desarrollar producto: debe seguir un ciclo de aprendizaje que conecte las ideas con los datos.



La clave no es recorrer el ciclo una vez, sino **minimizar el tiempo de cada vuelta**: cuanto antes se cierra el bucle, antes se aprende.

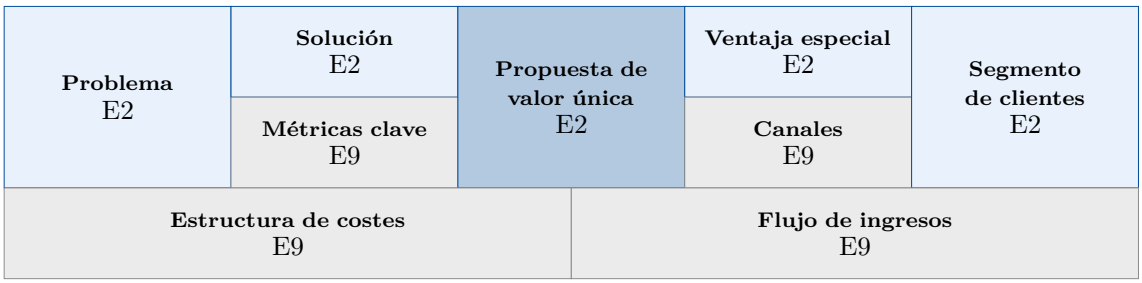
El producto mínimo viable (MVP)

Para acelerar el ciclo se construye un **producto mínimo viable** (MVP): la versión más sencilla que permite crear, medir y aprender. Se elimina todo lo que no contribuya a ese bucle. Su objetivo no es impresionar, sino *generar aprendizaje validado* cuanto antes.

2.6. Crear un plan: el Lean Startup Canvas

Lean no reniega del plan; reniega de tratar el plan como una verdad. La propuesta es crear y documentar un **Plan A**, identificar los riesgos y ponerlo a prueba sistemáticamente. Ese primer plan no contiene hechos demostrados: contiene **hipótesis** que el equipo deberá justificar y, más adelante, contrastar con la realidad.

La herramienta para capturarlo en una sola página es el **Lean Startup Canvas**. El lienzo completo tiene nueve casillas, pero en este tema trabajaremos deliberadamente con una versión reducida centrada en el producto y en el ajuste problema–solución.



Alcance de esta primera versión

La versión reducida se concentra en cinco casillas: **segmento de clientes, problema, solución, propuesta de valor única y ventaja especial**. Las casillas de **métricas clave, canales, estructura de costes y flujo de ingresos** se estudiarán más adelante, cuando el proyecto aborde su salida al mercado.

Idea decisiva: cada casilla es una hipótesis

Una frase escrita en el lienzo no se convierte en verdad por estar bien redactada. "Nuestro cliente es X", "su principal problema es Y" o "valorará Z" son hipótesis. El valor del Lean Startup Canvas no está en rellenar casillas, sino en hacer visibles los supuestos que sostienen el producto para poder razonar sobre ellos y decidir cuál comprobar primero.

2.7. Las cinco casillas del Lean Startup Canvas reducido

Las cinco casillas forman una cadena. El **segmento** determina de quién hablamos; el **problema**, qué necesidad merece atención; la **solución**, qué mínimo proponemos para afrontarla; la **propuesta de valor**, por qué esa solución importa; y la **ventaja especial**, por qué nuestra alternativa resulta preferible o difícil de copiar. Si una pieza no encaja con la anterior, el lienzo no es coherente.

2.7.1. 1. Segmento de clientes

El segmento no puede ser "todo el mundo", "los jóvenes" o "las empresas". Debe describir un grupo suficientemente concreto cuyos miembros compartan una necesidad y un contexto reconocibles.

Conviene distinguir tres conceptos:

- **Usuario:** la persona que utiliza el producto.
- **Cliente:** la persona u organización que paga o contrata. Puede coincidir con el usuario, pero no siempre.
- **Early adopter:** integrante del segmento que siente el problema con especial intensidad, ya intenta resolverlo y está dispuesto a probar una alternativa nueva antes que la mayoría.

De un segmento vago a uno útil

Demasiado amplio: "turistas".
Mejor acotado: parejas y grupos pequeños de turistas del norte de Europa, de 35 a 50 años, que visitan Alicante durante un fin de semana y reservan actividades con el móvil.
Early adopters: quienes ya buscan experiencias locales auténticas y evitan deliberadamente las ofertas turísticas masificadas.

2.7.2. 2. Problema

Un problema útil describe una dificultad del segmento, no una función que queremos construir. "No existe nuestra app" no es un problema; "me cuesta distinguir una experiencia local auténtica de una trampa para turistas" sí lo es.

En esta versión reducida se identifican los **tres problemas principales**, ordenados por importancia, y las **alternativas actuales**: lo que las personas hacen hoy para resolverlos, aunque la solución sea incompleta. Las alternativas pueden ser productos competidores, procesos manuales, búsquedas, recomendaciones o incluso no hacer nada.

Comprobación de coherencia

Cada problema debe pertenecer al segmento elegido y poder relacionarse después con una característica de la solución. Si el equipo no puede explicar quién sufre el problema, cuándo aparece y qué hace hoy para afrontarlo, todavía está formulado de manera demasiado genérica.

2.7.3. 3. Solución

La solución de este lienzo reducido no es una especificación técnica completa. Es el bosquejo de la versión más sencilla capaz de responder a los problemas priorizados. Puede resumirse en **tres características clave**, evitando listas largas de funciones.

Una forma sencilla de comprobarla consiste en construir esta correspondencia:

Problema priorizado	**Característica mínima que responde**
No distingue experiencias auténticas	Catálogo reducido y verificado presencialmente
Barrera de idioma	Información y atención en el idioma del visitante
Proceso de reserva incierto	Reserva inmediata desde el móvil

La tecnología concreta puede cambiar. Lo que debe mantenerse es la relación lógica entre problema y solución.

2.7.4. 4. Propuesta de valor única

La propuesta de valor única expresa, en una frase clara, **para quién** existe el producto, **qué beneficio relevante** ofrece y **por qué resulta diferente** de las alternativas. No es un eslogan vacío ni una enumeración de características.

Como ayuda, puede utilizarse esta estructura sin convertirla en una fórmula rígida:

Para [segmento], que necesita [resolver un problema], ofrecemos [beneficio principal] de una forma [diferencia relevante].

Propuesta de valor

"Vive Alicante como un local: experiencias gastronómicas auténticas, verificadas y en tu idioma." La frase conecta el segmento y su problema con un beneficio, sin describir toda la aplicación.

2.7.5. 5. Ventaja especial

La ventaja especial —también llamada ventaja injusta— es aquello que hace que el proyecto sea **difícil de copiar**: conocimiento propio, acceso privilegiado, comunidad, acuerdos, datos, experiencia del equipo o una combinación singular de recursos. "Usamos

IA", "tenemos una app bonita" o "seremos los mejores" no son ventajas defendibles si cualquiera puede reproducirlas.

Es normal que un proyecto inicial todavía no posea una ventaja injusta consolidada. En ese caso puede señalarse la **ventaja más valorable de la aplicación**: la característica que el segmento apreciará más y que mejor la diferencia de las alternativas. Hay que presentarla con honestidad y justificar la elección.

Ventaja especial o ventaja más valorable

Una red de productores y guías locales con acuerdos de exclusividad y verificación presencial sería difícil de replicar a corto plazo. Si esos acuerdos aún no existen, la ventaja más valorable podría ser la curación experta: todas las experiencias han sido probadas y verificadas, frente a un listado abierto de opiniones.

2.8. Idea no es lo mismo que demanda

Tener una idea no demuestra que exista mercado. El error habitual es razonar "tengo una idea, así que si la construyo aparecerán los clientes". Lean exige distinguir entre la **idea** y la **demanda real**.

Señales engañosas

No validan demanda frases como "suena interesante", "mis amigos dicen que la usarían", "es innovadora" o "tiene muchas funcionalidades". Las preguntas que sí importan son: ¿tienes realmente este problema?, ¿te importa lo suficiente?, ¿lo resolverías?, ¿pagarías por ello?

Ejemplo: la botella inteligente

Una botella con sensores y app puede ser una idea llamativa. Pero si el cliente no siente un problema importante con su botella actual, hay *idea*, pero no *demanda*.

2.9. Por qué fracasan muchas startups

Un equipo puede trabajar meses, programar mucho y lanzar una solución técnicamente impecable; pero si no resuelve un problema importante para un cliente real, no habrá tracción.

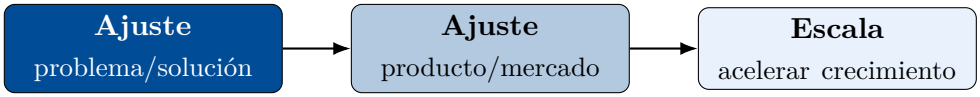
El error más común: construir algo que nadie quiere

La causa más repetida de fracaso es **construir productos que nadie quiere**. Detrás suele estar el *enamorarse de la solución*: comprometerse con el producto inicial en lugar de comprometerse con el **aprendizaje validado**. Si no hay evidencia de que el problema existe, importa y alguien pagaría, se está invirtiendo tiempo en

algo que puede ser irrelevante.

2.10. Las tres fases y la priorización de riesgos

El desarrollo de una startup atraviesa tres fases. La regla general es **pivotar antes (ajuste) y optimizar después (escala)**.



No todas las hipótesis son igual de peligrosas. Algunas, si resultan falsas, invalidan casi todo el proyecto; otras son detalles (el color de la interfaz, el nombre del producto). El primer trabajo es ordenar los riesgos.

Valida primero el riesgo clave

Si una hipótesis falsa destruye el proyecto, debe validarse antes. Los riesgos iniciales se concentran en el ajuste problema/solución: ¿existe el problema?, ¿duele lo suficiente?, ¿el cliente quiere resolverlo?, ¿pagaría por ello?

2.10.1. Del riesgo clave a una comprobación inicial

En esta primera aproximación no es necesario ejecutar todavía una validación empírica completa, pero resulta útil formular un **plan de comprobación concreto y viable**. No basta con escribir "haremos una encuesta". La técnica debe permitir aprender precisamente sobre la hipótesis seleccionada.

Hipótesis crítica	**Comprobación inicial posible**	**Qué observar**
El problema existe y es frecuente	Entrevistas breves al segmento sobre experiencias pasadas	Casos reales, frecuencia y gravedad
La propuesta resulta comprensible	Mostrar dos formulaciones y pedir que expliquen qué entienden	Comprensión sin ayuda y preferencia razonada
La solución mínima aporta valor	Prototipo simple o demostración del flujo principal	Capacidad para completar la tarea e interés real
La alternativa actual es insuficiente	Preguntar cómo resolvieron el problema la última vez	Esfuerzo, frustraciones y soluciones utilizadas

Una buena formulación tiene tres partes: **hipótesis**, **método** y **señal que permitiría decidir**. Por ejemplo: "Creemos que los turistas del segmento desconfían de las recomendaciones abiertas. Entrevistaríamos a cinco personas que hayan visitado Alicante

recientemente y les pediríamos que reconstruyeran cómo eligieron una actividad. Consideraríamos apoyada la hipótesis si al menos tres describen problemas concretos de confianza o verificación." Las cifras son una decisión inicial del equipo, no una prueba científica definitiva.

Qué no constituye una validación útil

Preguntar a amigos "¿te gusta nuestra idea?", recopilar únicamente opiniones favorables o formular preguntas que sugieren la respuesta solo produce confirmación. Conviene preguntar por comportamientos pasados y ejemplos reales, no por promesas vagas sobre lo que alguien dice que haría.

2.11. Pivotar o perseverar

Cuando los datos contradicen una hipótesis importante, hay que decidir entre **perseverar** (seguir el plan) o **pivotar** (cambiar de rumbo manteniendo la visión). Un pivote no es un fracaso: es el aprendizaje buscado. Existen muchos tipos (acercamiento o *zoom-in*, alejamiento o *zoom-out*, de segmento de cliente, de necesidad, de plataforma, de captura de valor, de canal, de tecnología...).

El crecimiento se sostiene en un **motor de crecimiento**; normalmente se usa uno solo:

- **Pegajoso (sticky):** retiene al consumidor. Crece si la tasa de abandono es menor que la de incorporación.
- **Viral:** un usuario atrae a otros. Se mide con el coeficiente viral (usuarios nuevos por usuario existente).
- **Remunerado (pagado):** cada usuario tiene un coste de adquisición (CPA); crece si el valor del cliente supera ese coste.

2.12. Ejemplo trabajado: experiencias gastronómicas auténticas

Este ejemplo muestra cómo las cinco casillas se sostienen unas a otras.

Lean Startup Canvas reducido

Segmento de clientes: parejas y grupos pequeños de turistas del norte de Europa, de 35 a 50 años, que pasan un fin de semana en Alicante y reservan desde el móvil. *Early adopters*: quienes ya buscan experiencias locales y evitan ofertas masificadas.

Problemas: (1) dificultad para distinguir experiencias auténticas; (2) barrera de idioma; (3) desconfianza ante recomendaciones y ofertas turísticas. *Alternativas actuales*: TripAdvisor, blogs, redes sociales y recomendaciones del hotel.

Solución: catálogo pequeño de experiencias verificadas; reserva inmediata desde el móvil; anfitriones locales con atención en varios idiomas.

Propuesta de valor única: "Vive Alicante como un local: experiencias gastronómicas auténticas, verificadas y en tu idioma."

Ventaja especial: red local verificada y acuerdos con productores y guías. Si esos acuerdos todavía no están consolidados, la ventaja más valorable es la curación experta frente a los listados abiertos.

Riesgo clave: quizá el segmento ya confía suficientemente en las plataformas y recomendaciones que utiliza. Si fuera así, la propuesta de verificación no aportaría valor diferencial.

Cómo comprobarlo primero: entrevistar a personas del segmento que hayan organizado recientemente un viaje y pedirles que expliquen cómo eligieron actividades, qué dudas tuvieron y qué alternativas usaron. El equipo debe decidir de antemano qué respuestas apoyarían o debilitarían la hipótesis.

2.13. Actividades

Ejercicio 1 — ¿Startup o empresa tradicional?

Clasifica cada afirmación:

1. "Ya conocemos bien a nuestros clientes y solo queremos mejorar eficiencia."
2. "Todavía no sabemos si el problema que imaginamos es suficientemente importante."
3. "Nuestro objetivo principal ahora es validar hipótesis."
4. "Nuestro proceso principal es optimizar una operación ya conocida."

Solución: 1 y 4, empresa tradicional; 2 y 3, startup.

Ejercicio 2 — Construye tu Lean Startup Canvas reducido

Aplicad esta secuencia a vuestro proyecto:

1. Delimitad el segmento y distinguid cliente, usuario y *early adopters*.
2. Enumerad y priorizad tres problemas; anotad las alternativas actuales.
3. Proponed tres características mínimas y relacionad cada una con un problema.
4. Redactad una propuesta de valor única en una sola frase.
5. Identificad una ventaja especial o, si aún no existe, la ventaja más valorable.

6. Elegid la hipótesis cuya falsedad comprometería el proyecto y proponed cómo comprobarla primero.

Al terminar, revisad la coherencia en sentido inverso: la ventaja debe reforzar la propuesta de valor; la propuesta debe derivarse de la solución; la solución debe responder a los problemas; y los problemas deben pertenecer al segmento.

Ejercicio 3 — Detecta formulaciones débiles

Reescribid estas afirmaciones para convertirlas en hipótesis útiles: "nuestro producto es para todo el mundo"; "el problema es que nuestra app no existe"; "la solución tendrá muchas funciones"; "nuestra propuesta de valor es usar IA"; "nuestra ventaja es que seremos mejores". Indicad qué información falta en cada caso.

2.14. Aplicación al proyecto del curso

El Lean Startup Canvas reducido que habéis aprendido a construir es el que levantaréis para vuestro propio producto, tratando sus cinco casillas como lo que son: hipótesis que habrá que sostener y, más adelante, comprobar. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

2.15. Ideas clave para recordar

- Una startup es una institución humana que crea algo nuevo bajo incertidumbre extrema.
- No se gestiona como una empresa pequeña: no optimiza, **descubre y valida**.
- Se conduce en **modo coche**, con el ciclo **Crear–Medir–Aprender** lo más corto posible.
- El **Lean Startup Canvas reducido** reúne cinco hipótesis: segmento, problema, solución, propuesta de valor y ventaja especial.
- Cliente y usuario pueden ser personas distintas; los *early adopters* son quienes primero sienten el problema y buscan una alternativa.
- Una solución mínima responde a problemas concretos; una propuesta de valor explica el beneficio y la diferencia.
- **Idea no es demanda**; el error más común es construir algo que nadie quiere.
- Primero el ajuste (pivotar), después la escala (optimizar); se comprueba antes el **riesgo clave**.

- Métricas, canales, estructura de costes y flujo de ingresos se estudiarán más adelante, al abordar la salida al mercado.

Apuntes basados en el método Lean Startup y en Running Lean *de Ash Maurya.*

TEMA 3

Análisis DAFO y análisis de contingencias

3.1. Sentido de la sesión

Cuando arrancas un proyecto —el tuyo y el de tu equipo— te haces, casi sin darte cuenta, dos preguntas incómodas: "¿seremos capaces de sacarlo adelante?" y "¿merece la pena hacerlo?". La tentación es responderlas de un plumazo, con optimismo o con miedo, según el día. El problema es que una respuesta basada en la intuición no te sirve para *decidir* ni para *convencer* a nadie: ni a un cliente, ni a un profesor, ni a tus propios compañeros.

Esta sesión te da dos herramientas sencillas y muy potentes para sustituir esa intuición por un análisis ordenado. La primera, el **DAFO**, te ayuda a mirar tu proyecto desde cuatro ángulos a la vez y a hacerte una idea realista de sus posibilidades de éxito. La segunda, el **análisis de contingencias**, convierte la mitad "mala" de ese diagnóstico en un plan de acción: en un plan B preparado de antemano.

Idea central

El DAFO no es un adorno para una memoria ni una casilla que rellenar al final. Es una herramienta de **diagnóstico** que se hace *antes* de decidir, y el análisis de contingencias es lo que le da utilidad: transforma los riesgos que el DAFO ha sacado a la luz en respuestas concretas. Un buen DAFO sin contingencias es un diagnóstico sin tratamiento.

3.1.1. Qué deberías saber hacer al terminar

- Explicar con tus palabras qué problema resuelve un DAFO y por qué es un tipo de *canvas* de análisis.
- Definir correctamente el **sujeto** del análisis ("nosotros" o "el proyecto") antes de

empezar a rellenar casillas.

- Distinguir sin dudar las cuatro casillas a partir de dos ejes: interno/externo y positivo/negativo.
- Rellenar cada casilla evitando los errores típicos.
- Cruzar las casillas para obtener **estrategias** (FO, DO, FA y DA) en lugar de quedarse en cuatro listas.
- Construir una tabla de contingencias (riesgo, probabilidad, impacto, remediación) a partir de las debilidades y amenazas detectadas.
- Leer esa tabla para demostrar que tu equipo tiene siempre un plan B.

3.2. El problema que resuelve el DAFO

Imagina que tu equipo tiene que entregar un prototipo en quince semanas. Tienes ganas, tienes una idea que te gusta y tienes un grupo dispuesto. ¿Saldrá bien? Si respondes "sí" o "no" a secas, no has analizado nada: has apostado. Y apostar está bien en una rifa, no en un proyecto del que dependen tu nota o tu dinero.

El problema de fondo es que el éxito de cualquier iniciativa depende de **muchos factores a la vez**, y nuestra cabeza no es buena sopesando muchos factores simultáneamente. Tendemos a fijarnos solo en lo que tenemos delante: si hoy estamos motivados, vemos el proyecto viable; si hoy ha fallado el servidor, lo vemos imposible. Necesitamos un método que nos *obligue* a mirar de forma ordenada y completa, sin olvidar ángulos importantes.

Ese método existe y es muy antiguo en su lógica: separar lo que *ayuda* de lo que *estorba*, y separar lo que está *dentro de nuestro control* de lo que viene *de fuera*. Cruza esas dos distinciones y obtienes cuatro casillas. Eso es un DAFO.

3.2.1. El DAFO es un *canvas* de análisis muy simple

Un *canvas* es, sencillamente, un lienzo dividido en zonas, donde cada zona te hace una pregunta. En lugar de enfrentarte a la pregunta enorme y vaga ("¿esto saldrá bien?"), el canvas la trocea en preguntas pequeñas y manejables, y al rellenarlas todas obtienes una visión de conjunto. El *Lean Startup Canvas* o el *Business Model Canvas* hacen esto con muchas casillas; el DAFO es el canvas **más simple de todos**: solo cuatro casillas.

Qué significan las siglas

Debilidades, **A**menazas, **F**ortalezas y **O**portunidades. En inglés es *SWOT* (Strengths, Weaknesses, Opportunities, Threats). Es exactamente la misma herramienta; cambia solo el idioma y el orden de las letras.

A pesar de su simpleza, el DAFO hace algo muy valioso: te da un **diagnóstico de tus posibilidades de éxito** y, además, te deja preparado el material para el siguiente paso.

Las fortalezas y oportunidades te dicen sobre qué *apoyarte*; las debilidades y amenazas te dicen de qué *protegerte* —y eso es justo lo que alimentará el análisis de contingencias de la segunda parte de estos apuntes.

3.3. La pregunta primero: ¿de quién es el DAFO?

Este es el punto donde más DAFO se tuercen, así que conviene detenerse. Antes de escribir una sola palabra en las casillas, tienes que decidir **qué estás analizando**. La herramienta es la misma, pero el contenido de las casillas cambia por completo según el problema que quieras resolver.

3.3.1. Caso 1: "¿Seremos capaces de acabar el proyecto a tiempo?"

Aquí el sujeto del análisis sois **vosotros, el equipo**. No estás juzgando si la idea es buena, sino si *tenéis lo que hace falta* para ejecutarla en plazo. Por tanto, las cuatro casillas hablan de vosotros:

- **Nuestras debilidades**: en qué flojeamos como equipo (poca experiencia con cierta tecnología, calendarios incompatibles, nadie sabe diseñar...).
- **Nuestras amenazas**: qué de fuera puede impedirnos terminar (exámenes de otras asignaturas a mitad del cuatrimestre, una herramienta de pago que se nos acaba, dependencia de un dato externo...).
- **Nuestras fortalezas**: en qué somos buenos (un programador sólido, buena comunicación interna, ya hemos trabajado juntos antes...).
- **Nuestras oportunidades**: qué de fuera juega a nuestro favor (acceso a un laboratorio, un profesor que tutoriza, una plantilla ya hecha que podemos reutilizar...).

3.3.2. Caso 2: "¿Es viable el proyecto?"

Aquí el sujeto cambia: ya no sois vosotros, es **el proyecto o la idea en sí misma**, con independencia de quién la ejecute. La pregunta es si la propuesta tiene sentido y futuro en su mercado o contexto:

- **Debilidades del proyecto**: puntos flacos de la propia propuesta (margen económico ajustado, depende de una tecnología inmadura, difícil de explicar al usuario...).
- **Amenazas al proyecto**: factores externos que pueden hundirlo (un competidor ya consolidado, un cambio de normativa, una moda pasajera...).
- **Fortalezas del proyecto**: ventajas propias de la idea (resuelve un problema real, es barato de mantener, es difícil de copiar...).

- **Oportunidades para el proyecto**: tendencias externas que lo favorecen (un nicho desatendido, una subvención disponible, un público que crece...).

Precaución: no mezcles los dos DAFO

El mismo hecho cambia de casilla —o desaparece— según el sujeto. "A nuestro equipo le falta un diseñador" es una *debilidad nuestra* en el Caso 1, pero **no es nada** en el Caso 2, porque la viabilidad del proyecto no depende de quién lo haga. Decide primero el sujeto y mantenlo durante todo el análisis.

Truco para no perderte

Antes de rellenar, escribe arriba del DAFO la frase exacta: *"DAFO de _____ para responder a _____"*. Si todas las frases que escribes después encajan en ese encabezado, vas bien. Si una no encaja, sobra.

3.4. La matriz: dos ejes, cuatro casillas

Toda la lógica del DAFO cabe en dos preguntas que aplicas a cada factor que se te ocurre:

1. **¿Viene de dentro o de fuera?** Lo *interno* es lo que tú controlas o casi (tus capacidades, tu organización, tus recursos). Lo *externo* es el entorno: lo que ocurre te afecta, pero no lo decides tú.
2. **¿Ayuda o estorba?** ¿Ese factor empuja hacia el éxito o tira en contra?

Cruzando ambas preguntas obtienes la matriz:



La regla que lo resuelve casi todo

Interno = depende de mí (fortalezas y debilidades).
Externo = depende del entorno (oportunidades y amenazas).
Positivo = ayuda (fortalezas y oportunidades).

Negativo = estorba (debilidades y amenazas).

Si dudas en qué casilla va algo, hazte las dos preguntas en este orden: *¿lo controlo yo?* y *¿me ayuda?*. Las dos respuestas te dan la casilla.

3.5. Las cuatro casillas, una a una

A partir de aquí usamos siempre el **Caso 1** ("¿seremos capaces de hacer el proyecto?") porque es el más útil para vuestros equipos de clase. El sujeto, por tanto, sois vosotros.

3.5.1. Debilidades (interno · negativo)

Son las carencias del propio equipo: aquello en lo que flojeáis y que está en vuestra mano (al menos en parte) mejorar.

- **Pistas para rellenarla**: piensa en habilidades que faltan, experiencia que no tenéis, recursos de los que andáis cortos, hábitos de trabajo flojos, dependencias internas ("solo una persona sabe hacer X").
- **Truco**: una buena debilidad se puede convertir en una frase de mejora. "No sabemos usar la API" → "deberíamos aprender la API". Si no puedes formular esa frase, probablemente no era una debilidad sino una queja.

El error más común en Debilidades

Confundir una **debilidad** (interna, tuya) con una **amenaza** (externa). "La fecha de entrega es muy justa" es externa: es una amenaza. "No sabemos repartirnos el trabajo" es interna: es una debilidad. Pregúntate siempre: *¿esto lo puedo cambiar yo, o solo lo padezco?*

3.5.2. Amenazas (externo · negativo)

Son los factores del entorno que pueden hacer que el proyecto no llegue a buen puerto, y que vosotros *no* controláis.

- **Pistas para rellenarla**: piensa en plazos impuestos, dependencias de terceros, herramientas o servicios que podrían fallar o encarecerse, solapamiento con otras asignaturas, cambios de requisitos por parte del cliente o del profesor.
- **Truco**: una amenaza siempre se puede escribir como "si ocurre X, nos perjudica". Esa forma condicional te será utilísima en la segunda parte, porque cada amenaza así escrita es ya casi una fila de la tabla de contingencias.

Precaución: amenazas, no catástrofes

No infles la casilla con desastres improbables ("y si hay un apagón nacional"). Una amenaza útil es la que tiene una probabilidad razonable de ocurrir *dentro del horizonte del proyecto*. Las amenazas de ciencia ficción solo sirven para asustarse y no preparan nada.

3.5.3. Fortalezas (interno · positivo)

Son las capacidades reales del equipo en las que os podéis apoyar.

- **Pistas para rellenarla**: habilidades técnicas concretas, experiencia previa, buena comunicación, motivación alta, recursos propios (equipos, licencias, contactos), haber trabajado juntos antes.
- **Truco**: una fortaleza solo cuenta si es **relevante para este proyecto**. "Se me da bien dibujar" es una fortaleza si el proyecto necesita ilustraciones; si no, es solo una virtud personal sin efecto aquí.

El error más común en Fortalezas

Escribir genéricos vacíos: "somos un equipo comprometido", "tenemos ganas". Eso lo dice todo el mundo y no se puede usar para nada. Una fortaleza útil es **específica y verificable**: "Marta ha hecho ya dos prototipos en Figma", "el equipo coincide tres tardes a la semana".

3.5.4. Oportunidades (externo · positivo)

Son las circunstancias del entorno que, sin haberlas creado vosotros, juegan a vuestro favor si sabéis aprovecharlas.

- **Pistas para rellenarla**: recursos disponibles que podéis usar (laboratorios, créditos gratuitos de una herramienta, plantillas, datasets abiertos), apoyo de tutores, posibilidad de reutilizar trabajo de otra asignatura, eventos o convocatorias en las que presentar el resultado.
- **Truco**: una oportunidad bien escrita lleva implícita una acción para aprovecharla. "Hay créditos gratuitos de la API hasta junio" → acción: "usémoslos antes de junio".

El error más común en Oportunidades

Confundir oportunidad (externa) con fortaleza (interna). "Sabemos programar bien" es fortaleza vuestra. "Existe una librería gratuita que hace medio trabajo" es oportunidad del entorno. El criterio vuelve a ser: *¿lo he traído yo, o estaba ahí fuera?*

Cómo se lee el DAFO completo

Las dos casillas de la izquierda (fortalezas y oportunidades) son tu **capital**: aquello sobre lo que construir. Las dos de la derecha (debilidades y amenazas) son tu **frente de riesgo**: aquello de lo que protegerte. Un proyecto con mucho capital y poco riesgo controlado tiene buenas posibilidades; uno con poco capital y mucho riesgo sin plan, no. El DAFO no "decide" por ti, pero te pone delante, ordenado, todo lo que necesitas para decidir tú.

3.6. Ejemplos comentados

Tomemos un caso realista de clase: un equipo de cuatro personas debe entregar en quince semanas un *prototipo de app turística asistida por IA*. La pregunta es la del Caso 1: **"¿seremos capaces de acabarlo a tiempo?"**.

Ejemplo: DAFO del equipo (Caso 1)

Fortalezas (interno · ayuda)
- Dos integrantes ya programaron una API REST el curso pasado.
- El grupo coincide presencialmente los martes y jueves por la tarde.

Debilidades (interno · estorba)
- Nadie del equipo ha diseñado nunca una interfaz de usuario.
- No tenemos un sistema claro para repartir y seguir las tareas.

Oportunidades (externo · ayuda)
- Hay créditos gratuitos de la API de IA disponibles este cuatrimestre.
- El profesor ofrece tutorías semanales de seguimiento.

Amenazas (externo · estorba)
- En la semana 8 se concentran exámenes parciales de otras asignaturas.
- El prototipo depende de un servicio externo que podría cambiar su API.

Comentario del ejemplo

Fíjate en tres cosas. Primero, todo está escrito desde el **equipo**, no desde la idea: encaja en el encabezado "DAFO del equipo para saber si acabaremos a tiempo". Segundo, las fortalezas son **específicas** ("dos integrantes", "martes y jueves"), no genéricas. Tercero, las amenazas están escritas en forma casi condicional ("podría cambiar su API"): eso las deja listas para convertirse en filas de la tabla de contingencias.

Un error sutil que este equipo evitó

Habría sido fácil escribir "la entrega es en quince semanas" como debilidad. Pero el plazo no es interno: no lo controla el equipo. Es una condición del entorno y, si aprieta, va en amenazas. Colocarlo bien es lo que luego permite tratarlo como riesgo

y no como un defecto propio imposible de arreglar.

3.7. El análisis cruzado: de las casillas a las estrategias

Con las cuatro casillas rellenas tienes un diagnóstico, pero todavía no tienes ninguna decisión. Un DAFO que se queda en cuatro listas es exactamente lo que critica todo el mundo: un adorno de memoria. Lo que lo convierte en herramienta útil es el paso siguiente, y es muy sencillo: **cruzar las casillas de dos en dos**, una de la izquierda con una de la derecha, y preguntarse qué haríais.

La idea del cruce

Las casillas no viven aisladas. Una fortaleza sirve *para algo*: para aprovechar una oportunidad o para protegeros de una amenaza. Una debilidad duele *frente a algo*: frente a una oportunidad que no podréis aprovechar o frente a una amenaza que os pillará desarmados. Cruzarlas es pasar de "esto es lo que hay" a "esto es lo que vamos a hacer".

Del cruce salen cuatro tipos de estrategia. Se nombran con las iniciales de las dos casillas que se combinan:

Estrategia	Cruce	La pregunta que responde
FO — ofensiva	Fortaleza + Oportunidad	¿Qué sabemos hacer que nos permita aprovechar lo que hay ahí fuera?
DO — de refuerzo	Debilidad + Oportunidad	¿Qué recurso externo puede tapar el agujero que tenemos?
FA — defensiva	Fortaleza + Amenaza	¿En qué nos apoyamos para que eso que no controlamos nos haga menos daño?
DA — de supervivencia	Debilidad + Amenaza	¿Qué hacemos para que nuestro punto flaco y el peligro externo no coincidan?

Fíjate en la lógica: las dos primeras miran hacia lo positivo del entorno (construir), las dos últimas hacia lo negativo (protegerse). Y las que empiezan por F se apoyan en lo que tenéis; las que empiezan por D admiten que os falta algo.

Ejemplo: las cuatro estrategias del equipo de la app turística

Retomamos el DAFO del apartado anterior y cruzamos sus casillas.

FO — *dos integrantes ya programaron una API REST + hay créditos gratuitos de la API de IA*: montamos la integración con la IA en las dos primeras semanas, mientras los créditos están vigentes y con la gente que ya sabe hacerlo.

DO — *nadie ha diseñado nunca una interfaz + el profesor ofrece tutorías semanales*:

pedimos una tutoría específica de diseño en la semana 3 y llevamos a ella el primer boceto, en vez de improvisar la interfaz al final.

FA — *el grupo coincide martes y jueves + en la semana 8 se concentran los exámenes*: usamos esas dos tardes fijas para adelantar trabajo en las semanas 1–7 y llegar a la 8 con colchón.

DA — *no tenemos un sistema para repartir tareas + el servicio externo podría cambiar su API*: si las dos cosas coinciden, nadie sabrá quién arregla la integración. Definimos en la semana 1 un tablero con un responsable por tarea y designamos a quién le tocaría ese incendio.

Cómo se escribe una estrategia cruzada

Una estrategia cruzada bien escrita tiene siempre la misma forma: *"usando [casilla interna], vamos a [acción concreta] para [aprovechar / protegernos de] [casilla externa]"*. Si al leerla no se identifica de qué dos casillas nace, o si no contiene ninguna acción, todavía no es una estrategia.

El error más común en el cruce

Escribir cuatro frases bonitas que no salen de ninguna casilla concreta. "Aprovecharemos nuestras fortalezas para superar las amenazas" no dice nada: es la definición de FA, no una estrategia FA. El cruce solo vale si señaláis **qué** fortaleza y **qué** amenaza, y qué haréis exactamente con ellas.

No hacen falta dieciséis estrategias

Si tenéis cuatro entradas por casilla, los cruces posibles son muchos. No se trata de agotarlos: basta con **una estrategia por tipo**, la que nazca del cruce más relevante. Cuatro decisiones bien elegidas valen más que dieciséis combinaciones mecánicas.

3.8. Ejercicios propuestos (DAFO)

Ejercicio 1 — Clasifica

Para el proyecto de app turística del ejemplo, decide en qué casilla va cada uno de estos factores y justifica por qué con las dos preguntas (¿interno?, ¿ayuda?):

1. "Una de las integrantes habla tres idiomas."
2. "Ha salido una app muy parecida la semana pasada."
3. "No hemos decidido todavía qué hace exactamente la app."
4. "El ayuntamiento publica datos abiertos de transporte que podríamos usar."

Ejercicio 2 — Vuestro propio DAFO

Con vuestro equipo real y vuestro proyecto real, rellenad un DAFO del Caso 1. Reglas: mínimo dos entradas por casilla, todas específicas y verificables, y cada amenaza escrita en forma condicional ("si ocurre…, nos perjudica"). Marcad con ▲ las amenazas y debilidades que os parezcan más serias: las necesitaréis en la segunda parte.

Ejercicio 3 — Cruza las casillas

Con el DAFO que acabáis de rellenar en el ejercicio anterior, escribid **una estrategia de cada tipo** —FO, DO, FA y DA— usando la forma "usando [casilla interna], vamos a [acción] para [aprovechar/protegernos de] [casilla externa]". Después pasadles el filtro: tapad las etiquetas y dádselas a otro equipo. Si no son capaces de decir de qué dos casillas nace cada una, la estrategia está mal escrita.

Ejercicio 4 — Caza el error

Aquí tenéis un DAFO mal hecho. Encontrad al menos tres errores y corregidlos:

- *Fortaleza*: "Somos un grupo muy motivado."
- *Debilidad*: "El plazo de entrega es muy corto."
- *Oportunidad*: "Sabemos manejar bien la herramienta de diseño."
- *Amenaza*: "Y si se cae internet en todo el país durante un mes."

3.9. Del diagnóstico al plan: el análisis de contingencias

Ya tienes el diagnóstico. El DAFO te ha dejado, en su mitad derecha, una lista de **debilidades y amenazas**: todo lo que puede salir mal. Pero saber qué puede salir mal no sirve de nada si no haces algo con esa información. Aquí entra el análisis de contingencias.

3.9.1. El problema que resuelve

Pregúntate qué pasa el día en que una de esas amenazas se cumple. Si la respuesta es "improvisaremos sobre la marcha", tu proyecto está a merced de la suerte. Un equipo profesional no improvisa ante los problemas previsibles: los ha **previsto** y tiene preparada una respuesta. A esa respuesta preparada de antemano la llamamos *plan B*, y al conjunto de todos los planes B, el **plan de contingencias**.

Idea clave

Una *contingencia* es algo que **podría** ocurrir y que, si ocurre, afecta al proyecto. El análisis de contingencias responde a una sola pregunta, repetida para cada riesgo: *"¿y si pasa esto, qué hacemos?"*. Prepararla por escrito, antes, es lo que diferencia un

equipo que controla su proyecto de uno que solo espera que no pase nada.

3.9.2. Cómo enfocar el análisis

El insumo del análisis es directo: **coge las debilidades y amenazas de tu DAFO** (las casillas de la derecha) y trátalas como riesgos. No hace falta inventar nada nuevo; el DAFO ya hizo el trabajo de detectarlos. Lo que añades ahora son tres cosas por cada riesgo: cómo de probable es, cómo de grave sería, y qué harías al respecto.

3.10. La tabla de contingencias

La herramienta del análisis es una tabla con cuatro columnas. Es deliberadamente simple, igual que el DAFO:

Riesgo	Probabilidad	Impacto	Remediación (plan B)
Qué puede salir mal, escrito de forma concreta	Cómo de probable es (alta/media/baja)	Cómo de grave sería (alto/medio/bajo)	Qué haremos: para prevenirlo y/o para reaccionar si ocurre

3.10.1. Qué poner en cada columna

- **Riesgo**: una frase concreta, no una categoría vaga. “El servicio externo cambia su API y rompe nuestra integración”, no “problemas técnicos”.
- **Probabilidad**: lo verosímil que es que ocurra dentro del horizonte del proyecto. Basta una escala de tres niveles (**alta / media / baja**); no necesitas porcentajes finos.
- **Impacto**: el daño que causaría si ocurriese, también en tres niveles (**alto / medio / bajo**). Mide impacto en lo que te importa: retraso, calidad, coste.
- **Remediación**: la parte que de verdad importa. Aquí va el plan B, y conviene distinguir dos tipos de acción: *preventiva* (lo que haces *antes* para que el riesgo no ocurra o sea menos probable) y *reactiva* (lo que harás *si* ocurre, a pesar de todo).

Precaución con la columna de remediación

“Tener cuidado” o “estar atentos” no son remediaciones: son buenas intenciones. Una remediación válida es una **acción concreta** con un responsable implícito y, si se puede, un momento. “Aislamos la llamada al servicio externo en un único módulo desde el principio, para poder sustituirlo en un día si cambia” ✓ sí es una remediación.

3.11. Cómo se interpreta la tabla

Una vez rellena, la tabla no se archiva: se *lee* para tomar decisiones. La clave es cruzar probabilidad e impacto, porque no todos los riesgos merecen la misma atención.

	Impacto bajo	**Impacto alto**
Probabilidad alta	Vigilar / mitigar	**Prioridad máxima**
Probabilidad baja	Aceptar / ignorar	Tener plan B listo

Cómo priorizar

Los riesgos de **probabilidad alta e impacto alto** son tu prioridad absoluta: necesitan prevención *y* reacción. Los de **impacto alto pero probabilidad baja** (raros pero graves) no se previenen, pero conviene tener el plan B escrito por si acaso. Los de **impacto bajo** se vigilan o simplemente se aceptan: gastar esfuerzo en ellos es malgastarlo.

Qué demuestra una buena tabla

Una tabla de contingencias bien hecha demuestra una sola cosa, pero muy importante: que **para cada cosa que puede salir mal, ya tienes una respuesta**. Es decir, que pase lo que pase, tienes un plan B. Esa es la frase que quieres poder decir ante un cliente o un tribunal: *"hemos pensado en lo que puede fallar y sabemos qué haríamos en cada caso"*. Eso transmite control, y el control es lo que da confianza en un equipo.

El error más común en contingencias

Hacer una tabla donde todos los riesgos son "probabilidad alta, impacto alto". Si todo es máxima prioridad, nada lo es: significa que no has discriminado. Esfuérzate en **diferenciar**; la utilidad de la tabla está precisamente en que te dice dónde concentrar el esfuerzo.

3.12. Ejemplo comentado de contingencias

Retomamos las amenazas y debilidades del DAFO del equipo de la app turística y las convertimos en tabla:

Ejemplo: tabla de contingencias del equipo

Riesgo	Prob.	Imp.	Remediación (plan B)
Exámenes en la semana 8 frenan el avance	Alta	Medio	Prevenir: adelantar el grueso del trabajo a las semanas 1–7. Reaccionar: reservar las semanas 9–10 como colchón.
El servicio externo cambia su API	Baja	Alto	Prevenir: aislar la llamada en un solo módulo. Reaccionar: tener identificado un servicio alternativo equivalente.
Nadie sabe diseñar la interfaz	Alta	Medio	Prevenir: usar una plantilla de UI existente en vez de partir de cero. Reaccionar: pedir una tutoría específica de diseño.
Mal reparto de tareas	Media	Alto	Prevenir: definir un tablero de tareas con responsable en la semana 1 y revisarlo cada martes.

Comentario del ejemplo

Observa cómo cada fila nace de una casilla de la derecha del DAFO: los exámenes y el cambio de API venían de *amenazas*; la falta de diseño y el reparto, de *debilidades*. Observa también que el cambio de API, aunque poco probable, tiene impacto alto, así que no se ignora: se le deja un plan B preparado. Y fíjate en que ninguna remediación dice "tener cuidado": todas son acciones concretas, con prevención y reacción separadas.

3.13. Ejercicios propuestos (contingencias)

Ejercicio 5 — De DAFO a tabla

Coged el DAFO del Caso 1 que rellenasteis en el Ejercicio 2 y construid la tabla de contingencias correspondiente. Una fila por cada debilidad y cada amenaza que marcasteis con ▲. Rellenad las cuatro columnas y, en remediación, separad siempre prevención y reacción.

Ejercicio 6 — Prioriza

Sobre vuestra tabla del ejercicio anterior, colocad cada riesgo en la matriz probabilidad × impacto. Señalad cuáles son vuestra "prioridad máxima" y comprobad que la remediación de esos es la más detallada de todas.

Ejercicio 7 — Defiende tu plan B

En dos minutos, y sin mirar la tabla, un compañero os lanza un "¿y si pasa esto...?" por cada riesgo. Tenéis que responder con vuestro plan B de memoria. Si no os sale, esa fila aún no está bien preparada.

3.14. Aplicación al proyecto del curso

El DAFO del equipo y la tabla de contingencias se aplican al proyecto de la asignatura. Son el diagnóstico honesto de lo que vuestro equipo puede y no puede hacer, y el plan B que lo acompaña: sobre ellos se apoyarán la planificación y los compromisos de las semanas siguientes. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

3.15. Ideas clave para recordar

- El DAFO es el **canvas de análisis más simple**: cuatro casillas que cruzan interno/externo con positivo/negativo.
- Antes de rellenar, decide el **sujeto**: "nosotros" (¿podremos hacerlo?) o "el proyecto" (¿es viable?). El contenido cambia por completo.
- Para colocar cada factor, dos preguntas: *¿lo controlo yo?* y *¿me ayuda?*.
- Las casillas útiles son **específicas y verificables**; los genéricos ("somos un equipo motivado") no sirven para nada.
- Un DAFO sin **cruce** es solo un diagnóstico: las estrategias **FO, DO, FA y DA** son las que lo convierten en decisiones, y cada una debe señalar de qué dos casillas nace.
- Las debilidades y amenazas del DAFO son la **materia prima** del análisis de contingencias.
- La tabla de contingencias tiene cuatro columnas: **riesgo, probabilidad, impacto, remediación**.
- La remediación es una **acción concreta** (preventiva y reactiva), no una buena intención.
- Se interpreta cruzando probabilidad e impacto para **priorizar**; si todo es prioridad máxima, no has discriminado.

- Una buena tabla demuestra que, pase lo que pase, **tienes un plan B**.

TEMA 4

Diseño de la interacción

4.1. Sentido de la sesión

Un producto multimedia no se distingue de un libro o de un vídeo por la cantidad de imágenes, sonidos o animaciones que contiene, sino por una propiedad que ninguno de esos medios tiene: deja que el usuario *intervenga*. La interactividad es la materia prima de lo multimedia, y diseñarla bien es lo que separa un producto que la gente usa con gusto de otro que se abandona a los pocos minutos. Esta sesión trata, precisamente, de cómo se diseña esa interactividad de forma deliberada y no por casualidad.

El tema tiene dos mitades que conviene leer como una sola historia. En la primera aprenderás un **método** para construir la interacción de un producto: qué hacer primero, qué hacer después y qué entregables produce cada fase, hasta llegar al *storyboard* y al *mockup*. En la segunda descubrirás **para qué** sirve en el fondo todo ese trabajo: ganar y conservar la atención de las personas, es decir, lograr *engagement*. Y veremos que ese objetivo, que ya planteaba la economía de la atención hace dos décadas, está siendo reescrito ahora mismo por la inteligencia artificial.

Idea central

La interactividad no se "añade" al final de un producto multimedia: se **diseña** desde el principio, con un método propio. Y todo ese método persigue un único objetivo escaso y valioso: la **atención** del usuario. Diseñar la interacción es, en última instancia, administrar la atención de las personas de forma honesta y eficaz.

4.1.1. Qué deberías saber hacer al terminar

- Explicar qué es la interactividad y distinguir sus distintos niveles.
- Reconocer los errores típicos que arruinan un diseño interactivo.
- Aplicar el método de diseño de la interacción (información, interacción, presentación) y nombrar el entregable de cada fase.

- Elaborar un *storyboard* correcto que sirva de puente hacia las especificaciones y el *mockup* o MVP.
- Entender la economía de la atención, por qué se llama "economía" y qué principios deben guiar una interacción que la use de forma responsable.
- Situar la "segunda ola" de la economía de la atención y comprender cómo la IA reconfigura el mecanismo.

4.2. El problema: ¿qué es la interactividad?

Empecemos por el problema, no por la solución. Cuando se encarga un producto multimedia, la tentación del equipo técnico es lanzarse a programar: botones, menús, pantallas. Pero antes de programar nada hay que responder a una pregunta incómoda: *¿qué significa exactamente que el usuario pueda interactuar, y qué gana el producto con ello?*

Una definición operativa, suficiente para esta asignatura, es esta: interactuar es **proporcionar acciones para que el sujeto participe** y, a la vez, **proporcionar un escenario para que ocurran cosas**. Las dos mitades importan. Si solo hay acciones sin un escenario donde tengan consecuencias, tenemos un mando a distancia, no una experiencia. Si solo hay un escenario vistoso pero el usuario no puede hacer nada en él, tenemos una película, no un producto interactivo.

Idea clave

Interactividad = acciones del usuario + un escenario donde esas acciones producen efectos perceptibles. Sin las dos cosas a la vez, no hay verdadera interacción.

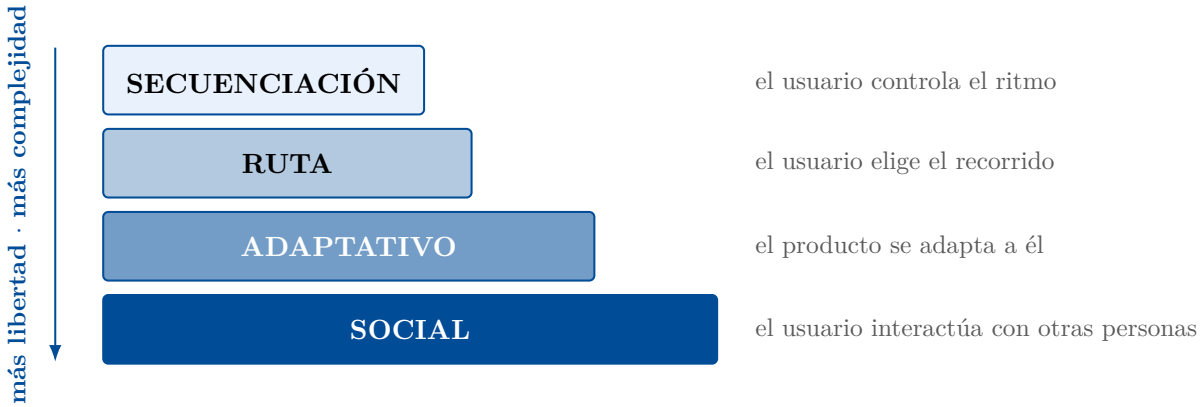
Conviene además recordar de dónde venimos. Hay una observación clásica de Paul Heckel que resume el cambio cultural que vivimos: el cine no floreció mientras lo controlaron los ingenieros, sino cuando el control pasó a los *artistas y comunicadores*; y lo mismo ocurre con el software interactivo. Es decir, la interactividad no es un problema puramente técnico, sino de *comunicación*. El reto no es "que funcione", sino "que signifique algo" para quien lo usa.

4.2.1. Cómo enfocar la interactividad para que enriquezca el producto

La interactividad enriquece un producto cuando **da más control al usuario sin abrumarlo**. Ese control puede darse en distintos grados, y no todos los productos necesitan el mismo. Pensar en *niveles de interactividad* ayuda a decidir cuánta libertad ofrecer:

- **Control de secuenciación.** El usuario decide el ritmo: avanzar, retroceder, pausar. Es el nivel mínimo (pasar páginas, controlar un vídeo).

- **Control de la ruta.** El usuario elige por dónde ir: menús, navegación no lineal, exploración libre del contenido.
- **Sistemas adaptativos.** El producto reacciona al usuario y cambia su comportamiento en función de lo que este hace (recomendaciones, dificultad ajustable, contenido personalizado).
- **Sistemas sociales.** El usuario interactúa con *otras personas* a través del producto. Es el nivel que ha redefinido la web moderna: la interacción ya no es solo persona–máquina, sino persona–máquina–persona.



Cada nivel *contiene* a los anteriores: un sistema social sigue permitiendo elegir la ruta y controlar el ritmo. Por eso las barras crecen; subir un escalón añade libertad al usuario, pero también complejidad y coste al proyecto.

Precaución: más interactividad no es siempre mejor

Subir de nivel cuesta dinero, complejidad y riesgo de confundir al usuario. Un producto que solo necesita control de secuenciación no mejora por incorporar un sistema social a medias. Elige el nivel que sirva al objetivo comunicativo, no el más vistoso.

4.2.2. Los errores más comunes al hacer un producto interactivo

Antes de ver el método, conviene tener delante los errores que ese método pretende evitar. Casi todos nacen de la misma raíz: **diseñar desde lo que el autor sabe hacer, no desde lo que el usuario necesita entender**.

El error más común: interactividad oculta

Esconder lo que el usuario puede hacer. Si una acción no se ve, no existe. "Que el usuario lo descubra" no es un principio de diseño, es una rendición. La posibilidad de interactuar debe **mostrarse**, no adivinarse (las únicas excepciones razonables son los juegos y los *easter eggs*, donde descubrir forma parte de la gracia).

Junto a ese error capital, los más frecuentes son:

- **Inconsistencia.** Que cosas que se comportan igual parezcan distintas, o que cosas distintas parezcan iguales. El usuario aprende patrones muy deprisa, pero luego no perdona que se rompan.
- **Falta de retroalimentación.** Que el usuario actúe y no pase nada visible. Sin respuesta, la persona no sabe si pulsó bien, si el sistema le oyó o si debe volver a intentarlo (y vuelve a pulsar, empeorando las cosas).
- **Comportamiento impredecible.** Que el usuario no pueda anticipar qué ocurrirá *antes* de actuar. La consecuencia se mide en errores de usuario.
- **La ilusión de simplicidad.** Hacer que el producto *parezca* simple escondiendo controles importantes, en vez de hacerlo *realmente* simple de entender. Es un caso particular y muy actual del primer error.
- **Atrapar al usuario.** No ofrecer una salida clara ni un camino de vuelta al inicio. La persona que se siente perdida o encerrada abandona y busca otra cosa.

Ejemplo: el formulario que borra lo escrito

Rellenas diez campos de un formulario, pulsas "Enviar" y un error te devuelve la página con los diez campos vacíos. Es la suma de varios errores: mala retroalimentación, comportamiento impredecible y desprecio por el trabajo del usuario. Un buen diseño conserva lo introducido, señala con claridad qué falla y permite corregir sin empezar de cero.

4.3. El método: el diseño de la interacción

Frente a la improvisación, el "diseño de la interacción" propone un método. Tomamos como guía las ideas de Ray Kristoff y Amy Satran en *Interactivity by Design*, que ordenan el trabajo en tres áreas de diseño encadenadas. La definición de referencia, recogida por la IxDA, lo expresa así: el diseño de la interacción define la **estructura** y el **comportamiento** de los sistemas interactivos, y busca crear relaciones *significativas* entre las personas y los productos y servicios que usan.

Las tres áreas del diseño según Kristoff y Satran

1. **Diseño de la información** — *¿qué hay y cómo se organiza?* Decidir qué contenidos existen y cómo se estructuran y relacionan entre sí.
2. **Diseño de la interacción** — *¿cómo se mueve y actúa el usuario?* Definir la navegación, las acciones posibles y las respuestas del sistema.
3. **Diseño de la presentación** — *¿cómo se ve y se siente?* Concretar el aspecto visual y sonoro que hace comprensible y atractivo todo lo anterior.

El orden no es caprichoso: no se puede diseñar cómo navegar por algo que aún no se ha organizado, ni vestir visualmente una interacción que todavía no existe.

Vale la pena situar el diseño de la interacción dentro de un mapa mayor. Es una pieza del diseño de experiencia de usuario (UX), junto a la arquitectura de la información, el diseño visual, los factores humanos o la interacción persona–computador. El UX se ocupa de la *visión completa* del producto; el diseño de la interacción se concentra en algo más acotado: las **acciones y reacciones concretas** entre el usuario y el producto. Su meta última es siempre la misma: ayudar al usuario a hacer lo que quiere de la forma más fácil y rápida posible.

4.3.1. Un vocabulario común: las cinco dimensiones de la interacción

Para hablar con precisión de "una interacción", Gillian Crampton Smith propuso cuatro dimensiones y Kevin Silver añadió una quinta. Son un buen mapa para no olvidarte de nada al diseñar:

Dimensión	De qué trata
1. Palabras	Los textos y etiquetas. Deben usar el vocabulario del usuario, no la jerga técnica del equipo.
2. Representaciones visuales	Iconos, formas, fondos. Se captan en una fracción de segundo y aprovechan los símbolos que el usuario ya tiene en memoria.
3. Objetos y espacio físico	El hardware con el que se interactúa: pantalla, ratón, mando, teclado, gesto.
4. Tiempo	Lo que cambia mientras se usa: sonidos, vibraciones, animaciones, esperas.
5. Comportamiento	La forma en que una acción desencadena una reacción; el flujo acción→respuesta.

4.4. Las fases del diseño de la interacción

Las tres áreas anteriores se traducen, en la práctica del aula y del taller, en una secuencia de fases con **entregables concretos**. Cada fase produce un documento que alimenta a la siguiente; esa cadena es la que da rigor al proceso.



- **Diseño de la información → *flowchart*.** Se inventaria el contenido y se dibuja su estructura: qué secciones hay, quién accede a cada una (clientes, editores, administradores, público) y cómo se relacionan. El diagrama de flujo es el plano del edificio antes de decidir por dónde se circula.

- **Diseño de la interacción → *storyboard*.** Sobre esa estructura se decide cómo se navega y qué ocurre en cada pantalla: las transiciones, las acciones disponibles y a dónde conducen. Es el entregable central de este tema y lo tratamos aparte.
- **Diseño del prototipo → *mockup* / MVP.** Se materializan las pantallas con un nivel de fidelidad creciente, desde bocetos hasta un producto mínimo viable que ya se puede probar con usuarios reales.

4.4.1. Los principios que hacen "buena" una interacción

Las fases dicen *qué hacer*; los principios dicen *cómo hacerlo bien*. Son los mismos en cualquier producto, sea una web, una app o un lavavajillas, porque describen cómo las personas entienden las máquinas. Estos cinco son el núcleo clásico:

Principio	Qué exige al diseño
Consistencia	Que lo que se comporta igual parezca igual, y lo distinto, distinto. La consistencia "pasa desapercibida": se nota cuando falta.
Visibilidad	Que la posibilidad de interactuar sea evidente; mostrar las acciones, no esconderlas.
Aprendizaje	Que la interacción sea fácil de aprender y de recordar; reforzar lo bien hecho y apoyarse en metáforas que el usuario ya conoce.
Predecibilidad	Que el usuario sepa qué pasará *antes* de actuar; los *previews* y los textos explicativos ayudan.
Retroalimentación	Que cada acción produzca una respuesta visible: dónde estoy, de dónde vengo, a dónde voy. Sin distraer.

A estos clásicos, la práctica contemporánea añade matices que conviene tener presentes: la **descubribilidad** (que quien nunca ha usado el producto encuentre lo que ofrece), la **legibilidad** (recuerda que los usuarios apenas leen una parte del texto de una pantalla, así que no confíes el mensaje solo a la palabra escrita), la **accesibilidad** (prever a usuarios con dificultades visuales o auditivas: alternativas al color, texto ALT, evitar parpadeos intensos) y la **reducción de latencia** (no hacer esperar: confirmar cualquier clic en menos de unos 50 ms y, si la tarea es larga, informar del progreso con una barra honesta). Por encima de todos, un principio que los engloba: el **diseño centrado en el usuario**, que pone sus necesidades por delante de las del autor y le da libertad para deshacer sus acciones sin miedo.

Mini-actividad: auditoría de principios

Elige una pantalla de un producto que uses a diario. Para cada uno de los cinco principios clásicos, anota una evidencia de que se cumple y otra de que falla. Después propón un cambio concreto para el punto más débil. Trae el ejemplo a clase: lo comentaremos antes de empezar vuestro *storyboard*.

4.5. El storyboard: el puente hacia las especificaciones y el mockup

El *storyboard* es el entregable que más se infravalora y el que más tiempo ahorra después. Hecho con cuidado, es el documento que permite escribir las especificaciones sin ambigüedad y construir el *mockup* o el MVP sin sorpresas. Hecho con prisa, condena al equipo a rehacer pantallas a mitad de proyecto, cuando ya cuesta dinero cambiarlas.

Qué es y para qué sirve un storyboard

Un *storyboard* es la representación, pantalla a pantalla, de cómo será la experiencia: cada pantalla, qué muestra, qué puede hacer el usuario en ella y a qué otra pantalla conduce cada acción. No es arte final; es el guion de la interacción.

4.5.1. Cómo hacer un storyboard correcto

Un *storyboard* bien hecho contiene, para cada pantalla, al menos estos elementos:

1. **Un identificador y un boceto de la pantalla.** Un nombre estable ("Inicio", "Lista de precios") y un esquema sencillo de su contenido. La fidelidad visual no importa aún: cajas y rótulos bastan.
2. **Las acciones disponibles.** Los controles de navegación presentes (inicio, anterior, siguiente, salir) y las acciones propias de esa pantalla.
3. **Los destinos de cada acción.** Una flecha desde cada control hasta la pantalla a la que lleva. Aquí es donde el *storyboard* se vuelve un mapa de navegación y deja al descubierto callejones sin salida o pantallas inalcanzables.
4. **Una nota descriptiva.** Una frase que explique el papel de la pantalla, con qué frecuencia se actualiza su contenido y qué reglas la gobiernan.

Advertencia: vigila los caminos, no solo las pantallas

El valor del *storyboard* no está en dibujar pantallas bonitas, sino en las **flechas** entre ellas. Comprueba dos cosas: que desde cualquier pantalla se pueda volver al inicio (que el usuario nunca quede atrapado) y que no haya pantallas a las que no llegue ninguna flecha (contenido huérfano que nadie verá).

Ejemplo: de la cafetería escolar al mockup

Imagina el *storyboard* de la web de una cafetería escolar con tres pantallas: *Inicio* (descripción y enlaces), *Precios* (tabla de comidas y bebidas) y *Ofertas de la semana* (novedades con imágenes). Cada una lleva su menú *Inicio / Página 2 / Página 3 / Salir* y una nota: "Precios se actualiza cada mes"; "Ofertas, cada semana". Con ese guion, la especificación se escribe casi sola ("la pantalla Precios contiene una

tabla de dos columnas y enlaces al resto") y el *mockup* solo añade el aspecto, sin reinventar la estructura.

Conviene además **prototipar pronto**. El primer borrador siempre está equivocado en algo; mejor descubrirlo en un *storyboard* o en un boceto en la pared que en un producto ya programado, donde cada cambio cuesta tiempo y dinero. Y el flujo no tiene por qué ser lineal: haz varios prototipos y vuelve sobre el último para detectar fallos y mejoras.

4.6. La economía de la atención: el engagement como meta

Llegamos a la pregunta de fondo. Hemos aprendido un método para diseñar la interacción, pero ¿con qué propósito último? La respuesta es el *engagement*: lograr que el usuario **preste su atención** al producto y la mantenga. Y la atención, en el mundo actual, es un recurso escaso. Por eso tiene sentido hablar de una verdadera **economía de la atención**, el marco que popularizaron Thomas Davenport y John C. Beck.

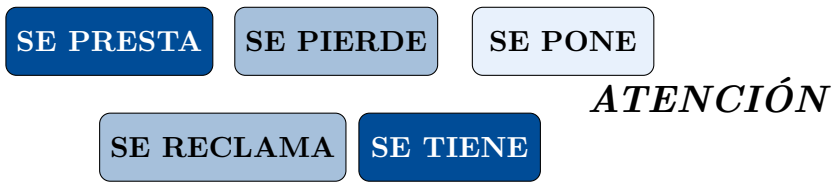
El punto de partida es una paradoja de la abundancia formulada por Herbert Simon: en un mundo rico en información, lo escaso no es la información, sino lo que la información consume. Y lo que la información consume es la **atención** de quien la recibe. Dicho de otro modo: *una riqueza de información genera una pobreza de atención.* Cuando hay más contenido del que nadie puede consumir, lo valioso deja de ser producir información y pasa a ser captar la atención que esa información necesita.

Por qué hablamos de "economía"

Hablamos de economía porque la atención se comporta como un recurso económico: es **escasa** (el día tiene 24 horas para todos), es **deseada** (todos los proveedores la quieren), y por tanto se puede **intercambiar**. En internet, los proveedores de información no compiten por nuestro dinero de forma directa: compiten por nuestra atención. La pregunta clave de cualquier diseñador es entonces: *¿qué entrego yo a cambio de la atención que pido?*

4.6.1. La atención como recurso: cinco verbos

Para entender por qué la atención funciona como una moneda, fíjate en los verbos que admite, exactamente los mismos que el dinero o el tiempo:



Si la atención se presta, se pierde, se pone, se reclama y se tiene, entonces es algo que poseemos en cantidad limitada y que gastamos al usarlo. Esa es la base de toda la

economía de la atención: cada minuto que un usuario dedica a tu producto es un minuto que ha decidido *no* dedicar a otra cosa.

4.6.2. Moneda social y star system

Si los proveedores compiten por nuestra atención, la siguiente pregunta es: *¿qué nos dan a cambio?* No siempre dinero. A menudo nos dan **moneda social**: reconocimiento, pertenencia, la posibilidad de comunicarnos con otras personas. La web social descubrió que la moneda más poderosa hoy no es la información, sino la **comunicación entre personas**. Por eso las redes sociales son máquinas de atención: nos dan contacto humano a cambio de tiempo y datos.

Ese mecanismo genera un *star system*: en internet, alguien (a menudo anónimo o con seudónimo) acumula atención, y esa atención le otorga reconocimiento y le genera privilegios. La atención atrae más atención.

Para tu proyecto: encuentra tu moneda social

Antes de diseñar la interacción de tu producto multimedia, responde con honestidad: *¿cuál es la moneda social que ofreces?* ¿Qué recibe el usuario a cambio de su atención: utilidad, entretenimiento, reconocimiento, conexión con otros? Si no sabes responder, todavía no tienes un producto, tienes una pantalla.

4.6.3. Principios para usar bien la economía de la atención

La economía de la atención es éticamente ambivalente: la misma técnica que ayuda a un usuario a encontrar lo que busca puede usarse para retenerlo en contra de su interés. Estos principios distinguen un diseño que *merece* la atención de otro que simplemente la *extrae*:

- **Da valor a cambio.** Cada vez que pides atención, entrega algo proporcionado: información útil, una emoción, una conexión. La atención mal pagada se retira deprisa.
- **Respeta la atención como recurso finito.** No la malgastes con fricción, esperas o ruido. Reducir la latencia y eliminar pasos innecesarios es respetar al usuario.
- **Haz visible el estado del sistema.** La retroalimentación honesta evita que el usuario gaste atención adivinando qué ocurre.
- **No abuses de la captación involuntaria.** Las notificaciones y alertas roban atención de forma involuntaria; úsalas solo cuando aporten, no para inflar métricas.
- **Devuelve el control.** Permite al usuario decidir cuándo y cuánta atención presta: silenciar, posponer, deshacer, salir. La libertad de marcharse es, paradójicamente, lo que hace que vuelva.

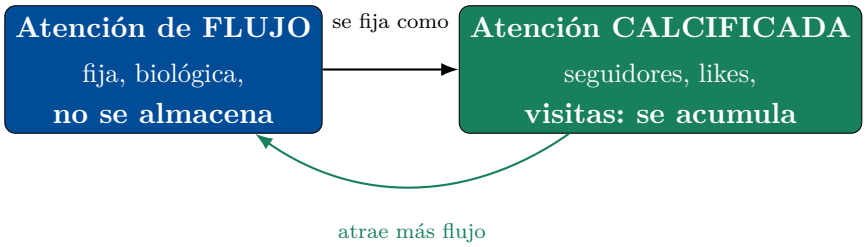
No hacer: confundir engagement con retención forzada

Maximizar el tiempo en pantalla a toda costa — mediante *scroll* infinito, recompensas variables o patrones que dificultan salir — produce métricas altas a corto plazo y usuarios resentidos a largo plazo. El *engagement* legítimo nace de aportar valor, no de atrapar. Diseñar para la adicción no es buen diseño de interacción: es explotación.

4.7. La segunda ola: la atención como moneda simbólica universal

La economía de la atención que describieron Davenport y Beck era, sobre todo, una economía publicitaria: las plataformas captaban nuestra atención para vendérsela a los anunciantes. Pero las redes sociales han hecho posible algo que antes era solo una conjetura teórica: una **"segunda ola"** en la que la atención no solo se capta, sino que se *acumula, almacena e intercambia* como una verdadera moneda entre personas. Esta es la tesis que desarrolla Maxi Heitmayer en su trabajo sobre la segunda ola de la economía de la atención.

La clave es un modelo de **doble flujo** que distingue dos formas de atención:



- **Atención de flujo** (*flow*). Es la que cada persona gasta y recibe momento a momento. Está limitada biológicamente, no se puede guardar y se pierde si no se usa. Es como el tiempo: fluye y se agota.
- **Atención calcificada** (*calcified*). Es la atención de flujo que las plataformas **registran y hacen visible**: el número de seguidores, los *likes*, las reproducciones acumuladas. A diferencia del flujo, sí se almacena, sí se acumula y, sobre todo, **promete atención futura**.

La idea decisiva: la atención calcificada se puede cambiar por otras monedas

La atención calcificada funciona como una moneda simbólica: igual que el dinero no tiene valor en sí mismo pero da acceso a bienes, un número alto de seguidores no es valioso en sí, pero se puede **traducir** en dinero (patrocinios), en influencia (un mensaje político amplificado) o en estatus. Esa conversión entre la atención y otras "monedas" de la sociedad es lo que convierte la economía de la atención en una economía de pleno derecho.

Este mecanismo se ve con claridad en el *streaming* en directo: el espectador puede convertir dinero en atención (una donación que el *streamer* agradece en pantalla, haciéndole visible un instante ante toda la audiencia), y el *streamer* convierte la atención de su audiencia en dinero. La atención circula en las dos direcciones, y la plataforma hace de intermediaria. Heitmayer plantea incluso una pregunta inquietante para el futuro: si la atención puede sustituir al dinero como medio de intercambio, ¿qué consecuencias tendrá para la desigualdad y para la salud mental, cuando la pobreza ya no signifique solo falta de recursos, sino literalmente falta de aprecio y reconocimiento?

Matiz: la atención se comporta más como el "crédito" que como el efectivo

A diferencia del dinero, gastar atención no es entregar una cantidad fija a cambio de algo: se parece más a conceder un crédito basado en la reputación. Por eso la atención calcificada crea bucles que se refuerzan solos — "ir viral" — pero también permite el castigo colectivo retirándola de golpe — la "cancelación".

4.8. La IA reconfigura el mecanismo

Si las redes sociales abrieron la segunda ola, la inteligencia artificial la está acelerando y rediseñando desde dentro. La IA es hoy el **motor** de la economía de la atención: ya no compite por nuestra atención de forma genérica, sino que la modela individuo a individuo.

Cómo opera la IA en la economía de la atención

- **Hiperpersonalización.** Los algoritmos analizan tu historial y tu comportamiento para servirte exactamente el contenido que te mantendrá mirando. El *feed* deja de ser una lista y pasa a ser un espejo optimizado.
- **Publicidad predictiva.** La IA no solo registra lo que pulsas: *anticipa* lo que querrás, segmenta audiencias y convierte la curiosidad en compra.
- **Analítica predictiva.** Al adelantarse a tus preferencias, reduce tu fatiga de decisión... pero suele empujarte hacia lo que conviene a la plataforma, no a ti.

El reverso oscuro de esta optimización son las **cámaras de eco** (la IA te muestra solo lo que confirma tus creencias, limitando el pensamiento crítico) y los **patrones oscuros** (*dark patterns*): interfaces diseñadas para engañar al usuario hacia acciones que no le benefician, como suscripciones difíciles de cancelar o cesión de datos por descuido. Aquí se cierra el círculo del tema: los patrones oscuros son, exactamente, la negación de todos los principios de buena interacción que estudiamos en la primera parte. La visibilidad, la predecibilidad y el control del usuario no son solo buenas prácticas de usabilidad: son una **defensa ética** frente a la manipulación.

Advertencia: de la economía de la atención a la economía de la intención

Se habla ya de una *economía de la intención*, en la que la IA no solo capta tu atención, sino que intenta anticipar y orientar tus *intenciones* antes de que las formules. El salto es importante: ya no compite por lo que miras, sino por lo que vas a querer. Para un futuro diseñador, entender este desplazamiento es decisivo: marca la frontera entre ayudar al usuario a decidir y decidir por él.

Actividad de cierre: el contrato de atención de vuestro producto

En vuestro grupo de proyecto, redactad un breve "contrato de atención" de media página que responda a tres preguntas: (1) ¿Qué atención pedimos al usuario y en qué momentos? (2) ¿Qué valor real le entregamos a cambio? (3) ¿Qué tres mecanismos incorporamos para que conserve el control (silenciar, deshacer, salir...)? Adjuntadlo a vuestro *storyboard*: será el criterio para juzgar si vuestro *engagement* es legítimo o forzado.

4.9. Aplicación al proyecto del curso

El *storyboard* del flujo de uso y el contrato de atención que acabáis de redactar se aplican al producto de cada equipo, y son además la materia prima directa del tema siguiente: cada pantalla, cada acción y cada función que dibujéis aquí se convertirá después en un requisito de la especificación. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

4.10. Ideas clave para recordar

- La interactividad es **acciones + escenario**; no se añade al final, se diseña desde el principio.
- Hay **niveles** de interactividad (secuenciación, ruta, adaptativo, social): elige el que sirva al objetivo, no el más vistoso.
- El error capital es la **interactividad oculta**; muéstrala, no la escondas.
- El método tiene tres áreas y tres entregables: **información (flowchart) → interacción (storyboard) → presentación (mockup/MVP)**.
- Cinco principios clásicos: **consistencia, visibilidad, aprendizaje, predecibilidad y retroalimentación**, sobre la base del diseño centrado en el usuario.
- El **storyboard** vale por sus *flechas*: que nadie quede atrapado y que ninguna pantalla quede huérfana.
- El objetivo último del diseño de la interacción es el **engagement**: ganar la atención, un recurso escaso. Por eso es una **economía**.

- Pregunta siempre cuál es tu **moneda social**: qué das a cambio de la atención que pides.
- La **segunda ola** convierte la atención en moneda simbólica: la atención *calcificada* (seguidores, likes) se acumula y se cambia por dinero, influencia o estatus.
- La **IA** es el motor de este mecanismo (hiperpersonalización, predicción); sus abusos (cámaras de eco, *dark patterns*) son la negación de los principios de buena interacción.

Elaborado a partir de los materiales de la asignatura y de: R. Kristoff y A. Satran, Interactivity by Design*; la definición de interacción de la IxDA y las cinco dimensiones de G. Crampton Smith y K. Silver; T. Davenport y J. C. Beck,* The Attention Economy*; H. Simon sobre la pobreza de atención; M. Heitmayer, "The Second Wave of Attention Economics" (*Interacting with Computers*, 2025); y literatura divulgativa sobre IA y economía de la atención.*

TEMA 5

Análisis y diseño de proyectos digitales

5.1. Sentido de la sesión

Ya tienes una idea de producto digital y la has trabajado con el *Lean Startup Canvas*: sabes a quién te diriges, qué problema resuelves y cuál es tu propuesta de valor. Ahora llega la pregunta incómoda: **¿cómo conviertes esa idea en algo que funciona de verdad?** Ese es el salto que da esta sesión. Entre la idea y el producto hay dos fases imprescindibles —el **análisis** y el **diseño**— y una decisión transversal: la **metodología** con la que vas a organizar el trabajo del equipo.

Estos apuntes están pensados para que, cuando termines, seas capaz de redactar la *especificación* de tu proyecto de prácticas (lista de funcionalidades y requisitos) y de justificar, con criterio, qué metodología vas a usar para llevarlo a cabo.

Idea central

Antes de programar nada hay que responder a tres preguntas, y en este orden: **¿qué hay que hacer?** (análisis), **¿cómo lo vamos a hacer?** (diseño) y **¿de qué forma nos organizamos para construirlo?** (metodología). Saltarse el análisis y ponerse a "picar código" es la causa más frecuente de proyectos que acaban mal.

5.1.1. Qué deberías saber hacer al terminar

- Distinguir una metodología tradicional (en cascada) de una ágil, y saber **cuándo conviene cada una**.
- Explicar con tus palabras qué es Scrum: sus roles, sus eventos y sus herramientas.
- Redactar **funcionalidades**, **requisitos** e **historias de usuario** bien formadas y verificables.
- Conocer las herramientas de diseño y prototipado, ordenadas de menor a mayor fidelidad.

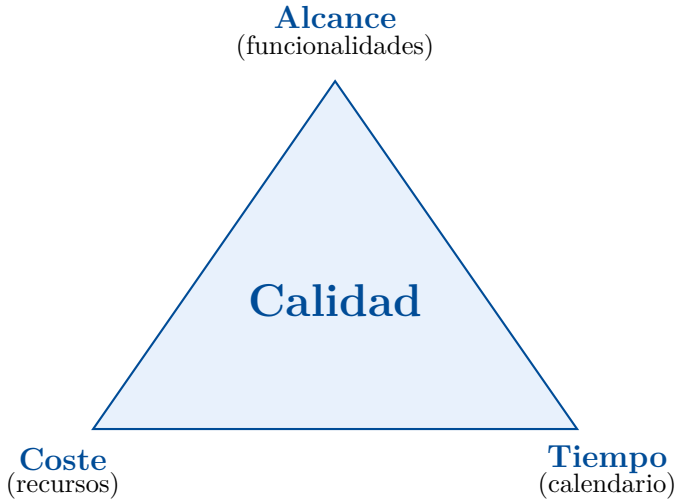
- Distinguir una decisión **arquitectónica** de una decisión técnica cualquiera, y resolver las cuatro preguntas de arquitectura de un proyecto multimedia.
- Dibujar un **diagrama de componentes** con sus conexiones etiquetadas y su frontera de confianza.
- Elegir y justificar una metodología para tu proyecto de prácticas.

5.2. ¿Qué es un proyecto?

Un proyecto es un **esfuerzo temporal** que se emprende para crear un producto, un servicio o un resultado nuevo. Que sea "temporal" es lo que lo diferencia de la operación rutinaria de una empresa: un proyecto tiene principio y final. Para que salga adelante necesita una visión y unos objetivos claros, unos requisitos, una planificación y, sobre todo, recursos: personas (el equipo), equipamiento y tecnología, tiempo y dinero.

5.2.1. El triángulo de hierro

Esos recursos no son independientes entre sí. La gestión de proyectos los representa con el llamado *triángulo de hierro*, cuyos tres vértices son el **alcance** (las funcionalidades que se ofrecen), el **coste** (recursos y presupuesto) y el **tiempo** (el calendario). En el centro queda la **calidad**, que depende del equilibrio entre los tres.



Idea clave: la manta corta

Los tres vértices están atados. Si quieres **más alcance** sin tocar el tiempo ni el presupuesto, la calidad cae. Si recortas el **tiempo**, tendrás que recortar alcance o subir el coste. Gestionar un proyecto es, en gran medida, negociar estas tres variables sin engañarse: no se puede tener todo a la vez.

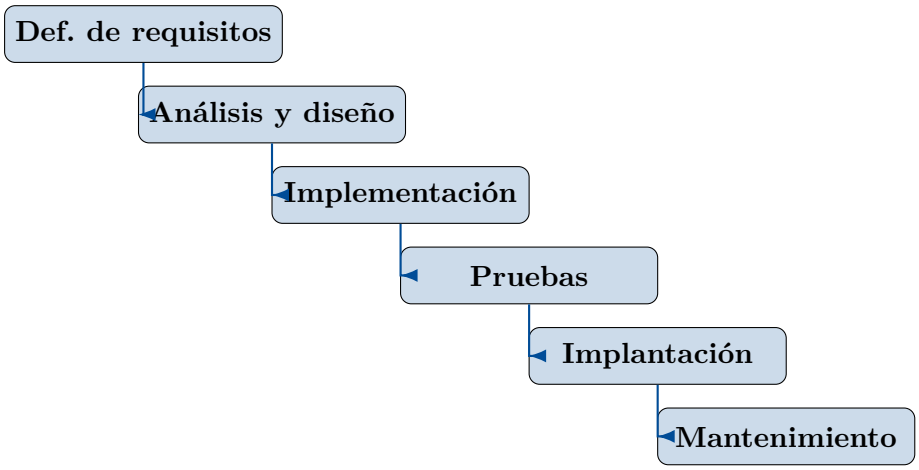
5.3. Metodologías de gestión de proyectos

Una metodología es la forma de organizar el trabajo de un equipo a lo largo del proyecto. No hay una "mejor" metodología en abstracto: la buena elección **depende de la naturaleza del proyecto**, sobre todo de dos factores: cuántos **cambios** esperamos en los requisitos y con qué frecuencia haremos **entregas**.

	Pocos cambios	**Muchos cambios**
Entrega única	Modelo en cascada (Waterfall)	Riesgo: requisitos volátiles con plan rígido
Entregas frecuentes	Incremental "ligero"	Métodos ágiles (Scrum, XP, Kanban)

5.3.1. Metodología tradicional: el modelo en cascada

El modelo en cascada (*Waterfall*) plantea el proyecto como una secuencia de fases en la que cada una empieza cuando termina la anterior, como el agua que cae de un escalón al siguiente.



En la primera fase se averigua qué quiere el cliente y cuáles son los objetivos. Después se decide qué se va a hacer y cómo (análisis y diseño), se implementa lo que el analista entendió que el cliente quería, se prueba que el sistema hace lo que debía, se instala y entrega el producto, y finalmente se mantiene resolviendo los problemas que surgen en su uso.

Problemas del modelo en cascada

El cliente **no siempre sabe lo que quiere** al principio, así que los requisitos quedan ambiguos. Como hay poco contacto con él durante el desarrollo, **los errores se detectan tarde**, cuando ya cuestan caros de arreglar. Añade a eso la comunicación pobre dentro del equipo, una documentación que envejece y es costosa de mantener, y poca flexibilidad ante cualquier cambio. Funciona bien cuando los requisitos son estables y conocidos; mal cuando el terreno se mueve.

5.3.2. Metodologías ágiles

A finales de los años 90, los equipos de proyectos pequeños y medianos reaccionaron contra ese enfoque "pesado": no querían perder meses en una planificación inicial mientras el cliente esperaba a ciegas. Las metodologías ágiles **priorizan las entregas rápidas** y están pensadas para proyectos donde **los requisitos cambian durante el desarrollo**. Sutherland, Schwaber y otros formalizaron en esos años el primer marco Scrum para el desarrollo ágil de software.

El espíritu del Manifiesto Ágil

Lo ágil se resume en unas pocas ideas: satisfacer al cliente con **entregas tempranas y continuas** de algo que funcione; aceptar los **cambios** de requisitos como una ventaja competitiva, aunque lleguen tarde; que negocio y desarrolladores **trabajen juntos a diario**; construir el proyecto en torno a **personas motivadas** a las que se da confianza; medir el progreso por el **producto que funciona**, no por la documentación; mantener un **ritmo sostenible**; y cuidar la **excelencia técnica** y la **simplicidad**. Las mejores soluciones emergen de equipos que se auto-organizan.

Sobre estos valores se asientan unos principios prácticos: participación del cliente durante todo el desarrollo; foco en eficacia y eficiencia; entrega incremental por iteraciones; orientación a las personas más que a los procesos; aceptación de que los requisitos cambiarán; y simplicidad, entendida como eliminar complejidad innecesaria.

5.3.3. Scrum

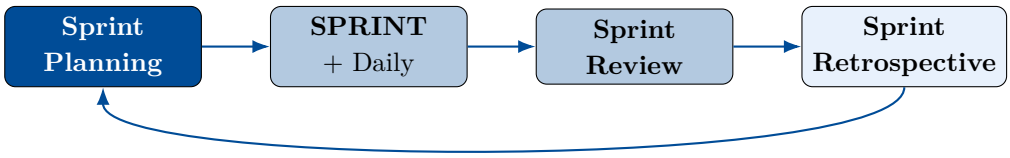
Scrum es el marco ágil más extendido. Se apoya en varios **pilares**: es un modelo iterativo e incremental (se dispone de producto utilizable desde el principio y los errores se detectan pronto); incluye reuniones diarias para controlar la evolución y anticiparse a los problemas; deja que **el cliente defina las prioridades** para maximizar el retorno de la inversión (primero lo de menor coste y mayor beneficio); detalla solo lo que se va a implementar a continuación (definición de bajo nivel progresiva); y se sostiene sobre un equipo con claridad, compromiso y respeto.

Roles

Rol	Responsabilidad
Product Owner	Dueño del producto. Define y prioriza requisitos y funcionalidades. Debe estar muy implicado; representa la voz del cliente.
Development Team	Equipo auto-organizado, autónomo y multifuncional, sin jerarquías, dispuesto a dar feedback y rectificar. Tamaño habitual: entre 3 y 9 personas.
Scrum Master	Aconseja, organiza y vela por que Scrum se aplique bien. Responsable de plazos y reuniones, soluciona impedimentos y apoya al Product Owner en la priorización.
Otros (stakeholders)	Interesados, responsables, usuarios y testers que orbitan alrededor del equipo.

Eventos

El trabajo se organiza en **sprints**: iteraciones de desarrollo de 2 a 4 semanas durante las cuales no se cambian los objetivos. Alrededor del sprint giran el resto de eventos.



- **Sprint Planning** (antes de cada sprint, <8 h): se decide qué puede entregarse en el incremento y cómo se hará el trabajo.
- **Daily** (diario, ∼15 min): cada miembro responde a tres preguntas —qué hice ayer, qué haré hoy y qué impedimentos veo— siempre en relación con el objetivo del sprint.
- **Sprint Review** (al final, <4 h): reunión informal en la que el equipo hace una *demo* del producto terminado, recibe feedback y se habla de qué hacer a continuación.
- **Sprint Retrospective** (tras la review, <3 h): el equipo inspecciona cómo fue el sprint, identifica qué salió bien y qué mejorar, y crea un plan de mejora de su forma de trabajar.

Herramientas

Scrum se gestiona con varias listas. El **Product Backlog** recoge los requisitos de alto nivel del proyecto; nunca está completo y puede cambiar. El **Sprint Backlog** contiene las tareas (requisitos, historias de usuario) seleccionadas para el sprint en curso. La **lista de incremento** reúne lo que se ha dado por terminado durante el sprint. A esto se suman un backlog de imprevistos y, con frecuencia, un tablero Kanban para visualizar las tareas.

Ventajas. Bien aplicado, Scrum mejora la calidad (menos errores en los entregables), aumenta el compromiso del equipo, acorta los ciclos de producción y mejora la productividad al asignar mejor los recursos.

La definición de "terminado"

"Casi terminado" no es terminado

Una tarea "casi terminada" puede suponer todavía muchas horas de trabajo. De ahí el conocido **síndrome del 90 %**: los proyectos avanzan deprisa hasta el 90 % y luego se quedan eternamente en ese punto. Por eso el equipo debe acordar, sin ambigüedad, qué significa que algo "está hecho", normalmente asociando a cada tarea unas pruebas que verifiquen que funciona.

No hacer: dar por terminado lo que no lo está

Decir que una tarea está acabada cuando todavía falla genera problemas en cadena: el resto del equipo construye sobre algo inestable, se pierde tiempo y, sobre todo, se erosionan la confianza y la imagen del equipo ante el cliente. Si no pasa sus pruebas, no está terminada.

5.3.4. Kanban

Kanban es un método de gestión del trabajo nacido en Toyota en los años 40 para organizar las cadenas de producción *Just in Time* y los sistemas *Lean*, orientados a producir bajo demanda. En desarrollo se usa muchas veces **combinado con Scrum**, como apoyo visual para organizar las tareas.

Sus principios son: **visualizar** el flujo de trabajo para poder mejorarlo; **gestionar** ese flujo de modo que sea constante e ininterrumpido; hacer las **políticas explícitas**; mantener la **retroalimentación** con reuniones regulares frente al tablero; y **mejorar colaborando** con una visión compartida.

Ejemplo: un tablero Kanban mínimo

Tres columnas: *Por hacer* → *En proceso* → *Hecho* (se pueden añadir más, como *En pruebas* o *Desestimado*). Cada tarjeta lleva referencia, responsable, título, prioridad, descripción, subtareas y fecha límite. Para evitar cuellos de botella se limita el **WIP** (*Work In Progress*): el número de tarjetas que pueden estar a la vez en un mismo estado.

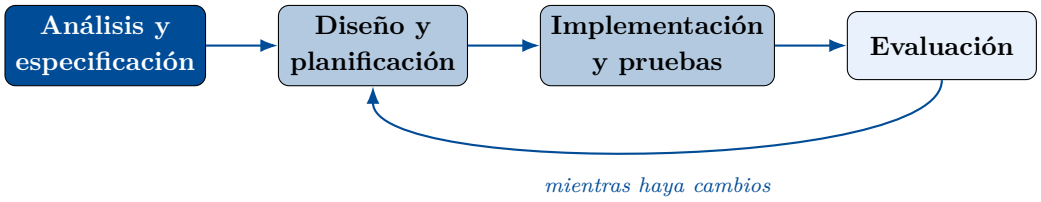
5.3.5. XP: programación extrema

XP pone más énfasis en la **adaptabilidad** que en la previsibilidad: asume que los cambios de requisitos son inevitables y que adaptarse a ellos en cualquier momento es mejor que intentar fijarlo todo al principio. Se apoya en cinco valores —**simplicidad**,

comunicación, **feedback** constante con el cliente, **valentía** ante los cambios y **respeto** en el equipo— y en prácticas características como el desarrollo incremental, las pruebas unitarias continuas, la *programación por parejas* (*pair programming*), la implicación del cliente, la propiedad colectiva del código y revisar todos los errores antes de entregar.

5.3.6. Prototipado e incremental

Entre el modelo rígido y lo ágil hay enfoques iterativos muy útiles para productos multimedia. El **prototipado** consiste en construir versiones preliminares del producto, evaluarlas y ajustarlas según las pruebas, repitiendo el ciclo de diseño–implementación–evaluación hasta lograr un prototipo final robusto; solo entonces se pasa a la producción. El **método incremental** sigue una lógica parecida pero entregando el producto por fases completas y funcionales, volviendo a planificar tras cada incremento.



Por qué iterar reduce el riesgo

Iterar permite **detectar y corregir problemas pronto**, cuando arreglarlos es barato, y asegura que el producto final encaje con lo que el usuario espera, porque este lo va probando por el camino. Es exactamente lo contrario del modelo en cascada, donde el cliente solo ve el resultado al final.

5.3.7. Dudas frecuentes

Conviene anticipar las objeciones típicas. Si el equipo es muy pequeño (una o dos personas), se aplica un **Scrum reducido**. Si preocupa quién paga los cambios, existen los **contratos Scrum**. Lo más habitual hoy no es "Scrum puro" sino **metodologías híbridas**. Para que funcione en tu equipo hacen falta **compromiso** y un buen Scrum Master. Y si no consigues implicar al cliente o no tienes Product Owner, el **prototipado** es una vía para mantener el contacto y validar ideas.

5.4. La fase de análisis

El análisis es la fase inicial donde se determinan los aspectos básicos del proyecto: cuál es su objetivo, qué problema resuelve, qué prestaciones y servicios debe ofrecer, a qué audiencia se dirige, qué necesidades de información y almacenamiento habrá, qué dependencias existen con otros sistemas, qué recursos hay disponibles, cuál es el calendario y a qué plataformas va destinado.

Para obtener toda esa información se usan varias técnicas: **entrevistas** con el cliente o con usuarios potenciales; **observación** del sistema que usan actualmente (ingeniería inversa); estudio del **estado del arte** y de la competencia; y a partir de ahí, listar requisitos, describir funcionalidades por perfiles, redactar historias de usuario y esbozar primeros *wireframes* y diagramas.

Qué entrega el análisis

El resultado del análisis es una **especificación** del proyecto, que normalmente toma una de estas formas (o varias a la vez): un **listado de funcionalidades**, una **relación de requisitos** formal y un conjunto de **historias de usuario**. Esa especificación es la base sobre la que se diseña y se construye todo lo demás.

5.4.1. Funcionalidades

Las funcionalidades dejan claro **cómo se espera que el proyecto funcione desde el punto de vista de los usuarios** (y de los administradores). Para organizarlas hay dos enfoques complementarios: dividir el proyecto en **módulos** y describir las funciones de cada uno, o definir los **perfiles de usuario** y enumerar, para cada perfil, las interacciones que puede realizar.

Ejemplo: una funcionalidad bien descrita

Título: Reservar visita a una bodega.
Perfil: Enoturista registrado.
Descripción: el usuario selecciona una bodega, elige fecha y número de personas y confirma la reserva.
Condiciones / dependencias: el usuario debe haber iniciado sesión; la bodega debe tener plazas disponibles en esa fecha.
Feedback: mensaje visual de confirmación y correo automático con los datos de la reserva.
Cambio de estado: se crea un registro de reserva y se decrementa el aforo disponible de esa franja.

Como ves, para cada funcionalidad conviene precisar las condiciones y dependencias, describir la acción del usuario, especificar el feedback (visual y auditivo) y indicar cómo cambia el estado o los datos del sistema.

5.4.2. Requisitos

Especificar por requisitos consiste en detallar todas las necesidades que el proyecto debe cumplir y las funcionalidades que debe ofrecer para satisfacer los objetivos de los usuarios y las expectativas de los *stakeholders*. Se suelen clasificar en **funcionales** (qué hace), **técnicos** (con qué tecnología), **de diseño y usabilidad** (apariencia, estilo, facilidad de

uso), **de contenido** (información y materiales) y **no funcionales** (accesibilidad, seguridad, rendimiento, mantenimiento).

Anatomía de un buen requisito

Un requisito debe definir una función de forma **clara, específica y medible**. Como mínimo contiene: **título**, **descripción**, **justificación**, **criterios de aceptación** y **tipo/prioridad**. Opcionalmente puede incluir especificaciones técnicas, restricciones y dependencias. Si un requisito no se puede medir ni verificar, no sirve.

5.4.3. Historias de usuario

Las historias de usuario describen una funcionalidad valiosa para el usuario, escrita **en su propio lenguaje** para que él pueda entenderla. Reflejan de forma resumida lo que desea y se componen de tres partes: la descripción escrita, las conversaciones que profundizan en los detalles y los elementos de prueba que determinan cuándo está completa.

Modelo de Mike Cohn

Una plantilla muy usada responde a tres preguntas: *¿qué tipo de usuario?*, *¿qué quiere?* y *¿por qué lo quiere?*

```
Como [tipo de usuario]
quiero [objetivo o accion]
para [beneficio o motivo].

Ejemplo:
Como enoturista
quiero filtrar las bodegas por denominacion de origen
para encontrar rapidamente las que me interesan.
```

5.5. La fase de diseño

El diseño es la fase —previa o simultánea al desarrollo— en la que se toman las decisiones orientadas a la implementación y a la tecnología, y se planifica la ejecución. Incluye elegir la metodología y planificar, diseñar el sistema de información (la base de datos), el esquema de navegación, la **interfaz de usuario** (UX, usabilidad, ergonomía, accesibilidad), las decisiones de implementación (patrones, servicios), el diseño gráfico, los prototipos y el diseño de las pruebas.

5.5.1. Diseño centrado en el usuario (UCA)

En multimedia, la prioridad es la **experiencia del usuario**. El paradigma UCA (*User Centered Analysis*), junto con UCD (diseño) y UT (*user testing*), proporciona métodos para analizar usuarios, metas y contexto (*¿quién?*, *¿qué?*, *¿dónde?*), diseñar con guías y

patrones, y probar la efectividad, la eficiencia y la satisfacción, siempre con el foco puesto en la interfaz.

5.5.2. Herramientas de prototipado

Las herramientas de diseño se ordenan según su **fidelidad** con el producto final, desde lo más abstracto hasta lo más parecido a lo real. Conviene avanzar por esta escala: empezar barato y aumentar el detalle solo cuando las ideas se han validado.

	Herramienta	**Para qué sirve**
+ baja	Mapas de ideas, *brainstorming*	Generar y agrupar ideas
↓	Storyboards o guiones	Contar el flujo de uso paso a paso
↓	Diagramas de tareas / flujo de datos	Estructurar procesos e información
↓	Wireframes	Esquema de contenidos sin estética
↓	Mockups	Prototipos tempranos con aspecto
↓	Prototipos nativos	Versión casi real, interactiva
+ alta	Proyecto final	El producto terminado

Precaución: no te enamores del pixel demasiado pronto

Es tentador saltar directamente a un *mockup* bonito, pero pulir la estética antes de validar la estructura es tiempo perdido si luego hay que rehacerlo. Usa wireframes y bocetos baratos para decidir *qué* va y *dónde*; deja el acabado para cuando esa parte esté estable.

5.6. Arquitectura del sistema

Hasta aquí has decidido *qué* hace el producto (el análisis) y *cómo se ve y se usa* (el diseño de la interfaz). Queda una tercera capa de decisiones que no se ve en ninguna pantalla y que, sin embargo, condiciona todo lo demás: **de qué piezas se compone el sistema, dónde vive cada una y cómo se hablan entre ellas**. Eso es la arquitectura.

Idea central: la arquitectura son las decisiones caras de cambiar

No toda decisión técnica es arquitectónica. Cambiar el color de un botón o el texto de un mensaje es barato: se toca un sitio y ya está. Cambiar dónde se guardan los vídeos, si la lógica corre en el móvil o en un servidor, o de qué proveedor externo dependes, obliga a tocar muchas partes a la vez y a rehacer trabajo ya hecho. **Una decisión es arquitectónica cuando revertirla es caro.** Por eso se toman pronto, con la cabeza fría, y se dejan escritas.

Conviene además situarla frente a lo que ya conoces. Los requisitos dicen *qué* debe hacer el sistema; la arquitectura dice *con qué piezas* lo hará. Un mismo conjunto de

requisitos admite varias arquitecturas posibles, y elegir una u otra no cambia lo que el usuario ve, pero sí cuánto cuesta construirlo, cuánto aguanta creciendo y qué pasa el día que algo falla.

5.6.1. Las cuatro decisiones que todo proyecto multimedia tiene que tomar

No hace falta un tratado. En un proyecto como el vuestro, la arquitectura se resuelve respondiendo con honestidad a cuatro preguntas. Ninguna tiene una respuesta "correcta" universal: cada opción trae una consecuencia, y de lo que se trata es de elegir sabiendo cuál estáis aceptando.

1. ¿Dónde vive la lógica?

O dicho de otro modo: cuando el usuario hace algo, ¿quién decide qué ocurre?

Opción	Cuándo conviene	Qué estás aceptando
Todo en el cliente	Producto que funciona solo, sin cuentas ni datos compartidos (una herramienta local, un juego de un jugador).	Nada es fiable ni compartible: el usuario puede modificarlo todo, y no hay manera de sincronizar entre dispositivos.
Todo en el servidor	Necesitas cuentas, datos compartidos, rankings, pagos o contenido que cambia.	Dependes de la conexión y pagas infraestructura; sin red, el producto no funciona.
Repartida	Lo habitual: el cliente presenta y responde rápido, el servidor decide lo que importa.	Tienes dos sitios donde puede fallar algo, y hay que definir muy bien la frontera.

La frontera no es arbitraria

Lo que **importa** se decide en el servidor: puntuaciones, economía del juego, compras, permisos. Lo que solo afecta a la comodidad —animaciones, previsualizaciones, orden de una lista— puede vivir en el cliente. Es el mismo principio de "nunca te fíes del cliente" que verás en el Tema 8, dicho ahora como decisión de diseño y no como medida de seguridad. Las dos cosas son la misma.

2. ¿Cómo se guarda y se sirve el contenido pesado?

Esta pregunta es propia de lo multimedia y suele olvidarse hasta que duele. Un proyecto de gestión mueve texto y números; el vuestro mueve imágenes, audio, vídeo, modelos y escenas, que pesan órdenes de magnitud más.

- **Empaquetado con la aplicación.** Los assets viajan dentro del instalable. Arranque inmediato y funciona sin red, pero cada cambio de una imagen obliga a publicar una versión nueva, y el tamaño de descarga crece sin freno.
- **Servido bajo demanda.** Los assets se descargan cuando hacen falta. Permite actualizar contenido sin republicar y mantiene pequeño el instalable, a cambio de gestionar esperas, caché y qué ocurre si la descarga falla a la mitad.
- **Subido por los usuarios.** Si vuestro producto acepta ficheros de la gente, tenéis además un problema de tamaño, de formatos, de almacenamiento que crece y de contenido que puede ser ilícito.

Precaución: la base de datos no es un almacén de vídeos

Un error clásico es meter los ficheros pesados dentro de la base de datos porque "así está todo junto". Los ficheros van a un almacenamiento pensado para ficheros; en la base de datos se guarda *la referencia* (la ruta o el identificador) más sus metadatos. Confundirlo hace lenta la base de datos y carísimas las copias de seguridad.

3. ¿De qué terceros dependemos y cómo los aislamos?

Casi ningún proyecto se construye entero: hay un servicio de autenticación, una pasarela de pago, un modelo de IA, un servicio de mapas, un motor de vídeo. Cada dependencia externa es una pieza de vuestro sistema que **no controláis**: puede cambiar su interfaz, subir de precio, degradarse o desaparecer.

Aislar una dependencia

Aislar significa que todo el proyecto habla con el servicio externo a través de un **único punto** vuestro, en lugar de llamarlo desde veinte sitios distintos. Si el proveedor cambia o hay que sustituirlo, se reescribe ese punto y el resto del código ni se entera. Es una decisión de arquitectura que cuesta una hora al principio y ahorra semanas después.

Ejemplo: la remediación del DAFO, dibujada

¿Recuerdas la fila de la tabla de contingencias del Tema 3 que decía *"prevenir: aislar la llamada al servicio externo en un único módulo desde el principio"*? Esto es exactamente eso. Lo que allí era una intención escrita en una tabla, aquí es una caja concreta en el diagrama de componentes. La arquitectura es donde los planes B dejan de ser promesas.

4. ¿Una pieza o varias?

La última pregunta es cuántos "trozos" independientes tendrá vuestro sistema. Un **monolito** es una sola aplicación que lo hace todo; una arquitectura de **servicios** reparte el trabajo en piezas que se despliegan por separado.

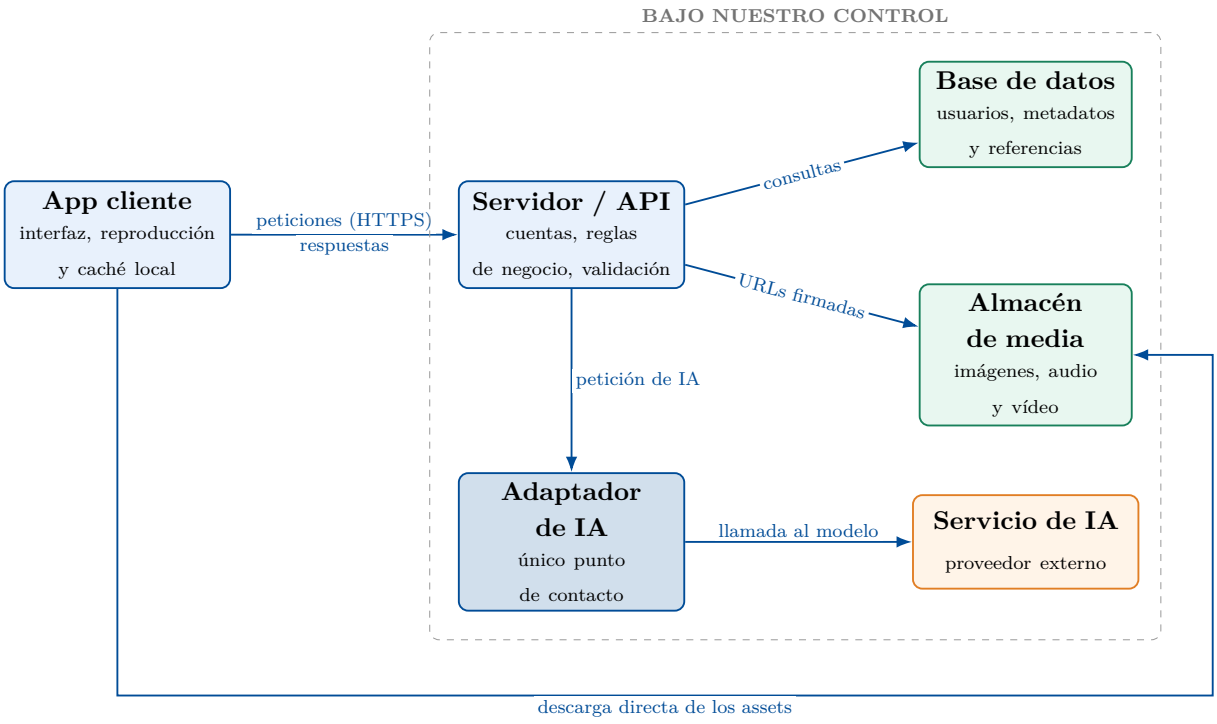
El error más común en un proyecto de curso

Elegir microservicios "porque es lo moderno". Para un equipo de cuatro personas y quince semanas, repartir el sistema en seis servicios multiplica el trabajo de integración, despliegue y depuración sin aportar nada: la ventaja de los servicios aparece cuando hay muchos equipos y mucha escala, y vosotros no tenéis ni lo uno ni lo otro. **Empezad monolíticos y separad solo lo que tenga una razón concreta para vivir aparte** —por ejemplo, un procesado de vídeo que tarda minutos y no puede bloquear al resto—.

5.6.2. El diagrama de componentes

Las cuatro decisiones se cuentan mejor con un dibujo que con tres páginas de texto. Un **diagrama de componentes** muestra las piezas del sistema, dónde se ejecuta cada una y qué viaja por cada conexión. No es un diagrama de clases ni un esquema de base de datos: opera un escalón por encima.

Cada pieza necesita un nombre y una responsabilidad en una línea; cada flecha, una etiqueta que diga *qué* se intercambia. Una flecha sin etiquetar no informa de nada.



Fíjate en lo que el dibujo deja dicho sin una sola frase: la lógica está repartida (decisión 1); los ficheros pesados viven fuera de la base de datos y el cliente los descarga directamente (decisión 2); el proveedor de IA está detrás de un adaptador propio y es el único elemento fuera de la frontera (decisión 3); y el servidor es una sola pieza, no seis (decisión 4).

Advertencia: dibujad la frontera de confianza

La línea discontinua no es decoración. Separa lo que controláis de lo que no, y es la que os dice dónde hay que validar, dónde no pueden estar vuestras claves y qué parte del sistema seguiría funcionando si el proveedor externo se cae. Un diagrama de componentes sin frontera de confianza se queda a medias.

5.6.3. Cómo encaja con lo que ya sabéis

La arquitectura no es un tema aislado: es el lugar donde aterrizan decisiones que ya habéis tomado en otros entregables. Esta es la razón por la que se estudia aquí y no antes.

Viene de	Se materializa en la arquitectura como
Tema 3 (DAFO y contingencias)	Cada dependencia que identificasteis como riesgo necesita un punto de aislamiento en el diagrama. El plan B deja de ser una intención y pasa a ser una caja.
Tema 4 (interacción)	Los momentos críticos del *storyboard* marcan qué debe responder rápido: eso decide qué se resuelve en el cliente y qué puede esperar al servidor.
Tema 7 (la IA en la cadena de valor)	Si la IA es *apoyo*, vive fuera del producto y su caída no lo tumba. Si es *núcleo*, está en el camino crítico y la arquitectura tiene que preverlo: latencia, coste por petición y qué se muestra cuando el modelo falla.
Tema 8 (legal y seguridad)	El servidor autoritativo, los secretos que no bajan al cliente y los datos personales que no salen de vuestra frontera son decisiones arquitectónicas antes que medidas de seguridad.

Síntesis

Una arquitectura bien planteada no es la más sofisticada: es la que un equipo de cuatro personas puede construir en quince semanas, la que explica dónde va cada cosa y la que deja claro qué pasa cuando algo falla. Si vuestro diagrama no responde a esas tres preguntas, todavía no está terminado.

5.7. Actividades

Ejercicio 1 — ¿Cascada, ágil o prototipado?

Para cada encargo, decidid qué metodología encaja mejor y justificadlo con los dos factores del tema: cuántos **cambios** esperáis en los requisitos y con qué **frecuencia** vais a entregar.

1. Digitalizar el catálogo de un museo: 400 fichas con una estructura fija que el cliente ya tiene definida y aprobada; una sola entrega en junio.
2. Una app de rutas urbanas para un ayuntamiento que quiere ver resultados cada tres semanas y que ya ha cambiado de idea dos veces.
3. Un videojuego de mecánica novedosa: el equipo todavía no sabe si el núcleo jugable resultará divertido.
4. Migrar una web corporativa a un gestor de contenidos nuevo, con el mismo contenido y el mismo diseño.

Solución orientativa: 1 y 4, cascada —requisitos estables y entrega única—; 2, ágil —cambios frecuentes y entregas cortas—; 3, prototipado: la incógnita no son los requisitos, sino si la idea funciona, y eso solo se averigua construyendo algo y probándolo.

Ejercicio 2 — De la buena intención al requisito verificable

Ninguna de estas cinco frases sirve como requisito. Reescribid cada una añadiendo lo que le falta y, sobre todo, un **criterio de aceptación** que permita responder "sí" o "no" sin discutir:

1. "La aplicación debe ser rápida."
2. "El usuario podrá gestionar su perfil."
3. "La interfaz será intuitiva y moderna."
4. "El sistema soportará muchos usuarios."
5. "Habrá una pantalla de búsqueda."

Ejemplo resuelto (1): "El listado de resultados se muestra en menos de 1,5 s con 10.000 registros en la base de datos, medido en un móvil de gama media con conexión 4G." Ahora es medible; antes era un deseo.

Ejercicio 3 — Historias de usuario que fallan

Estas tres historias tienen la plantilla bien puesta, pero están mal formuladas. Identificad el fallo de cada una y corregidla:

1. *Como usuario, quiero una base de datos MySQL, para guardar los datos.*
2. *Como administrador, quiero gestionar el sistema, para que funcione bien.*
3. *Como cliente, quiero un botón azul en la esquina superior derecha, para pulsarlo.*

Solución: (1) describe una decisión técnica, no un valor para el usuario; (2) "gestionar el sistema" es tan amplia que nunca se podrá dar por terminada ni probar; (3) impone la solución de interfaz en lugar de expresar la necesidad. Una historia útil

dice *quién*, *qué quiere conseguir* y *por qué le importa*, y deja el "cómo" para el diseño.

Ejercicio 4 — Las cuatro decisiones, sobre un caso

Una app para descubrir rutas de senderismo: mapas descargables para usar sin cobertura, fotos y reseñas subidas por los propios usuarios, cuentas con rutas favoritas sincronizadas entre móvil y web, y una función que genera una descripción de la ruta con IA a partir de las fotos. Equipo: cuatro personas, quince semanas.
Responded a las cuatro preguntas —dónde vive la lógica, cómo se guarda y se sirve el contenido pesado, de qué terceros dependéis y cómo los aisláis, y una pieza o varias— y dibujad el diagrama de componentes con su frontera de confianza.
Pistas para discutir en clase: los mapas descargables empujan a que parte del contenido viva *en* el cliente, pero las rutas favoritas sincronizadas obligan a un servidor; las fotos de usuarios no caben en la base de datos y traen un problema de contenido ajeno; la IA aquí es *apoyo* —genera un texto acompañante, no es el producto—, así que la app tiene que seguir funcionando si el modelo no responde. ¿Qué mostraríais en ese caso?

Ejercicio 5 — La primera ficha de vuestro proyecto

En equipo, elegid **la funcionalidad más importante** de vuestro producto —aquella sin la cual el proyecto no tiene sentido— y completad para ella la ficha del apartado de funcionalidades: título, perfil, descripción, condiciones y dependencias, *feedback* y cambio de estado. Después escribid esa misma funcionalidad como historia de usuario. Comparad las dos versiones: ¿cuál entendería mejor un cliente?, ¿cuál necesitaría un programador para implementarla? Esa diferencia es justamente la razón por la que existen las dos.

5.8. Aplicación al proyecto del curso

El análisis funcional y la elección razonada de una metodología permiten trasladar estas ideas al proyecto de la asignatura. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

5.9. Ideas clave para recordar

- Antes de programar: **análisis** (qué) → **diseño** (cómo) → **metodología** (cómo nos organizamos).
- No hay metodología "mejor": elige según los **cambios** esperados y la **frecuencia de entregas**. Pocos cambios y entrega única → cascada; muchos cambios y entregas frecuentes → ágil.

- En **Scrum**: roles (Product Owner, equipo, Scrum Master), eventos (planning, daily, review, retrospective) y herramientas (product/sprint backlog).
- "Casi terminado" **no** es terminado: cuidado con el síndrome del 90 %.
- El análisis produce una **especificación**: funcionalidades, requisitos e historias de usuario, todos **medibles y verificables**.
- En diseño, prioriza la **experiencia de usuario** (UCA) y sube por la escala de fidelidad solo cuando la idea está validada.
- Una decisión es **arquitectónica** cuando revertirla es caro. Por eso se toma pronto y se deja escrita.
- Cuatro preguntas resuelven la arquitectura de un proyecto multimedia: **dónde vive la lógica**, **cómo se sirve el contenido pesado**, **de qué terceros dependéis y cómo los aisláis** y **una pieza o varias**.
- Lo que importa se decide en el **servidor**; los ficheros pesados no van dentro de la base de datos; cada dependencia externa se aísla en un único punto; y un equipo pequeño empieza **monolítico**.
- El **diagrama de componentes** necesita flechas etiquetadas y una **frontera de confianza**: sin ellas no dice dónde validar ni qué pasa si un tercero se cae.

Apuntes elaborados a partir de las transparencias de la asignatura Sistemas Multimedia (DLSI, Universidad de Alicante), bloques "Metodologías de gestión de proyectos" y "Análisis y diseño". Referencia adicional: Understanding the SDLC *(plutora.com).*

TEMA 6

El diagrama de Gantt y cómo aplicarlo

6.1. Sentido de la sesión

Cuando un proyecto crece, una simple lista de tareas se queda corta. La lista te dice *qué* hay que hacer, pero no *cuándo*, ni *en qué orden*, ni *cuánto dura* cada cosa, ni *quién* se encarga. En este tema vas a planificar un proyecto multimedia en el tiempo, y para eso necesitas una de las herramientas de planificación visual más extendidas: el diagrama de Gantt.

Idea central

Un diagrama de Gantt es un gráfico de barras que coloca las tareas de un proyecto sobre una línea de tiempo. Su poder está en algo muy sencillo y muy valioso: **hacer visible el trabajo en el tiempo**. Convierte una lista dispersa de actividades en un cronograma comprensible, coordinado y controlable.

6.1.1. Qué deberías saber hacer al terminar

- Explicar con tus palabras qué es un Gantt y para qué sirve.
- Leer un diagrama e interpretar tareas, duraciones, hitos y dependencias.
- Preparar la planificación de un proyecto antes de dibujarlo (fases, tareas verificables, responsables, fechas).
- Construir el cronograma paso a paso y elegir la herramienta adecuada.
- Usar el Gantt para hacer seguimiento y reaccionar ante cambios.
- Elaborar un cronograma completo y coherente.

6.2. Fundamentos: qué es y para qué sirve

Un diagrama de Gantt (también llamado *gráfica de Gantt*, *carta Gantt* o *cronograma*) representa cada tarea del proyecto mediante una barra horizontal situada sobre un eje temporal. De un vistazo se ve qué ocurre en cada momento, desde el inicio hasta la finalización del proyecto.

Idea clave: de la lista al cronograma

Si una lista de tareas responde a **qué** hay que hacer, un Gantt responde además a **cuándo** hay que hacerlo, **en qué orden**, **cuánto dura** cada tarea y, muchas veces, **quién** se encarga de ella.

6.2.1. Las tres funciones del Gantt

El diagrama cumple tres funciones complementarias que conviene no confundir:

Función	En qué consiste
Planificación	Decidir qué tareas deben realizarse.
Programación	Situar esas tareas en fechas concretas.
Seguimiento	Comprobar avances, retrasos y cumplimiento de plazos.

Ejemplo: el proyecto como un viaje

Imagina el proyecto como un viaje largo con varias paradas. Una lista te dice cuáles son las paradas; el Gantt te muestra además el calendario del viaje, cuánto dura cada tramo, qué trayectos dependen de otros y quién conduce cada parte. En la construcción de un edificio ocurre igual: si se retrasa el diseño estructural, se retrasa la cimentación y eso arrastra a las fases siguientes. El Gantt deja ver esa secuencia temporal de un solo golpe.

6.2.2. Estructura básica

Como mínimo, todo Gantt se compone de cinco piezas:

- **Eje vertical:** la lista de tareas (y, si conviene, fases y subtareas).
- **Eje horizontal:** la escala temporal (horas, días, semanas o meses).
- **Barras horizontales:** indican cuándo empieza, cuándo termina y cuánto dura cada tarea.
- **Hitos:** puntos clave del proyecto (revisiones, entregas).
- **Dependencias:** relaciones de orden entre tareas.

La regla para elegir la escala temporal es sencilla: proyecto corto → días; proyecto largo → semanas o meses. Las barras pueden enriquecerse con el porcentaje completado, las fechas previstas frente a las reales o una marca de criticidad.

6.3. Un poco de historia

El Gantt no nació de golpe ni de una sola persona. Hay tres referencias que conviene situar para entender de dónde viene.

Época	Autor	Aportación
Siglo XVIII	Joseph Priestley	Ilustraciones cronológicas: un antecedente visual de representar procesos en el tiempo.
1896–1903	Karol Adamiecki	El ingeniero polaco desarrolla el *Harmonogram*: ya organiza tareas con barras temporales, pero publica en polaco y ruso, lo que limita su difusión.
1910–1915	Henry Gantt	El ingeniero estadounidense desarrolla y difunde el método, lo consolida como herramienta de gestión y acaba dándole nombre.

Idea para recordar

Adamiecki formuló antes la idea de las barras temporales, pero fue Gantt quien la difundió internacionalmente. La herramienta lleva su nombre por su impacto, no por ser el primero.

6.4. ¿Por qué sigue vigente hoy?

Aunque es un método clásico, el Gantt sigue plenamente vigente porque responde a necesidades que no han desaparecido: coordinar equipos, ordenar tareas, prever retrasos, comunicar fechas y controlar avances. Su principio básico —hacer visible el trabajo en el tiempo— nunca pasa de moda.

Del papel al software

Un Gantt puede dibujarse a mano, y hacerlo ayuda a entender su lógica. Pero el método manual consume tiempo, dificulta las actualizaciones y resulta poco práctico ante cambios. Por eso hoy se trabaja casi siempre en digital, donde el cronograma puede integrarse con responsables, costes, riesgos y colaboración en tiempo real. El Gantt no sustituye toda la gestión del proyecto, pero es una pieza muy útil para organizar y comunicar el calendario.

6.5. Elementos esenciales y cómo leer un Gantt

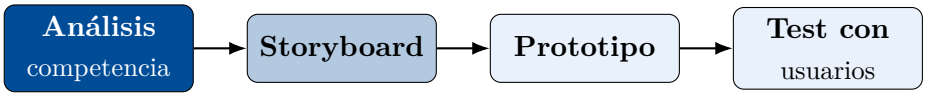
Leer un Gantt no consiste en mirar barras, sino en interpretar estructura, secuencia, relaciones, responsables y estado del proyecto. Estas son las piezas que debes reconocer:

- **Fase:** bloque grande del proyecto.
- **Tarea:** trabajo concreto dentro de una fase.
- **Subtarea:** paso más específico dentro de una tarea.
- **Barra:** representa la tarea en el tiempo; su posición indica inicio y fin, su longitud indica la duración.
- **Escala temporal:** la graduación del eje horizontal.
- **Hito:** momento clave del proyecto.
- **Dependencia:** una tarea condiciona el inicio o el fin de otra.
- **Responsable:** persona o equipo que responde por cada tarea.

Tarea frente a hito

No los confundas: una **tarea** tiene duración (ocupa tiempo en el eje); un **hito** marca un momento clave pero no consume tiempo prolongado (se dibuja como un punto, no como una barra).

Las dependencias son la información más rica de un Gantt: indican que algo no puede empezar hasta que otra cosa termine. Un ejemplo típico en tu proyecto:



Ejemplo: leer un cronograma sencillo

Supón un proyecto académico con dos fases. En *Investigación*, el "Análisis de competencia" dura 14 días y las "Encuestas a usuarios" arrancan antes de que aquel termine: se solapan. En *Diseño*, el "Storyboard" depende del análisis, y el "Prototipo" depende del storyboard. Leyendo el diagrama puedes responder de un vistazo: ¿qué tareas están activas ahora?, ¿cuáles van en paralelo?, ¿qué depende de qué?, ¿quién responde por cada parte?, ¿qué hito está próximo?

6.5.1. Vista de alto nivel y vista detallada

No siempre interesa el mismo nivel de detalle. Conviene manejar dos vistas y saber cuándo usar cada una:

Vista	Qué muestra	Para qué sirve
Alto nivel	Fases principales, hitos y fechas clave.	Informar a clientes, presentar el proyecto, dar el panorama general.
Detallada	Tareas, subtareas, responsables, dependencias y % de avance.	Gestionar el trabajo diario y coordinar al equipo.

6.5.2. Cómo organizar la información para que sea legible

Buenas prácticas de legibilidad

1. Agrupa las tareas por fases.
2. Nómbralas con claridad (“Redacción de textos”, no “Avanzar contenidos”).
3. Elige una escala temporal adecuada a la duración del proyecto.
4. Muestra solo el detalle necesario en cada vista.
5. Haz visibles los responsables.
6. Señala hitos y dependencias importantes.
7. Mantenlo actualizado y evita la sobrecarga visual.

El error más común

Convertir el Gantt en “una lista bonita”. Si solo aparecen tareas y fechas pero no se distinguen duraciones, solapamientos ni dependencias, el diagrama pierde casi todo su valor. Un Gantt sin relaciones temporales es una agenda maquillada, no una herramienta de gestión.

6.6. Antes del diagrama: preparar la planificación

Aquí está el error de fondo de muchos proyectos: dibujar el gráfico antes de pensar el plan. Si el proyecto está mal definido, las tareas serán vagas, las fechas irreales y las dependencias confusas; el Gantt resultante será poco útil.

Idea central

Antes de construir el gráfico, hay que construir el plan. Es decir, decidir qué trabajo hay que hacer, quién responde de cada parte, cuándo debería empezar y terminar y qué momentos servirán de control.

6.6.1. Descomponer: de lo general a lo concreto

Piensa el proyecto en tres niveles: **fases** (grandes bloques), **tareas** (piezas concretas) y **subtareas** (divisiones más pequeñas, solo cuando aporten claridad). Una tarea debe ser *concreta y verificable*: tienes que poder responder con claridad si está hecha o no.

Mala formulación (vaga)	Buena formulación (verificable)
"Trabajar en el proyecto"	"Preparar la encuesta a usuarios"
"Avanzar diseño"	"Aplicar la encuesta a 20 usuarios y recopilar respuestas"
"Trabajar contenidos"	"Diseñar el storyboard del prototipo"

No hacer

Tareas como "Avanzar con el diseño" o "Trabajar en contenidos" no se pueden planificar ni seguir: nunca sabrás si están terminadas. Reformúlalas siempre hasta que sean comprobables.

6.6.2. Asignar responsables sin repartir al azar

Cada tarea necesita un **responsable principal único**. Puede haber colaboradores, pero debe existir una persona de referencia; así se evita el clásico "pensábamos que lo hacía otro". El reparto se apoya en cinco criterios: especialización, disponibilidad, equilibrio de carga, continuidad lógica y facilidad de seguimiento.

Qué evitar al asignar

Poner "todo el equipo" como responsable, repartir sin mirar la disponibilidad, concentrar varias tareas críticas en una sola persona o asignar trabajo sin relación con las capacidades de cada cual.

6.6.3. Fechas realistas, dependencias y solapamientos

Planificar con realismo significa considerar la disponibilidad real, la carga de trabajo, las dependencias, los imprevistos y el tiempo de revisión. No todas las tareas pueden empezar cuando queramos: el storyboard depende del análisis, el prototipo del storyboard, el vídeo de los textos y las fotos.

Dos tareas pueden **solaparse** si no hay dependencia directa entre ellas y los responsables tienen capacidad; **no deben** solaparse si una depende de la otra o si recaen sobre la misma persona sin margen.

6.6.4. Identificar hitos de control

Un buen hito es relevante, fácil de identificar, útil para tomar decisiones y está suficientemente espaciado de los demás. Ejemplos en tu proyecto: cierre de la investigación,

revisión del prototipo con el profesor, primera versión del estado del arte, ensayo de la presentación y entrega final.

6.7. Construcción paso a paso

Construir el Gantt es trasladar la planificación a una línea de tiempo visual. El proceso, una vez tienes el plan, sigue siempre la misma secuencia:

Los siete pasos

1. Parte de la lista de tareas y de sus fechas (inicio/fin o duración).
2. Elige construcción manual o digital.
3. Configura la herramienta: escala temporal, orden por fases y campos visibles.
4. Coloca las barras según inicio y fin, y revisa los solapamientos.
5. Identifica y establece las dependencias críticas.
6. Refleja visualmente esas dependencias (flechas o conexiones).
7. Personaliza el formato para comunicar mejor, sin recargar.

6.7.1. Manual o digital

Opción	Ventajas	Inconvenientes
Manual	Ayuda a comprender la lógica; útil para bocetos y aprendizaje.	Actualizarlo cuesta; poco práctico si hay muchos cambios.
Digital	Actualización sencilla, mejor presentación, fácil de compartir y de gestionar cambios.	Requiere elegir y aprender una herramienta.

6.7.2. Reflejar las dependencias

Las dependencias se muestran con flechas o conexiones entre barras. Conviene verlas con claridad porque condicionan el orden de todo el proyecto.

Analogía de la receta

Una dependencia funciona como en una receta de cocina: no puedes hornear antes de mezclar, ni decorar antes de hornear, ni servir antes de decorar. El orden no es opcional.

Errores frecuentes al construir

Ocultar los solapamientos (da una imagen irreal del trabajo), omitir las dependencias críticas (impide anticipar retrasos en cadena), abusar de colores y etiquetas, o elegir una escala temporal inadecuada (demasiado detalle hace el gráfico inmanejable; demasiado poco lo hace inútil).

6.8. Herramientas y entornos de creación

No hay una única herramienta correcta. La elección depende de la complejidad del proyecto, la frecuencia de cambios, el tipo de colaboración y a quién va dirigido el cronograma. Hay tres grandes familias:

Familia	Cuándo conviene	Limitación principal
Software de gestión (GanttPRO, TeamGantt, GanttProject, Merlin Project)	Proyectos con muchas tareas, cambios frecuentes y seguimiento continuo.	Más especializado; curva de aprendizaje.
Herramientas de diagramación (Lucidchart)	Presentaciones, documentación, explicación visual del plan.	Menos orientadas a la gestión diaria.
Hojas de cálculo (Excel, Google Sheets)	Proyectos pequeños o académicos; equipos que ya usan tablas.	Mantenimiento más manual y tedioso.

Regla práctica

Manual para *entender, esbozar y enseñar*; digital para *mantener, actualizar, compartir y colaborar*. Y un principio que no falla: la mejor herramienta no es la más sofisticada, sino la que te permite crear, mantener y comunicar el cronograma de forma clara y útil.

Para un trabajo de clase como el del Entregable 6 suele bastar una hoja de cálculo compartida o una herramienta sencilla como TeamGantt o GanttProject. Si el equipo va a revisar el avance con frecuencia, inclínate por software de gestión; si lo más importante es la presentación final, una herramienta de diagramación.

Errores al elegir herramienta

Usar una herramienta muy compleja para un proyecto simple; trabajar a mano sabiendo que habrá muchos cambios; elegir algo “bonito” pero poco práctico para el seguimiento; o no actualizar el diagrama aunque la herramienta lo permita.

6.9. Seguimiento, control y gestión de cambios

Un Gantt no sirve solo para planificar al principio. Su verdadero valor aparece cuando se usa durante el proyecto para comparar lo previsto con lo real, detectar desviaciones y reajustar el plan.

Planificado frente a real

El seguimiento compara dos planos. El **planificado**: fechas previstas, duración estimada y responsable asignado. El **real**: fecha real de inicio y fin, porcentaje completado e incidencias. Como mínimo, actualiza tareas terminadas, tareas en curso, % de avance, fechas reales e hitos alcanzados.

Analogía del mapa antiguo

Planificar con un Gantt y no actualizarlo es como usar un mapa antiguo en un viaje largo: sirve al principio, pero deja de ayudar (e incluso engaña) si no incorpora los cambios del terreno.

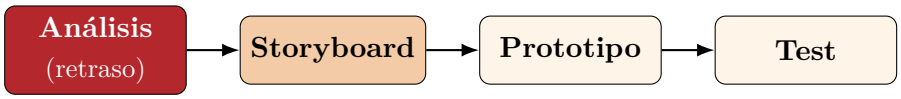
6.9.1. Qué vigilar: recursos, riesgos y cuellos de botella

Señales de alerta en el cronograma

Sobrecarga: una misma persona con varias tareas largas solapadas en la misma semana. **Riesgo temporal:** tareas muy largas, dependencias encadenadas o hitos cercanos sin margen. **Cuello de botella:** una tarea, una persona o una aprobación de la que dependen muchas otras y que limita todo el avance.

6.9.2. Retrasos y efecto dominó

La mayor utilidad del seguimiento es ver cómo un retraso se propaga a las tareas dependientes:



Ante un retraso, hazte cinco preguntas: ¿la tarea dependiente puede empezar igualmente?, ¿hay margen temporal?, ¿afecta a un hito importante?, ¿compromete la entrega final?, ¿hay que mover fechas o responsables? Según la respuesta, podrás mover el inicio de las tareas dependientes, ajustar el fin de una fase, reasignar trabajo o revisar la secuencia.

Reajuste concreto

Las encuestas debían terminar el 10/03 pero acaban el 14/03, y el storyboard necesitaba esos datos. Decides retrasar el storyboard al 15–22/03 y, en cadena, mover el prototipo al 23/03–05/04. Además valoras apoyar a la responsable del

storyboard, que esa semana tiene otra entrega. El Gantt te ha servido para ver el impacto y decidir.

6.10. Ejemplo completo: desarrollo de un producto multimedia

El ejemplo siguiente reúne los principales elementos estudiados en el tema y muestra cómo convertir la planificación de un proyecto multimedia en un Gantt coherente.

Título: Desarrollo de una aplicación multimedia interactiva.
Periodo: del 15/02/2025 al 30/05/2025.

Cód.	Fase	Tarea	Resp.	Inicio	Fin
T1.1	Investigación	Análisis de competencia	María	15/02	28/02
T1.2	Investigación	Encuestas a usuarios potenciales	Carlos	20/02	10/03
T1.3	Investigación	Recopilación de referencias y assets	Lucía	15/02	05/03
T2.1	Diseño	Definición del concepto del producto	Ana	01/03	08/03
T2.2	Diseño	Diseño del storyboard	Ana	09/03	15/03
T2.3	Diseño	Creación del prototipo	Luis	16/03	30/03
T2.4	Diseño	Test con usuarios	Carlos	31/03	07/04
T3.1	Contenidos	Redacción de textos	Marta	20/03	05/04
T3.2	Contenidos	Producción fotográfica	Lucía	22/03	06/04
T3.3	Contenidos	Edición del vídeo promocional	Diego	07/04	20/04
T3.4	Contenidos	Integración de contenidos en el prototipo	Luis	21/04	30/04
T4.1	Validación y cierre	Revisión global del proyecto	María	01/05	08/05
T4.2	Validación y cierre	Correcciones finales	Ana	09/05	18/05
T4.3	Validación y cierre	Preparación de la presentación	Marta	19/05	25/05
T4.4	Validación y cierre	Entrega final	María	30/05	30/05

Esta tabla, con sus 15 tareas, es la base a partir de la cual la herramienta dibuja las barras del Gantt. Las dependencias críticas y los hitos se documentan aparte para que se lean de un vistazo:

Tarea dependiente	Depende de
T2.1 Definición del concepto	T1.1, T1.2, T1.3
T2.2 Diseño del storyboard	T2.1
T2.3 Creación del prototipo	T2.2
T3.3 Edición del vídeo	T3.1 y T3.2
T3.4 Integración de contenidos	T2.3 y T3.3

Hitos: cierre de investigación (01/03), revisión del concepto y storyboard (20/03), fin del test con usuarios (07/04), revisión global completada (08/05) y entrega final (30/05).

6.10.1. Plan de contingencia básico

No hace falta un análisis de riesgos complejo: basta con identificar algunos riesgos operativos, su impacto y una respuesta razonable.

Riesgo	Impacto posible	Respuesta básica
Retraso en una tarea crítica	Desplaza tareas dependientes y hitos.	Reprogramar el Gantt el mismo día y priorizar la tarea crítica.
Ausencia temporal de un miembro	Tareas sin seguimiento.	Reasignar la tarea y mantener un responsable único actualizado.
Retraso en la entrega de contenidos	Bloqueo de la integración.	Adelantar tareas no dependientes y reservar margen.
Problemas en revisión o test	Correcciones adicionales.	Añadir una fase breve de ajustes y proteger los últimos días.

Ejercicio guiado: planifica un proyecto

Utiliza este esqueleto para practicar la planificación antes de trasladarla a una herramienta de Gantt:

```
TÍTULO DEL PROYECTO:
PERIODO DE EJECUCIÓN: del __/__/____ al __/__/____

FASES Y TAREAS
  Fase 1: ____ -> T1.1, T1.2, T1.3
  Fase 2: ____ -> T2.1, T2.2, T2.3
  Fase 3: ____ -> T3.1, T3.2, T3.3
  Fase 4: ____ -> T4.1, T4.2

TABLA POR TAREA
  | Tarea | Responsable | Inicio | Fin |
```

```
DEPENDENCIAS CRÍTICAS (3-5)
  - Tarea X depende de Tarea Y

HITOS (4-6)
  - Hito 1: ____ (fecha)

CONTINGENCIAS
  - Riesgo / Impacto / Respuesta
```

Errores que vuelven inútil un Gantt

Un Gantt aporta poca información si contiene muy pocas tareas, deja actividades sin responsable, no muestra hitos ni dependencias o utiliza descripciones vagas como "Trabajar proyecto". El nivel de detalle debe permitir comprender, coordinar y actualizar el plan.

6.11. Aplicación al proyecto del curso

El Gantt se aplicará al proyecto del curso: convertiréis el alcance ya especificado en fases, tareas verificables, responsables, dependencias e hitos, y lo trasladaréis a un cronograma que podáis mantener durante el semestre. Las instrucciones de entrega, las cantidades exigidas, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

6.12. Ideas clave para recordar

- Un Gantt es un gráfico de barras orientado al tiempo: eje vertical = tareas, eje horizontal = tiempo, barras = duración, hitos = puntos clave, dependencias = relaciones de orden.
- Sus tres funciones son planificar, programar y hacer seguimiento.
- Antes de dibujar el gráfico, construye el plan: tareas verificables, un responsable por tarea, fechas realistas, dependencias e hitos.
- Sus ventajas: visibilidad del cronograma, control de dependencias, seguimiento de hitos, previsión de recursos, claridad de responsabilidades y transparencia.
- Sus límites: puede volverse visualmente complejo, exige actualización constante y no encaja igual en todos los proyectos.
- Un Gantt sin actualizar engaña; un Gantt sin dependencias es solo una lista bonita.
- Para dominarlo de verdad: explícalo, léelo, constrúyelo y úsalo para decidir.

6.13. Glosario básico

Barra Representación horizontal de una tarea en el tiempo.

Cronograma
Plan temporal del proyecto.

Dependencia
Relación por la que una tarea condiciona el inicio o el fin de otra.

Duración Tiempo previsto para completar una tarea.

Eje horizontal
Parte del diagrama que representa el tiempo.

Eje vertical
Parte del diagrama que muestra las tareas.

Fase Bloque grande del proyecto que agrupa tareas relacionadas.

Hito Momento clave o punto de control del proyecto.

Responsable
Persona o equipo que responde por una tarea.

Solapamiento
Coincidencia temporal entre dos o más tareas.

Subtarea División más pequeña de una tarea.

Tarea crítica
Actividad especialmente sensible para el calendario general.

6.14. Autoevaluación rápida

Comprueba si lo has entendido

Responde con tus palabras. Si dudas en alguna, vuelve a la sección correspondiente.

1. ¿Qué diferencia hay entre una lista de tareas y un Gantt?
2. ¿Qué representan el eje vertical, el eje horizontal y las barras?
3. ¿En qué se distinguen una tarea y un hito?
4. ¿Qué es una dependencia y por qué condiciona el calendario?
5. ¿Qué pasos hay que dar *antes* de construir el diagrama?
6. ¿Cuándo conviene una hoja de cálculo y cuándo un software de gestión?

7. ¿Por qué debe actualizarse el cronograma durante el proyecto?
8. ¿Qué elementos hacen que un Gantt sea útil para planificar y hacer seguimiento?

Material elaborado para Sistemas Multimedia, Grado en Ingeniería Multimedia, Universidad de Alicante. A partir del paquete de contenidos "Qué es un diagrama de Gantt y cómo aplicarlo".

TEMA 7

La IA en la cadena de valor de Porter

7.1. Sentido de la sesión

Cuando diseñas un proyecto con inteligencia artificial es fácil deslumbrarse con la tecnología y perder de vista lo único que importa al final: *qué valor recibe el cliente y por qué te elige a ti*. Esta sesión te da una herramienta clásica de gestión —la cadena de valor de Porter— para ordenar tus ideas y, sobre todo, para **distinguir lo accesorio de lo principal**. No todo lo que reluce en un proyecto de IA forma parte del núcleo del negocio; mucho es fontanería útil pero secundaria. Aprender a separar una cosa de otra es lo que diferencia un proyecto bien argumentado de una colección de funciones impresionantes sin rumbo.

Idea central

La cadena de valor de Porter separa las actividades que **crean directamente** aquello por lo que el cliente paga (las *primarias* o nucleares) de las que solo las **hacen posibles** (las de *apoyo* o auxiliares). La pregunta clave de toda la sesión es: ¿tu IA está creando el valor, o solo está sosteniendo a quien lo crea?

7.1.1. Qué deberías saber hacer al terminar

- Explicar qué es la cadena de valor de Porter y distinguir sus actividades primarias (nucleares) de las de apoyo (auxiliares).
- Identificar el papel que tradicionalmente han desempeñado las tecnologías de la información dentro de esa cadena.
- Reconocer cuándo la IA repite ese papel auxiliar y cuándo, en cambio, pasa a formar parte del núcleo del valor.
- Situar y **justificar con criterio** cada uso de IA de tu proyecto, separando lo accesorio de lo principal.

7.2. La cadena de valor de Porter

Michael Porter propuso en 1985 mirar la empresa no como un todo indivisible, sino como un conjunto de actividades encadenadas que van añadiendo valor hasta que el cliente paga por el producto o servicio. La diferencia entre el valor total que el cliente reconoce y el coste de generarlo es el *margen*. Porter agrupa esas actividades en dos grandes familias: las **primarias**, que tocan directamente lo que se entrega al cliente, y las de **apoyo**, que sostienen a las primarias sin formar parte por sí mismas del producto.

7.2.1. Actividades primarias: el núcleo

Son cinco y suelen leerse como una secuencia: lo que entra, se transforma, sale, se vende y se atiende después.

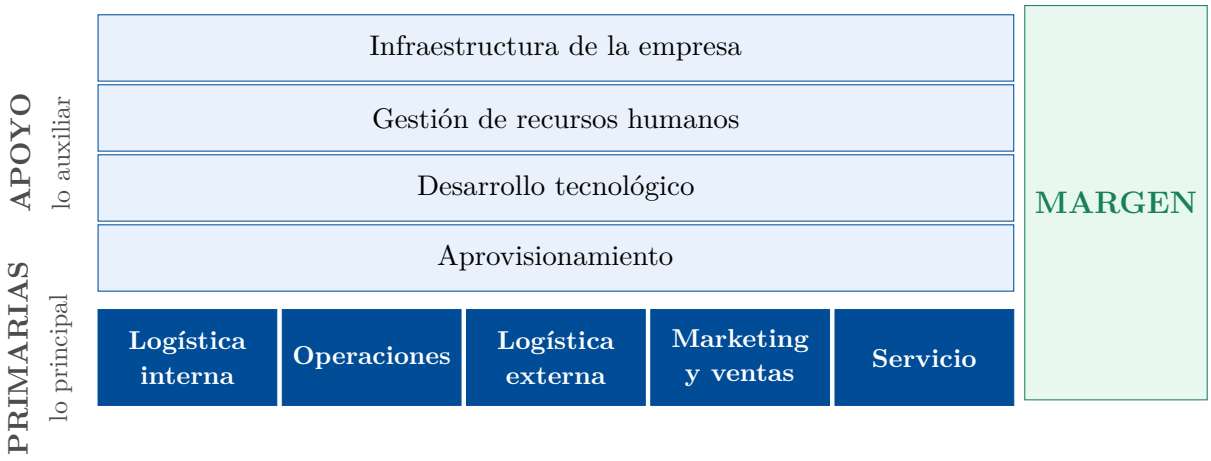


En un producto multimedia —una app, un videojuego, una experiencia interactiva— este encadenamiento es muy tangible: la logística interna recibe los assets, los datos y las dependencias; las operaciones *producen y ejecutan* la experiencia (el juego que se juega, la app que se usa, la pieza interactiva); la logística externa publica y distribuye (la *build*, la tienda, el despliegue); marketing y ventas captan y fidelizan usuarios; y el servicio resuelve incidencias y mantiene el producto con soporte y actualizaciones. Todo lo que el usuario vive y valora ocurre en esta fila.

7.2.2. Actividades de apoyo: lo auxiliar

Por debajo, sosteniéndolas, están cuatro actividades que el cliente normalmente *no ve*: la infraestructura de la empresa (dirección, finanzas, calidad), la gestión de recursos humanos, el desarrollo tecnológico y el aprovisionamiento. Son imprescindibles —sin ellas la cadena se para— pero su papel es habilitar, no protagonizar.

Puestas juntas, las dos familias forman la figura clásica de Porter: cuatro bandas de apoyo que descansan sobre la fila de actividades primarias, y a la derecha el *margen*, que es lo que queda cuando el valor reconocido por el cliente supera al coste de generarlo.



Idea clave

"Auxiliar" no significa "poco importante"; significa que su valor es **indirecto**. Una actividad de apoyo aporta porque mejora a las primarias, no porque el cliente la consuma. Esa es exactamente la frontera que tendrás que vigilar al colocar tu IA.

7.3. El papel tradicional de las tecnologías de la información

Durante décadas, las TI han vivido casi siempre en la fila de apoyo, concretamente en el "desarrollo tecnológico" y en la "infraestructura". Eran la fontanería del negocio: el control de versiones, el sistema de integración y despliegue continuo (CI/CD), el gestor de incidencias, el CRM. Fundamentales para que el equipo funcione, pero accesorios respecto a la experiencia que el cliente compra. Su éxito se medía en eficiencia —más rápido, más barato, menos errores— no en diferenciación.

Ejemplo: TI clásica en un estudio de desarrollo

Un sistema de integración continua (CI/CD) compila el proyecto, ejecuta los tests y publica las *builds*. El usuario no "usa" el *pipeline* ni paga por él; paga por el juego o la app. El CI/CD hace mejor la logística interna, pero vive en la fila de apoyo.

Ya en 1985, Porter y Millar advirtieron de que la información podía hacer algo más que sostener la cadena: podía reconfigurarla y crear ventaja competitiva. Esa intuición es justo la que la IA lleva al extremo.

7.4. Cuando la IA repite el guion: la IA como apoyo

Buena parte de la IA que se aplica hoy hace exactamente lo que hacía la TI tradicional, solo que mejor: optimiza procesos internos. Es IA útil, rentable y perfectamente legítima, pero sigue siendo **accesoria**, porque el cliente no la consume directamente.

Ejemplo: IA que sigue siendo apoyo

Previsión de carga del servidor para dimensionar la infraestructura; optimización del *pipeline* de *build*; clasificación automática de reportes de bugs; un copiloto que ayuda al equipo a escribir código; análisis interno de métricas de uso para el equipo de producto. Todo ello mejora el desarrollo y la gestión, pero el usuario final no lo toca.

Precaución

La novedad tecnológica **no asciende sola de fila**. Que algo use un modelo de lenguaje de última generación no lo convierte en nuclear. Un asistente de soporte que solo consulta las preguntas frecuentes sigue siendo apoyo, por muy moderno que sea por dentro.

7.5. Cuando la IA se sale del guion: la IA como núcleo

La IA deja de ser apoyo en el momento en que **es la propuesta de valor** o produce directamente aquello que el cliente consume y por lo que paga. Entonces ya no está debajo de la cadena: se ha mudado a las actividades primarias —operaciones y servicio— e incluso puede reorganizar la cadena entera a su alrededor.

Ejemplo: IA que pasa a ser nuclear

Un NPC con diálogo generativo que *es* la experiencia del juego; un generador de niveles o de contenido procedural con IA que constituye el producto que se vende; una herramienta de edición de imagen o vídeo cuya función de IA es justo lo que el usuario paga; un compañero conversacional que *es* la app en sí. En todos, si quitas la IA, desaparece o se degrada lo que el usuario venía a buscar.

Síntesis

La misma tecnología puede ser accesoria o nuclear según **dónde** la coloques en la cadena. Lo decisivo no es la sofisticación del modelo, sino si lo que produce llega o no a las manos del cliente como parte del valor.

7.6. Cómo distinguir lo accesorio de lo principal

Esta es la destreza que debes dominar y saber justificar. Tres pruebas, en orden de utilidad:

La prueba de la desaparición

Imagina que retiras la IA. Si el cliente solo nota un cambio en **precio o velocidad**, era *apoyo* (accesorio). Si desaparece o se degrada **el producto mismo**, era *primaria* (nuclear).

- **Prueba de la factura.** ¿El cliente paga —aunque sea indirectamente— por lo que esa IA produce? Si sí, apunta a nuclear.
- **Prueba de la ventaja competitiva.** ¿Esa IA es la razón por la que te eligen a ti y no al competidor? Si sí, está en el corazón del valor.

Uso de IA en un producto multimedia	Actividad de Porter	Naturaleza
Previsión de carga para dimensionar la infraestructura	Desarrollo tecnológico (apoyo)	Accesorio: mejora la gestión interna
Copiloto de código para el equipo	Desarrollo tecnológico (apoyo)	Accesorio
NPC con diálogo generativo que usa el jugador	Operaciones / Servicio (primaria)	Nuclear: el usuario lo usa
Generación procedural de contenido que se vende	Operaciones (primaria)	Nuclear: *es* el producto

El error más común

Presentar como "nuclear" lo que en realidad es un accesorio de eficiencia interna —o, al revés, despreciar como "cosa técnica" algo que sí define el producto. Etiquetar mal la IA lleva a justificarla mal: a vender humo o a infravalorar tu mejor activo. La etiqueta hay que ganársela con la prueba de la desaparición, no con el entusiasmo.

7.7. Cómo justificarlo en vuestro proyecto

No basta con afirmar "nuestra IA es el núcleo". Hay que demostrarlo. Para cada función de IA de tu proyecto, sitúala en la cadena, aplica la prueba de la desaparición y escribe la justificación en pocas líneas.

Mini-actividad

En grupo, listad todas las funciones de IA previstas en vuestro proyecto. Para cada una, completad la ficha de abajo. Al final, mirad cuántas caen en "apoyo" y cuántas en "primaria": si *todas* son de apoyo, vuestra IA es accesoria y conviene revisar la propuesta de valor.

```
FICHA - Situar una funcion de IA en la cadena de valor

Función de IA: _____________________________________________

¿A qué actividad de Porter afecta? ___________________________

¿Apoyo (auxiliar) o primaria (nuclear)? ______________________

Prueba de la desaparicion - si la quito, el cliente nota...
  [ ] solo precio o velocidad -> APOYO / accesorio
  [ ] el producto desaparece/degrada -> PRIMARIA / nuclear

Justificación (3-4 líneas):
_____________________________________________________________
_____________________________________________________________
```

7.8. Aplicación al proyecto del curso

Cada equipo deberá mapear la cadena de valor de su propio proyecto y situar en ella todas sus funciones de IA, distinguiendo con argumentos las que sostienen el trabajo del equipo de las que forman parte de aquello que el usuario recibe. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

7.9. Ideas clave para recordar

- La cadena de valor de Porter separa actividades **primarias** (crean lo que paga el cliente) de actividades de **apoyo** (lo hacen posible).
- Las TI han vivido tradicionalmente en la fila de apoyo: fontanería que mejora la eficiencia, no la propuesta de valor.
- La IA repite ese papel auxiliar cuando optimiza procesos internos que el cliente no consume.
- La IA se vuelve nuclear cuando produce o constituye aquello que el cliente recibe y por lo que paga.
- La etiqueta “accesorio o principal” se gana con la **prueba de la desaparición**, no con la sofisticación del modelo ni con el entusiasmo.
- Justificar un proyecto es demostrar dónde vive cada función de IA en la cadena, no afirmarlo.

Marco de referencia: M. E. Porter, Competitive Advantage (1985); M. E. Porter y V. E. Millar, "How Information Gives You Competitive Advantage", Harvard Business Review (1985). Ejemplos adaptados a productos multimedia para Sistemas Multimedia, Grado en Ingeniería Multimedia.

TEMA 8

Impacto social, ética y responsabilidad del ingeniero

8.1. Sentido de la sesión

Sabéis construir software: diseñar una aplicación, montar un backend, programar la lógica de un videojuego, integrar un motor y publicar un *build*. En esta sesión cambiamos la pregunta. Ya no es si vuestro producto *compila y corre*, sino si puede **salir al mundo**: subirse a Steam, a Google Play o a la App Store, entregarse a un cliente o defenderse ante un jurado, sin pisar los derechos de otra persona y sin poner en riesgo a quien lo use.

Esto no es "la parte de derecho que no toca a los ingenieros". Es justo lo contrario: el ingeniero responde de lo que firma y pone en producción. Una librería con la licencia equivocada, una clave de API embebida en el binario o una política de privacidad que no existe pueden tumbar un lanzamiento tan rápido como un *crash*.

Idea central

Todo producto digital arrastra tres capas que el código no resuelve por sí solo: **de quién es lo que usas** (propiedad intelectual, incluido el código de terceros), **qué haces con los datos de las personas** (privacidad) y **cómo proteges a quien confía en ti** (ciberseguridad). Un producto no está listo para publicar hasta que esas tres capas están resueltas y *documentadas*.

8.1.1. Qué deberías saber hacer al terminar

- Identificar la procedencia y la licencia de cada activo y cada dependencia de tu proyecto (assets, librerías, motor, fuentes, contenido de IA), y razonar su *compatibilidad*.
- Distinguir qué datos personales trata tu aplicación o juego y sobre qué base legal lo hace, con especial cuidado si hay menores.

- Saber qué documentación legal exige una publicación: aviso legal, política de privacidad, política de cookies y condiciones de uso o EULA.
- Aplicar una lista de comprobación de seguridad de paso a producción pensada para clientes que no controlas (apps y juegos).
- Reconocer tu responsabilidad como ingeniero: transparencia (también con la IA), accesibilidad y no manipulación del usuario.

8.1.2. Cómo trabajaremos la sesión

Bloque	De qué trata	Hacia dónde lleva
Propiedad intelectual	De quién es lo que usas (assets y código) y bajo qué licencia	Inventario de activos y dependencias
Privacidad (RGPD)	Qué datos personales tratas y cómo se lo cuentas al usuario	Política de privacidad y aviso legal
Ciberseguridad	Cómo proteges datos y servicio frente a un cliente hostil	Lista de comprobación + EULA/*disclaimers*

Los tres bloques desembocan en el mismo sitio: el **entregable de revisión legal y de cumplimiento de vuestro proyecto**, que preparáis a lo largo de la sesión.

8.2. Propiedad intelectual: de quién es lo que usas

Un proyecto multimedia es un collage: *sprites*, modelos 3D, texturas, *shaders*, efectos de sonido, música, fuentes tipográficas, librerías de terceros, el propio motor y, cada vez más, assets o código generados por IA. **Casi nada de eso es vuestro por defecto.** La propiedad intelectual nace en el momento de la creación, sin registro: el modelo que descargaste tiene autor aunque venga sin marca de agua.

Dos familias que se confunden

Derechos de autor (*copyright*): protegen obras —arte, música, texto, vídeo y también el **código fuente**—. Es lo que más os afecta.

Propiedad industrial: marcas, logotipos y patentes. Aquí entra usar un personaje, un nombre o un logo registrado (Nintendo, Disney, una franquicia) sin permiso, algo muy habitual —y muy peligroso— en proyectos de videojuegos.

8.2.1. Assets: nada se usa “porque estaba en internet”

Que un asset aparezca en un buscador o en un foro no lo hace libre. El criterio sano es el inverso: *cada elemento debe tener una razón por la que puedes usarlo*. Suele ser una de estas.

Vía de uso legítimo	Qué significa en la práctica
Obra propia	La has creado tú (arte, audio, código).
Licencia libre / Creative Commons	El autor permite el uso bajo condiciones (atribución, no comercial, sin derivar...). Hay que leer *cuál*.
Asset store / banco con licencia	Unity Asset Store, marketplaces, stock; respeta su EULA (uso, número de proyectos, reventa).
Dominio público	La protección ha caducado o el autor renunció a ella.

Ejemplo: las licencias Creative Commons no son todas iguales

Un asset **CC BY** puedes usarlo incluso comercialmente *citando al autor*. El mismo con **CC BY-NC** queda fuera de un proyecto comercial. Y **CC BY-ND** te prohíbe modificarlo (recolorear una textura, recortar un *loop* de audio). Las siglas te dicen exactamente qué puedes hacer; aprender a leerlas evita rehacer medio juego antes de lanzarlo.

8.2.2. El código también tiene licencia: el riesgo del *copyleft*

Aquí está lo que distingue a un ingeniero. Cada librería, paquete o fragmento que integráis llega con una licencia, y **no todas son compatibles entre sí ni con cómo queréis distribuir**. La diferencia clave:

- **Permisivas** (MIT, Apache 2.0, BSD): casi todo permitido, normalmente solo exigen conservar el aviso de copyright. Compatibles con producto propietario.
- **Copyleft** (GPL, AGPL): si las enlazas o las distribuyes con tu obra, pueden *obligarte* a liberar tu propio código bajo la misma licencia. La AGPL extiende esa obligación incluso a software que ofreces como servicio en red.

El error más caro

Meter una dependencia **GPL** en el corazón de un juego o app que pensabais vender como cerrado. No es un detalle de abogados: el *copyleft* puede obligaros a publicar todo el código fuente, o a retirar la librería y reescribir esa parte poco antes de lanzar. Revisad las licencias de las dependencias **al elegirlas**, no la semana del *release*.

8.2.3. Fuentes, música y motores

Las fuentes tipográficas tienen licencia (incrustar una de pago en una app puede ser una infracción). La música es de lo que más reclamaciones genera, sobre todo en *trailers* y *streams* —el *Content ID* la detecta— y en juegos. Y los motores y *middleware* (Unity, Unreal y otros) imponen sus propios términos: regalías por encima de cierta facturación, pantalla de créditos obligatoria o restricciones de uso. Leedlos: forman parte de los costes y obligaciones del proyecto.

Precaución: la atribución es una obligación, no un detalle

Muchas licencias libres exigen **citar al autor** de forma concreta (nombre, enlace, licencia). Si la licencia pide atribución y no la pones, el uso deja de ser legítimo aunque el recurso fuera "gratis". Reserva desde el principio un apartado de créditos en el juego o app, y un fichero de licencias de terceros (*third-party notices*).

8.2.4. Contenido generado por IA: terreno nuevo y resbaladizo

La IA generativa —de arte, audio o código— complica el panorama por dos vías. La **autoría**: en muchos ordenamientos (España incluida) lo generado *íntegramente* por una máquina, sin aportación creativa humana, difícilmente se protege como obra; eso afecta a si *vosotros* podéis reclamar derechos sobre el arte o el código de vuestro juego. Y el **origen**: un modelo puede producir algo muy parecido a una obra protegida, a un personaje registrado o a código con licencia restrictiva, y responde quien lo publica —vosotros—, no el modelo.

No hacer

No publiques arte de IA que reproduzca un personaje o el estilo inconfundible de un artista, ni integres código que la IA haya "calcado" de un repositorio con licencia incompatible. Los *fan games* que reproducen IP ajena reciben requerimientos de retirada constantemente; que el asset lo generara una herramienta no te protege.

8.3. Derecho a la privacidad: qué haces con los datos de las personas

En cuanto tu producto tiene cuenta de usuario, ranking online, compras integradas, un chat, telemetría o simplemente analítica de uso, estás tratando **datos personales**. En Europa lo regula el RGPD junto con la ley española (LOPDGDD), y las tiendas lo exigen además como condición para publicar.

Qué es un dato personal

Cualquier información que permita identificar a una persona, directa o indirectamente: nombre, correo, IP, ubicación, un *advertising ID* o identificador de dispositivo, datos de juego asociados a una cuenta. No hace falta el DNI: un identificador de dispositivo persistente ya lo es.

8.3.1. Los principios que debes respetar

El RGPD se resume en unas pocas ideas que conviene grabar:

- **Minimización**: recoge solo los datos que de verdad necesitas. Una demo no necesita la geolocalización precisa "por si acaso".

- **Finalidad**: usa los datos solo para lo que declaraste. Telemetría para corregir *bugs* no es lo mismo que perfilado para publicidad.

- **Licitud**: necesitas una **base legal** (consentimiento, ejecución de contrato —una compra—, interés legítimo).

- **Transparencia**: la persona tiene derecho a saber qué haces antes de dártelos. Eso es la política de privacidad.

- **Seguridad**: debes proteger esos datos (bloque siguiente).

Precaución: los SDK que integras tratan datos por ti

Un SDK de analítica, de *crash reporting* o de anuncios suele enviar identificadores y eventos a un tercero en cuanto arranca la app. Aunque el dato lo recoja el SDK, el responsable ante el usuario eres tú: debes declararlo, elegir bases legales correctas y, cuando aplique, no activarlo hasta tener consentimiento.

8.3.2. Menores: protección reforzada

Si vuestro juego o app puede llegar a menores —y muchos juegos lo hacen— el listón sube. El RGPD da protección reforzada a los datos de los niños, y fija una **edad mínima para consentir** el tratamiento: la norma europea la sitúa en 16 años por defecto y permite rebajarla; **en España son 14 años**. Por debajo, hace falta el consentimiento de quien ejerce la patria potestad. A esto se suma la **clasificación por edades** (PEGI en Europa), que las tiendas exigen y que condiciona qué contenido y qué tratamiento de datos es admisible.

Atención

Diseñar un juego "para todos los públicos" y, a la vez, meter publicidad con perfilado o pedir datos como a un adulto es una combinación que las autoridades vigilan de cerca. Si tu producto puede usarlo un menor, decide su tratamiento de datos *pensando en él*, no a posteriori.

8.3.3. La documentación legal que exige publicar

Antes de publicar un producto digital es necesario preparar, como mínimo, la siguiente documentación:

Documento	Quién lo exige	Qué contiene
Aviso legal	LSSI-CE	Quién es el responsable (titular, contacto, datos identificativos).
Política de privacidad	RGPD/LOPDGDD *y las tiendas*	Qué datos tratas, para qué, base legal, conservación, cesiones (SDK incluidos) y derechos del usuario.
Política de cookies	RGPD + guía AEPD	Para la web/landing: qué cookies usas y cómo aceptarlas o rechazarlas.

Las tiendas piden "ficha de datos"

Google Play (*Data safety*) y la App Store (*privacy nutrition labels*) obligan a declarar qué datos recoge tu app y para qué, de forma coherente con tu política de privacidad. Una declaración falsa o incoherente es motivo de rechazo o retirada. La política de privacidad deja de ser opcional: sin ella, no publicas.

8.3.4. Cookies: el banner que casi todo el mundo hace mal

Para las partes web (landing del juego, panel) conviene seguir al pie de la letra el criterio de la AEPD, de los más sancionados.

Precaución: rechazar tiene que ser tan fácil como aceptar

El banner debe ofrecer **aceptar** y **rechazar** con la *misma visibilidad, tamaño y nivel* (no escondas "Rechazar" en otra pantalla), más la opción de **configurar**. Nada de casillas premarcadas, ni "seguir navegando equivale a aceptar", ni contrastes que empujen a aceptar sin querer.

8.4. Ciberseguridad: proteger antes de publicar

Un producto puede ser legalmente impecable y seguir siendo un peligro si filtra los datos que prometió proteger o si confía en un cliente que no controla. La seguridad no es un módulo final: es una forma de construir desde el diseño.

Principio rector: nunca te fíes del cliente

En una app o un juego, el binario corre en una máquina ajena que puede inspeccionarse, modificarse y decompilarse. Todo lo que llega del cliente —puntuaciones, moneda, compras, peticiones— es **potencialmente falso**. Lo que importa se valida y se decide en el **servidor** (servidor autoritativo). El cliente propone; el servidor dispone.

8.4.1. Lo que cualquier ingeniero debe vigilar

Los fallos más frecuentes y más caros en producto multimedia:

- **Cifrado en tránsito (HTTPS/TLS)**: ninguna comunicación con datos viaja en claro.
- **Gestión de secretos**: claves de API, tokens y credenciales *nunca* en el repositorio. Y, en apps y juegos, *nunca* embebidas en el binario del cliente: se extraen decompilando. Los secretos viven en el servidor.
- **Contraseñas**: nunca en claro; *hash* con una función pensada para ello (por ejemplo, de la familia bcrypt/Argon2) y *salt*.
- **Validar lo que entra**: todo dato del cliente es sospechoso. Validar en servidor evita inyecciones (SQL) y, en partes web, XSS.
- **Servidor autoritativo**: rankings, economía del juego y compras se comprueban en el servidor, no en el cliente.
- **Mínimo privilegio**: cada componente, servicio y persona tiene solo los permisos que necesita.
- **Copias de seguridad probadas**: una copia que nunca has restaurado no es una copia.
- **Límite de peticiones (*rate limiting*)**: protege APIs, *login* y servidores de abuso y de ataques automáticos.

Precaución: APIs abiertas y secretos en el cliente

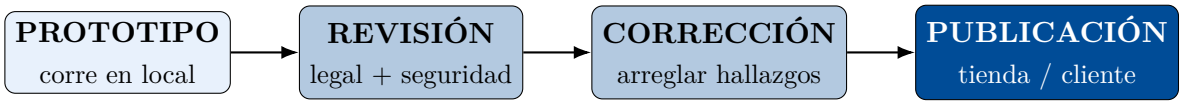
Dos descuidos típicos al integrar *backend*: dejar **CORS** permitiendo cualquier origen y publicar una API **sin límite de peticiones**; y confiar un secreto al cliente. Lo primero convierte un servicio que funciona en clase en uno explotable; lo segundo regala tus claves a quien abra el *build* con un decompilador.

No hacer

No subas a un repositorio —ni embebas en el binario— una clave de API "solo para probar". Los rastreadores encuentran claves filtradas en minutos y el coste llega después. Si ya se escapó una, no basta con borrarla: hay que **revocarla y rotarla**.

8.4.2. De prototipo a publicación

El salto de "corre en mi máquina" a "está en la tienda" tiene un camino reconocible. Recórrelo en orden:



Ejemplo: una lista mínima de "antes de publicar"

Comprueba: ✓ HTTPS activo; ✓ ningún secreto en el repo ni en el binario; ✓ contraseñas *hasheadas*; ✓ entradas validadas en servidor; ✓ rankings/economía con servidor autoritativo; ✓ *backups* probados; ✓ documentación legal enlazada; ✓ clasificación por edades y ficha de datos de la tienda. Casillas que separan un experimento de un producto responsable.

8.5. Ética y responsabilidad: el hilo que une los tres bloques

Las tres capas anteriores son, en el fondo, una sola pregunta ética: *¿estoy tratando con respeto a las personas que usan lo que he construido?* La ley pone el suelo mínimo; la responsabilidad profesional del ingeniero va un paso más allá.

Idea clave: respondes de lo que publicas

Cuando un producto sale a producción, quien lo firma responde: de los assets y el código que usó, de los datos que pidió y de la seguridad de quien confió en su servicio. "No sabía que esa librería era GPL" o "lo generó la IA" no son defensas. La responsabilidad no se delega en una herramienta.

8.5.1. Transparencia con la IA: una obligación que ya está aquí

El Reglamento Europeo de IA (AI Act) introduce obligaciones de transparencia que afectan de lleno a productos multimedia con IA. Las dos más relevantes para vosotros: un sistema que **conversa con personas** —un chatbot, un NPC con diálogo generativo— debe dejar claro que es una IA; y el **contenido generado o manipulado por IA** (arte, voces, vídeo) debe poder identificarse como tal. Estas reglas de transparencia son de aplicación general desde agosto de 2026, de modo que ya están plenamente vigentes: cualquier producto que publiquéis a partir de ahora debe cumplirlas.

Advertencia: los "patrones oscuros" y la monetización predatoria

Diseñar para manipular —una suscripción difícil de cancelar, un "quedan 2 unidades" falso, o mecánicas de monetización agresivas— daña al usuario y a tu reputación. Las **cajas de botín** (*loot boxes*) están reguladas o prohibidas en varios países por su parecido con el juego de azar, sobre todo cuando hay menores. La ética del diseño empieza donde acaba la letra pequeña.

8.6. Actividades

Ejercicio 1 — Auditoría de licencias

Un equipo va a publicar un juego **comercial** para móvil. Este es su inventario. Señalad qué entradas son un problema, por qué, y qué haríais con cada una.

Elemento	Licencia	Uso previsto
Pack de sprites de un foro	CC BY-NC	Personajes del juego
Librería de físicas	MIT	Enlazada en el cliente
Motor de diálogos	GPL-3.0	Enlazado en el cliente
Tipografía de pago	EULA sin *embedding*	Incrustada en el binario
Música de una biblioteca	CC BY	Banda sonora y *trailer*
Retrato generado por IA	—	Portada en la tienda

Solución: los sprites ✗ el *NC* excluye el uso comercial. La librería MIT ✓ basta con conservar el aviso de copyright. El motor GPL ✗ el *copyleft* os obligaría a abrir el código del juego. La tipografía ✗ su licencia no permite incrustarla. La música ▲ es válida, pero solo si ponéis la atribución exacta que exige la licencia, también en el *trailer*. El retrato de IA ▲ probablemente no podréis reclamar derechos sobre él, y respondéis vosotros si se parece demasiado a una obra ajena.

Ejercicio 2 — ¿Es un dato personal?

Decidid para cada elemento si es dato personal a efectos del RGPD y justificadlo en una línea:

1. El alias que el jugador elige libremente (“DragonSlayer99”).
2. La dirección IP desde la que se conecta.
3. El número total de partidas jugadas en el servidor, sin asociar a nadie.
4. El identificador de publicidad del dispositivo.
5. Una captura del ranking con los diez mejores alias.
6. Un informe de *crash* con el modelo de móvil y la versión del sistema.

Solución: 1, 2, 4 y 5 sí —permiten singularizar a una persona, directa o indirectamente, y el alias además va unido a una cuenta—; 3 no, es un dato agregado; 6 depende ▲: si el informe incluye un identificador de instalación o de dispositivo, sí lo es.

Ejercicio 3 — Caza los fallos del banner

La *landing* de un juego muestra esto al entrar: un recuadro con el texto “Usamos cookies para mejorar tu experiencia”, un botón verde grande que dice **ACEPTAR**, un enlace pequeño en gris al pie que dice “más información” y, al final de esa segunda página, unas casillas que se pueden desmarcar pero que vienen *marcadas por defecto*. Mientras tanto, la analítica ya se ha cargado. Enumerad todos los incumplimientos que veáis según el criterio de la AEPD y redactad cómo debería ser el banner.

Ejercicio 4 — Vuestro paso a producción

Aplicad a vuestro propio proyecto la lista mínima de "antes de publicar". Para cada punto anotad ✓ cumplido, ▲ pendiente o ✗ incumplido y, en los dos últimos casos, qué acción concreta lo resolvería y quién la hará. Si vuestro proyecto no trata datos personales o no usa recursos de terceros, escribidlo y justificadlo: la ausencia también es una conclusión válida.

8.7. Aplicación al proyecto del curso

Los principios de propiedad intelectual, privacidad, seguridad y responsabilidad profesional deben aplicarse al producto que desarrolla cada equipo. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

8.8. Ideas clave para recordar

- Todo producto digital tiene tres capas: **propiedad intelectual, privacidad y seguridad**. No está listo para publicar hasta que las tres están resueltas y documentadas.
- Cada asset *y cada dependencia de código* necesita una vía legítima y, si la licencia lo pide, **atribución**. Vigila la compatibilidad de licencias: el **copyleft** (GPL/AGPL) puede obligarte a abrir tu código.
- El contenido de IA no te exime: respondes de lo que publicas, y puede que no puedas reclamar derechos sobre lo generado por la máquina.
- Pide **solo los datos necesarios**, declara para qué (también lo que recogen los SDK) y publica la documentación legal. Con **menores**, protección reforzada (en España, consentimiento desde los 14). Sin política de privacidad no hay tienda.
- Seguridad: **nunca te fíes del cliente**; valida en servidor, no embebas secretos en el binario, *hashea* contraseñas, usa servidor autoritativo, *backups* y *rate limiting*.
- La IA que conversa o genera contenido debe identificarse (transparencia del AI Act); evita patrones oscuros y monetización predatoria.
- La ley es el suelo mínimo; la **responsabilidad del ingeniero** va por encima.

Material docente del Grado en Ingeniería Multimedia, Universidad de Alicante. Estos apuntes tienen finalidad formativa y no constituyen asesoramiento jurídico; ante un proyecto real, consulta las fuentes oficiales (AEPD, BOE, normativa europea) o a un profesional.

TEMA 9

Plan de difusión digital: del producto al mercado

9.1. Sentido de la sesión

Tenéis un producto. Lo habéis pensado como una *startup*: arrancasteis por el porqué (Entregable 1), definisteis a quién servís y qué valor le aportáis con el **Lean Startup Canvas reducido** (Entregable 2), analizasteis riesgos, lo especificasteis, lo diseñasteis y lo construisteis. Funciona, es legal y es seguro (Tema 8). Y ahora... ¿quién se entera de que existe?

Aquí está el último eslabón, el que casi nadie del mundo técnico domina: un buen producto del que nadie habla no llega a nadie. Esta sesión es el **puente entre diseñar el producto como startup y convertirse en empresa para sacarlo al mercado**. Y la buena noticia es que no partís de cero: el plan de difusión no se inventa, *se deriva* de lo que ya decidisteis en el canvas.

Idea central

Un plan de difusión no es una lista de "cosas de marketing" sueltas. Es la continuación lógica del **Lean Startup Canvas reducido**: el **segmento de clientes** elaborado en E2 dice a quién hablar, la **propuesta de valor única** dice qué decir y, en E9, la casilla de **canales** concreta dónde hacerlo. *Cada acción de difusión tiene que poder justificarse señalando la hipótesis del lienzo de la que nace.* Eso es "cerrar el ciclo".

9.1.1. Qué deberías saber hacer al terminar

- Recuperar la propuesta de valor, el segmento y la ventaja diferencial del canvas y convertirlos en la base de un plan de difusión.
- Traducir la propuesta de valor en un **mensaje** y un posicionamiento claros para tu segmento.

- Elegir **medios y canales digitales** adecuados a tu producto y a tu público, y *justificar* cada elección.
- Ordenar las acciones en un **calendario** por fases (pre-lanzamiento, lanzamiento, crecimiento) con recursos realistas.
- Definir un **embudo** y las **métricas** con las que sabrás si cada acción funciona.

9.1.2. Cómo trabajaremos la sesión

Bloque	De qué trata	Hacia dónde lleva
Del canvas al mercado	Reutilizar propuesta de valor, segmento y ventaja como base	Objetivos y mensaje
Medios y canales	Qué medios digitales usar y por qué	Selección justificada de canales
Medición y plan	Embudo, métricas, calendario y recursos	Plan ejecutable y medible

Todo desemboca en el **Entregable 9: plan de difusión digital de vuestro producto**, que preparáis a lo largo de la sesión.

9.2. Del canvas al mercado: reutilizar lo que ya sabéis

El error típico es tratar el marketing como una asignatura aparte que empieza ahora. No: vuestro material de partida ya existe y lo elaborasteis vosotros. Recuperad tres casillas del Lean Startup Canvas reducido del Entregable 2.

Casilla del canvas	Para qué la usas ahora
Segmento de clientes (+ *early adopters*)	Define a **quién** te diriges primero: no a "todo el mundo", sino a los primeros que de verdad tienen el problema.
Propuesta de valor única	Es la materia prima de tu **mensaje**: qué problema resuelves y por qué tu solución importa.
Ventaja diferencial	Lo que te hace difícil de copiar; es tu **ángulo** frente a las alternativas.
Canales	La semilla del plan: dónde llegas a tu segmento. Aquí lo conviertes en acciones concretas.

El principio que ordena todo el tema

Si una acción de difusión no se puede justificar señalando una casilla del canvas ("hacemos esto porque nuestro segmento son *early adopters* técnicos que viven en estas comunidades"), probablemente no deberíais hacerla. La justificación no es un

adorno del entregable: es el criterio para decidir.

Ejemplo: dos productos, dos planes opuestos

Un juego indie de nicho cuyo segmento son jugadores de estrategia veteranos vivirá de una página de Steam con *wishlists*, devlogs y comunidades especializadas (Reddit, Discord). Una app de productividad para estudiantes vivirá de ASO en la tienda, vídeo corto en redes y boca a boca en campus. *Mismo tema, planes distintos*, porque el segmento y la propuesta de valor son distintos. El canvas decide.

9.3. El mensaje: qué dices

Antes de elegir dónde publicar, hay que saber *qué* se dice. El mensaje nace de la propuesta de valor, pero la propuesta de valor del canvas está escrita para vosotros; el mensaje está escrito para el cliente.

De propuesta de valor a posicionamiento

El **posicionamiento** es la frase que queréis que aparezca en la cabeza del cliente: *para [segmento] que [problema], [tu producto] es [categoría] que [beneficio clave], a diferencia de [alternativa].* De ahí salen el *claim* corto, el *pitch* de ascensor y el texto de la ficha de la tienda.

Precaución: vender el beneficio, no la lista de funciones

"Motor de físicas propio y 60 fps" es una característica. "Combate que se siente justo" es un beneficio. El cliente compra el beneficio; la característica solo lo respalda. Traducid cada funcionalidad importante en el beneficio que produce para vuestro segmento.

9.4. Medios y canales digitales: dónde lo dices

Conviene ordenar los medios en tres familias antes de elegir. Es el marco **POE** (*owned, earned, paid*).

Familia	Qué es	Ejemplos digitales
Medios propios (*owned*)	Lo que controlas tú	Landing, ficha de tienda/Steam, blog o devlog, lista de correo, Discord propio.
Medios ganados (*earned*)	Lo que otros dicen de ti	Prensa, *streamers*, reseñas, boca a boca, posts de la comunidad.
Medios pagados (*paid*)	Lo que pagas por difundir	Anuncios en redes, búsqueda, *influencers* patrocinados.

La regla práctica para equipos pequeños: empezad por los **propios** (los construís con tiempo, no con dinero), perseguid los **ganados** (los más creíbles) y usad los **pagados** con cabeza, solo cuando ya sabéis qué mensaje convierte.

9.4.1. Canales digitales habituales en apps y juegos

Canal	Para qué sirve	Fase típica
ASO / ficha de tienda	Que te encuentren al buscar; convertir la visita en instalación	Lanzamiento y continuo
Página de Steam + *wishlists*	Acumular intención de compra antes de salir	Pre-lanzamiento
Landing + lista de correo	Capturar interesados y poder avisarles	Pre-lanzamiento
Comunidad (Discord, Reddit)	Construir núcleo de *early adopters* y *feedback*	Todas
Vídeo (trailer, gameplay, *short*)	Mostrar el valor en segundos	Lanzamiento
Prensa y *streamers* (press kit)	Alcance creíble de terceros	Lanzamiento
Anuncios pagados	Escalar lo que ya funciona	Crecimiento

Precaución: no estés en todos los canales

Un equipo pequeño que se reparte entre seis redes lo hace mal en las seis. Es mejor dominar los dos o tres canales *donde de verdad está tu segmento* que tener presencia simbólica en todos. La elección, otra vez, sale del canvas, no de la moda.

El error más común: "si lo construyo, vendrán"

Lanzar el día 1 sin haber construido antes una audiencia —sin *wishlists*, sin lista de correo, sin comunidad— es el camino más corto al silencio. La difusión **empieza antes del lanzamiento**, no después. El día del lanzamiento solo recoges lo que sembraste semanas o meses antes.

9.5. El embudo y las métricas: cómo sabes si funciona

Cada acción persigue mover a una persona por un **embudo**: desde que te descubre hasta que se queda. Una forma sencilla de pensarlo son las llamadas métricas "pirata" (AARRR).

Etapa	Qué mides (ejemplos)
Adquisición	Alcance, visitas, CTR de un anuncio, *wishlists*.
Activación	Instalaciones, registros, primera partida o primer uso real.
Retención	Vuelven al día siguiente (D1), a la semana (D7).
Ingresos	Conversión a compra, ingreso por usuario.
Recomendación	Compartidos, invitaciones, reseñas.

Idea clave: una acción sin métrica es un gasto, no marketing

Para cada acción del plan debéis poder decir *qué objetivo persigue* y *cómo sabréis si lo cumple*. Si no sabéis medir una acción, no sabréis si repetirla, corregirla o pararla. Un par de métricas bien elegidas valen más que un panel lleno de números que nadie mira.

Ejemplo: justificar y medir una acción

"Publicaremos un devlog mensual en Reddit (canal propio/ganado) porque nuestro segmento son jugadores de estrategia que viven ahí (casilla *segmento*). Objetivo: adquisición. Métrica: *wishlists* nuevas atribuibles a cada post." La acción está **justificada por el canvas** y **tiene métrica**: así debe leerse cada línea de vuestro plan.

9.6. El plan como plan: calendario y recursos

Lo que convierte una lista de buenas ideas en un plan es ponerle **tiempo** y **recursos**. Aquí es donde de verdad pensáis como empresa: una acción cuesta horas y, a veces, dinero, y compite con las demás.



Cada fase tiene su lógica: en **pre-lanzamiento** se siembra (lista de correo, *wishlists*, comunidad, press kit listo); en el **lanzamiento** se concentra el esfuerzo (trailer, prensa, anuncios, presencia en la tienda); en **crecimiento** se retiene y se escala lo que funciona.

El puente a "empresa"

Hablar de coste de adquisición, de presupuesto por canal y de calendario es exactamente el salto de *startup que diseña* a *empresa que vende*. El producto era el modelo de negocio; el plan de difusión es cómo ese modelo se encuentra con el mercado.

9.7. Comunicación responsable: el ciclo se cierra con ética

El plan de difusión hereda lo que aprendisteis en el Tema 8. Comunicar también es una responsabilidad del ingeniero.

Advertencia: ni prometer de más ni jugar sucio

No prometáis lo que el producto no hace: el primer cliente decepcionado deja la peor reseña. Si vuestro producto usa IA, la transparencia del AI Act también vale en el marketing (no presentéis como humano lo que genera una máquina). Y huid de las prácticas que las tiendas y la ley sancionan: reseñas falsas, seguidores comprados, métricas infladas o *urgencia* ficticia. La confianza es vuestro activo más caro de recuperar.

9.8. Actividades

Ejercicio 1 — De la característica al beneficio

Traducid cada característica al beneficio que produce. Elegid vosotros el segmento y hacedlo explícito, porque el beneficio cambia según a quién se lo contéis:

1. "Sincronización en la nube."
2. "Editor con cuarenta filtros."
3. "Modo sin conexión."
4. "Cifrado de extremo a extremo."
5. "Subtítulos generados con IA."

Ejemplo resuelto (1): para un estudiante que alterna móvil y portátil, "sigue tomando notas donde las dejaste, sin acordarte de guardar".

Ejercicio 2 — El canal se justifica desde el canvas

Dos productos y el mismo presupuesto: cero euros. Para cada uno, elegid **tres** canales, ordenadlos por prioridad y escribid junto a cada uno la casilla del canvas que lo justifica.

1. Un simulador de gestión ferroviaria para PC, muy de nicho, cuyo segmento son aficionados adultos que ya juegan a títulos parecidos.
2. Una app que ayuda al alumnado de primero a organizar entregas y exámenes,

con el campus como territorio natural.

Al terminar, tachad el canal que *más os apetecía* usar y que no habéis sabido justificar. Ese es exactamente el que os habría costado más tiempo y os habría dado menos.

Ejercicio 3 — Cada acción, su métrica

Indicad, para cada acción, a qué etapa del embudo AARRR pertenece y con qué métrica sabríais si funciona:

1. Publicar un *devlog* mensual en una comunidad especializada.
2. Enviar un correo a la lista el día del lanzamiento.
3. Añadir un tutorial de treinta segundos en el primer arranque.
4. Ofrecer un código para invitar a un amigo.
5. Rehacer las capturas de la ficha de la tienda.

Solución orientativa: 1 adquisición (*wishlists* o visitas nuevas por publicación); 2 activación (tasa de apertura y de instalación desde el correo); 3 activación y retención (porcentaje que completa el primer uso, y D1); 4 recomendación (invitaciones enviadas y aceptadas); 5 adquisición y conversión (porcentaje de visitas a la ficha que acaban en instalación).

Ejercicio 4 — El plan de una sola página

En equipo, escribid en una página el esqueleto de vuestro plan: segmento y *early adopters*, posicionamiento en una frase, tres canales justificados, tres acciones por fase (pre-lanzamiento, lanzamiento y crecimiento) y una métrica por acción. Al terminar, pasadle el filtro del tema: ¿podéis señalar, para cada línea, la casilla del canvas de la que nace? Lo que no supere el filtro, fuera.

9.9. Aplicación al proyecto del curso

Los conceptos del tema permiten llevar el Lean Startup Canvas reducido desde la definición del producto hasta su encuentro con el mercado. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

9.10. Ideas clave para recordar

- El plan de difusión **se deriva del canvas**: segmento (a quién), propuesta de valor (qué dices), canales (dónde). Cada acción debe poder justificarse señalando una casilla.
- Primero el **mensaje** (beneficio, no lista de funciones) y el posicionamiento; después los canales.

- Ordena los medios en **propios, ganados y pagados**: empieza por los propios, persigue los ganados, paga con cabeza.
- No estés en todos los canales: domina los dos o tres **donde está tu segmento**.
- La difusión **empieza antes del lanzamiento**: "si lo construyo, vendrán" es falso.
- Una acción sin **métrica** es un gasto. Define embudo (adquisición, activación, retención, ingresos, recomendación) y mide.
- Ponle **calendario y recursos**: ahí piensas como empresa. El plan cierra el ciclo del producto.
- Comunica con responsabilidad: sin promesas falsas, con transparencia (también de la IA) y sin métricas o reseñas trucadas.

9.11. Glosario

Términos, siglas y anglicismos que aparecen en esta sesión y que reaparecerán en vuestro entregable. Si una acción de vuestro plan usa una de estas palabras, debéis saber qué significa y por qué la elegís.

AARRR (métricas "pirata")
Modelo del embudo en cinco etapas —Adquisición, Activación, Retención, Ingresos (*Revenue*) y Recomendación (*Referral*)— que ayuda a decidir qué medir en cada momento.

ASO (*App Store Optimization*)
Optimizar la ficha de una app en la tienda (título, palabras clave, icono, capturas) para que aparezca en las búsquedas y convierta visitas en instalaciones. Es el "SEO de las tiendas".

CAC (coste de adquisición de cliente)
Cuánto cuesta de media conseguir un nuevo usuario o cliente, sobre todo a través de medios pagados.

Canal
Cada vía por la que llegas a tu público (ficha de tienda, redes, correo, comunidad, prensa, anuncios...).

Claim
Frase corta y memorable que resume la propuesta de valor; el "eslogan" del producto.

Conversión
Porcentaje de personas que dan el paso buscado (instalar, registrarse, comprar) respecto a las que tuvieron la oportunidad.

CTR (*click-through rate*)
Porcentaje de personas que hacen clic en un enlace o anuncio respecto a las que lo vieron.

Devlog
Publicación periódica (texto o vídeo) que cuenta el avance del desarrollo; construye comunidad y confianza antes del lanzamiento.

Early adopters
Los primeros usuarios dispuestos a probar el producto aunque esté verde; suelen ser el subconjunto del segmento que más sufre el problema.

Embudo (*funnel*)
Representación del camino del usuario, de descubrirte a quedarse; en cada etapa se "cae" parte de la gente, de ahí la forma de embudo.

Go-to-market
La estrategia de salida al mercado: cómo el producto llega a sus primeros clientes. Es, en esencia, el objeto de este tema.

KPI (*key performance indicator*)
Indicador clave elegido para saber si una acción u objetivo va bien. Mejor pocos y bien elegidos que muchos que nadie mira.

Lean Startup Canvas
Lienzo de una página que reúne hipótesis sobre el modelo de negocio. En E2 se trabaja su versión reducida —segmento, problema, solución, propuesta de valor y ventaja especial— y en E9 se incorporan las casillas pospuestas.

LTV (*lifetime value*)
Valor total que aporta un cliente durante toda su relación con el producto. Comparado con el CAC, dice si el negocio es sostenible.

POE (*owned, earned, paid*)
Clasificación de los medios en **propios** (los que controlas: web, ficha, lista de correo), **ganados** (lo que otros dicen de ti: prensa, reseñas, boca a boca) y **pagados** (lo que compras: anuncios).

Posicionamiento
El lugar que quieres ocupar en la mente del cliente; se expresa como "para [segmento] que [problema], [producto] es [categoría] que [beneficio]".

Press kit
Paquete de materiales (descripción, capturas, trailer, logos) que facilitas a prensa y *streamers* para que hablen de tu producto.

Propuesta de valor única (UVP, *unique value proposition*)
La promesa central —qué problema resuelves, para quién y por qué tu solución importa— . Es la casilla del canvas de la que nace el mensaje.

Retención (D1, D7)
: Proporción de usuarios que vuelven al día siguiente (D1) o a la semana (D7) de su primer uso. Mide si el producto "engancha".

Segmento de clientes
: El grupo concreto al que te diriges primero, no "todo el mundo". Define a quién hablas.

SEO (*search engine optimization*)
: Optimizar contenido para aparecer en buscadores como Google. El ASO es su equivalente dentro de las tiendas de apps.

Ventaja diferencial
: Lo que te hace difícil de copiar y te distingue de las alternativas; tu ángulo en el mensaje.

***Wishlist* (lista de deseados)**
: En tiendas como Steam, marca de interés previa al lanzamiento: acumula intención de compra y la tienda avisa a esos usuarios cuando el producto sale.

Material docente del Grado en Ingeniería Multimedia, Universidad de Alicante. Sesión de cierre del ciclo de proyecto: del Lean Startup Canvas reducido (Entregable 2) al plan de salida al mercado (Entregable 9).

TEMA 10

Integración y validación: el proyecto final

10.1. Sentido de la sesión

El Tema 10 no es un tema nuevo: es el momento en que todo lo anterior converge. A lo largo del curso habéis producido nueve entregables —del Círculo Dorado al plan de difusión—. Ahora toca convertirlos en **un único proyecto final**: una compilación *ordenada y cohesionada*, validada con personas reales y defendida en público.

La trampa más habitual es entender el proyecto final como "grapar los nueve entregables". No lo es. Integrar es contar **una sola historia** en la que cada entregable es un capítulo que encaja con los demás; validar es comprobar que esa historia se sostiene fuera del aula; y defender es saber explicar y justificar lo que habéis hecho.

Idea central

Integrar no es concatenar. El proyecto final cuenta **una** historia coherente, no nueve. Y antes de darlo por bueno hay que **validarlo**: hasta ahora casi todo eran hipótesis (el problema, la propuesta de valor, los *early adopters*); la validación las confronta con la realidad. El curso entero ha sido el viaje de una idea a un producto listo para el mercado; este tema lo cierra y lo somete a prueba.

10.1.1. Qué deberías saber hacer al terminar

- Integrar los entregables en un documento con **hilo conductor**, eliminando contradicciones y redundancias.
- Comprobar la **coherencia y la trazabilidad**: que la propuesta de valor, el segmento y las decisiones sean los mismos de principio a fin.
- Elegir y aplicar **técnicas de validación empírica** (entrevistas, test de prototipo, encuestas, evaluación heurística) adecuadas a cada hipótesis.
- Convertir los **hallazgos** de la validación en mejoras concretas (iterar).

- Preparar una **presentación y defensa** que cuente la historia y justifique las decisiones.

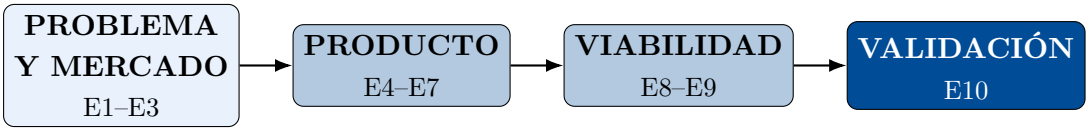
10.1.2. Cómo trabajaremos estas sesiones

Trabajo	De qué trata	Hacia dónde lleva
Integrar ("pegar")	Unir los entregables en un relato único y coherente	Memoria integradora
Validar	Confrontar las hipótesis con personas reales	Evidencia y mejoras
Presentar y defender	Contar la historia y justificar las decisiones	Defensa del proyecto

Todo desemboca en el **Entregable 10: el proyecto final**, que preparáis durante las últimas semanas del curso.

10.2. De los entregables al relato único: integrar

Vuestros nueve entregables no son piezas sueltas: forman el arco completo de un proyecto, del problema al mercado. Integrarlos es reconocer ese arco y contarlo sin costuras.



Cada entregable aporta un capítulo a esa historia, y cada capítulo debe ser coherente con los anteriores.

Entregable	Qué aporta al relato	Con qué debe ser coherente
E1 Golden Circle	El porqué: el propósito y el problema	Es la raíz de todo lo demás
E2 Lean Startup Canvas reducido	Segmento, problema, solución, propuesta de valor y ventaja	Con el porqué de E1
E3 DAFO y riesgos	Capacidades y riesgo clave	Con el canvas y el equipo
E4–E5 Diseño y especificación	Cómo es el producto	Con la propuesta de valor
E6 GANTT	Cuándo y con qué recursos	Con el alcance especificado
E7 IA en la cadena de valor	Dónde aporta valor la IA	Con la propuesta de valor
E8 Aspectos legales	Que se puede publicar	Con el producto real
E9 Plan de difusión	Cómo llega al mercado	Con el segmento del canvas

Idea clave: trazabilidad

Todo en el proyecto debe poder **rastrearse** hasta el porqué. El segmento que diseñasteis (E2) es el mismo al que dirigís la difusión (E9) y con el que validáis (E10). La funcionalidad que especificasteis (E5) responde a la propuesta de valor (E2). Si algo no se puede trazar, sobra o falta una pieza.

Precaución: las contradicciones entre entregables

El fallo típico de integración es que la propuesta de valor "deriva" por el camino: el canvas habla de un público, las especificaciones de otro y la difusión de un tercero. Al integrar, leed los nueve entregables *seguidos* y cazad las incoherencias: un término que cambia de nombre, un segmento que se amplió sin avisar, una funcionalidad que prometía algo que el plan ya no contempla.

No hacer: el "corta y pega"

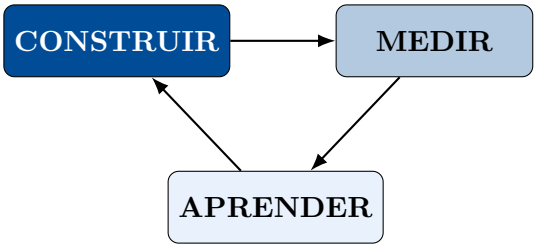
Entregar los nueve documentos uno detrás de otro, con sus portadas y sus repeticiones, no es un proyecto final: es una carpeta. Sin un hilo conductor que los una y sin haber resuelto las contradicciones, el conjunto vale menos que las partes.

10.3. Validación empírica: ¿de verdad resuelve el problema?

Hasta aquí, vuestro proyecto es una colección de decisiones razonadas, pero *no demostradas*. Validar es salir del aula y confrontar las hipótesis más importantes —¿existe el problema?, ¿esta solución lo resuelve?, ¿es usable?— con personas de vuestro segmento real.

El bucle que ordena la validación

La validación no es un examen final: es un **bucle**. Construyes algo (aunque sea un prototipo), lo mides con usuarios y aprendes; con lo aprendido, mejoras y vuelves a empezar. Es el ciclo *Crear–Medir–Aprender* del que nació vuestro Lean Startup Canvas.



10.3.1. Cuatro técnicas y para qué sirve cada una

Técnica	Qué responde	Cuándo usarla
Entrevista	El *porqué*: motivaciones, problemas, contexto (cualitativo)	Al inicio, para entender al usuario en profundidad
Test de prototipo (usabilidad)	Si la gente *sabe usarlo*; dónde se atasca	Con un prototipo o producto manejable
Encuesta	Cuánto/cuántos: medir a escala (cuantitativo)	Cuando ya sabes qué preguntar y quieres números
Evaluación heurística	Problemas de usabilidad según principios de diseño	Revisión experta, sin usuarios, rápida y barata

Lo cualitativo (entrevistas, observación) explica *por qué* pasa algo; lo cuantitativo (encuestas, métricas) dice *cuánto*. Se complementan: primero entiendes, luego mides.

Valida primero lo más arriesgado

No valides todo a la vez. Empieza por la **hipótesis más arriesgada** —la que, si es falsa, hunde el proyecto— que normalmente identificasteis en el canvas (E2) y en el DAFO (E3). Si el problema no existe para tu segmento, da igual lo bonita que sea la interfaz.

Ejemplo: un test de prototipo que enseña más que diez opiniones

Pones tu prototipo delante de cinco personas de tu segmento, les das una tarea ("intenta crear tu primera lista") y **callas**, observando. Cuatro de cinco no encuentran el botón de crear. No necesitas estadística: tienes un problema de usabilidad claro y una mejora evidente. Con **cinco usuarios** suele aflorar la mayoría de los problemas graves de usabilidad.

10.3.2. Evaluación heurística

Cuando no tenéis acceso a usuarios, una persona del equipo puede revisar la interfaz contra una lista de principios de usabilidad reconocidos. Algunos de los más citados (heurísticas de Nielsen):

- Visibilidad del estado del sistema (el usuario sabe qué está pasando).
- Correspondencia con el mundo real (lenguaje y conceptos familiares).
- Control y libertad del usuario (poder deshacer, salir).
- Consistencia y estándares (lo igual se ve y se comporta igual).
- Prevención de errores y mensajes de error claros.
- Reconocer mejor que recordar (opciones visibles, no memorizadas).

Precaución: cómo NO validar

No preguntes "¿usarías esto?" ni "¿te gusta?": la gente es amable y miente sin querer. Pregunta por su **comportamiento pasado** ("¿cómo resolviste esto la última vez?"), no por intenciones futuras. No hagas **preguntas que sugieren la respuesta**. Y no valides solo con amigos y familia: no son tu segmento y te dirán que sí a todo.

Atención: validar para confirmar, no para aprender

El peor error es montar una validación diseñada para *darte la razón*: elegir a quien ya te quiere, contar solo los comentarios positivos, ignorar lo que no encaja. Eso es **sesgo de confirmación** disfrazado de método. Validar es buscar honestamente dónde te equivocas, mientras es barato corregirlo.

10.4. De los hallazgos a las mejoras: iterar

Una validación que no cambia nada no ha servido. El valor está en convertir lo aprendido en decisiones: priorizar los problemas detectados (¿cuáles son graves y frecuentes?) y aplicar las mejoras que estén a vuestro alcance en el tiempo que queda.

Idea clave: el proyecto final muestra que habéis aprendido

Un buen proyecto final no esconde los fallos que encontró: los muestra y explica qué cambió por ellos. "Detectamos que nadie encontraba el botón de crear; lo movimos y rediseñamos el *onboarding*" vale más que un proyecto sin una sola arruga, porque

demuestra que el equipo sabe aprender de la realidad.

10.5. La presentación y la defensa

El curso termina contando la historia en voz alta y defendiéndola. Una buena presentación no recita los entregables: cuenta el viaje.

Estructura de una buena presentación

Una guía que funciona: (1) abrid por el **porqué** (el Círculo Dorado de E1, el problema que merece resolverse); (2) presentad el **producto** y su propuesta de valor; (3) mostradlo funcionando (prototipo o demo); (4) enseñad la **evidencia** de validación y lo que mejorasteis; (5) cerrad con la **salida al mercado** (el plan de difusión de E9). Reutilizad el posicionamiento que ya redactasteis: el mensaje del E9 *es* el guion de vuestro *pitch*.

Defender es justificar, no recitar

En la defensa esperad preguntas del tipo "¿por qué este segmento y no otro?" o "¿cómo sabéis que esto resuelve el problema?". La respuesta no es repetir la diapositiva, sino **justificar la decisión** con lo que hicisteis: el canvas, el DAFO, los resultados de validación. Si todo el proyecto es trazable, la defensa se sostiene sola.

Advertencia: ensayad con tiempo y demo de respaldo

La demo en directo falla en el peor momento. Tened una grabación o capturas de respaldo, ensayad el tiempo (un pase que se va de minutos resta) y repartid quién dice qué. La defensa también se prepara.

10.6. Actividades

Ejercicio 1 — Caza la incoherencia

Estos cinco fragmentos vienen de los entregables de un mismo equipo ficticio. Encontrad las contradicciones y decidid cuál de las versiones en conflicto debe prevalecer en la memoria integradora:

1. **E1:** "Creemos que cocinar en casa debería ser fácil para cualquiera que llega cansado del trabajo."
2. **E2:** "Segmento: estudiantes universitarios de 18 a 22 años con poco presupuesto."
3. **E4:** el storyboard muestra una pantalla de "planificación semanal de menús para toda la familia".
4. **E5:** entre los requisitos no aparece ninguna funcionalidad de planificación

semanal.

5. **E9:** "Canal principal: colaboraciones con *influencers* de nutrición deportiva."

Pista: hay al menos tres derivas —de segmento, de alcance y de mensaje— y una funcionalidad huérfana. Integrar no consiste en quedarse con la versión más bonita, sino en decidir cuál sostiene al resto y corregir las demás.

Ejercicio 2 — ¿Qué técnica usaríais?

Para cada pregunta, elegid la técnica de validación adecuada y decid a cuántas personas acudiríais:

1. "¿La gente entiende para qué sirve nuestra app al abrirla por primera vez?"
2. "¿Cuánta gente de nuestro segmento tiene realmente este problema?"
3. "¿Por qué los usuarios abandonan justo antes de registrarse?"
4. "¿Nuestra interfaz cumple los principios básicos de usabilidad?"

Solución orientativa: 1 test de prototipo —cinco personas bastan—; 2 encuesta —necesitáis número, no matiz—; 3 entrevista —necesitáis el porqué, no el cuánto—; 4 evaluación heurística —una o dos personas del equipo, sin usuarios—.

Ejercicio 3 — Reescribid la entrevista

Estas cuatro preguntas están mal planteadas. Identificad el defecto de cada una y reescribidlas siguiendo la regla del *Mom Test*: preguntad por el comportamiento pasado, no por intenciones futuras.

1. "¿Usarías una app que te ayudara a organizar tus apuntes?"
2. "¿No te resulta incómodo tener los apuntes en carpetas distintas?"
3. "¿Cuánto pagarías por algo así?"
4. "¿Te gusta nuestro diseño?"

Pista: las cuatro piden una promesa o sugieren la respuesta. Cambiadlas por preguntas del tipo "cuéntame la última vez que…" o "¿qué hiciste cuando te pasó?".

Ejercicio 4 — El plan de validación de vuestro proyecto

Escribid vuestro plan *antes* de ejecutarlo: (1) la hipótesis más arriesgada del proyecto, tomada del canvas (E2) o del DAFO (E3); (2) la técnica que la pone a prueba y por qué; (3) a quién acudiréis y cuántas personas; y (4) —el punto que casi todos se saltan— **qué resultado os haría concluir que la hipótesis es falsa**. Si no sabéis responder al punto 4 antes de empezar, vuestra validación solo servirá para daros la razón.

10.7. Aplicación al proyecto del curso

La integración, la validación empírica y la defensa permiten cerrar el recorrido desarrollado durante la asignatura. Las instrucciones de entrega, el formato, la plantilla y los criterios de evaluación se encuentran exclusivamente en el Anexo II de la guía docente.

10.8. Ideas clave para recordar

- Integrar **no es concatenar**: el proyecto final cuenta una sola historia coherente, no nueve.
- Asegura la **trazabilidad**: el mismo segmento y la misma propuesta de valor de principio a fin; caza las contradicciones entre entregables.
- Hasta validar, todo son **hipótesis**. Valida con personas de tu segmento real, no con amigos.
- Elige la técnica según la pregunta: **entrevista** (porqué), **test de prototipo** (usabilidad), **encuesta** (cuánto), **evaluación heurística** (revisión experta).
- Valida primero la **hipótesis más arriesgada**; con cinco usuarios afloran casi todos los problemas graves de usabilidad.
- Validar es aprender, no confirmar: huye del **sesgo de confirmación** y de las preguntas que sugieren la respuesta.
- Convierte los hallazgos en **mejoras**: un buen proyecto muestra qué cambió y por qué.
- **Defender es justificar**: el mensaje del E9 es el guion del *pitch*; ensaya y ten demo de respaldo.

10.9. Glosario

Términos que aparecen en esta sesión y que necesitaréis para el proyecto final y su defensa.

Crear–Medir–Aprender
Bucle del método *Lean Startup* (Tema 2): se crea algo, se mide con usuarios reales y se aprende; con lo aprendido se mejora y se repite.

Defensa
Acto de exponer el proyecto y *justificar* sus decisiones ante preguntas; no es recitar, es razonar lo que se hizo.

Encuesta
Técnica cuantitativa: preguntas estructuradas a muchas personas para obtener números (cuántos, en qué proporción).

Entrevista
Técnica cualitativa: conversación abierta para entender motivaciones, problemas y contexto de un usuario (el "porqué").

Evaluación heurística
Revisión experta de una interfaz contra una lista de principios de usabilidad (p. ej. las heurísticas de Nielsen), sin necesidad de usuarios.

Hipótesis
Suposición aún no demostrada (que el problema existe, que la solución lo resuelve, que el segmento pagará); la validación las pone a prueba.

Iterar
Repetir el ciclo de mejora: aplicar lo aprendido en la validación para corregir el producto y volver a probarlo.

Mom Test
Regla para entrevistar bien: pregunta por el **comportamiento pasado** de la persona, no por lo que cree que haría; así ni tu madre podría engañarte con un "sí" amable.

MVP (producto mínimo viable)
Versión más simple del producto que ya permite aprender de usuarios reales; el mínimo que sirve para validar.

Pitch
Presentación breve y persuasiva del proyecto; aquí, el guion que cuenta la historia del producto.

Prototipo
Versión preliminar y manejable del producto (interactiva o no) que sirve para probar ideas y usabilidad antes de construirlo del todo.

Sesgo de confirmación
Tendencia a buscar y quedarse solo con lo que confirma lo que ya creías; el gran enemigo de una validación honesta.

SUS (*System Usability Scale*)
Cuestionario breve y estándar de diez ítems para puntuar la usabilidad percibida de un sistema.

Test de usabilidad
Observar a usuarios reales realizando tareas con el producto para detectar dónde se atascan; muy revelador con pocos participantes.

Think-aloud (pensar en voz alta)
Técnica en la que el usuario verbaliza lo que piensa mientras usa el producto, para entender sus dudas en el momento.

Trazabilidad
Propiedad de que cada decisión del proyecto puede rastrearse hasta su origen (el porqué, la propuesta de valor), garantizando coherencia.

Validación empírica
: Confrontar las hipótesis del proyecto con datos obtenidos de personas reales, en lugar de darlas por buenas.

Material docente del Grado en Ingeniería Multimedia, Universidad de Alicante. Cierre del ciclo de proyecto: integración de los Entregables 1–9 y validación empírica en el Proyecto final (Entregable 10).